TRAFFICSENSE: AN AI-BASED SMART TRAFFIC MANAGEMENT SYSTEM

A CAPSTONE PROJECT REPORT

Capstone Project ID: 3328F4C8 · Fall Semester 2026-27

Submitted in partial fulfillment of the
requirement for the award of the
Degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

by

C BHARGAV KUMAR (23BCE8195)
ACHALLA AKUL LAKHAN (23BCE8015)
DARSI AMIT (23BCE8288)
MYLAPALLI ABHISHEK RAJ (23BCE9470)

Under the Guidance of

DR. MOHD IRFAN



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VIT-AP UNIVERSITY
AMARAVATI- 522237

NOVEMBER 2026

CERTIFICATE

This is to certify that the Capstone Project work titled **"TRAFFICSENSE: AN AI-BASED SMART TRAFFIC MANAGEMENT SYSTEM"** that is being submitted by **C Bhargav Kumar (23BCE8195), Achalla Akul Lakhan (23BCE8015), Darsi Amit (23BCE8288) and Mylapalli Abhishek Raj (23BCE9470)** is in partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. Mohd Irfan
Guide

The thesis is satisfactory / unsatisfactory

_____ _____

Internal Examiner **External Examiner**

Approved by

_____ _____

PROGRAM CHAIR **DEAN**

Name: _____ Name: _____

B. Tech. CSE *School of Computer Science and Engineering*

ACKNOWLEDGEMENTS

We are deeply grateful to our guide, Dr. Mohd Irfan, School of Computer Science and Engineering, VIT-AP University, for the guidance, the careful questions at every review and the encouragement to report our results as they are, including the ones that did not turn out as we had hoped.

We thank the School of Computer Science and Engineering and VIT-AP University for the capstone programme, the academic environment and the support that made this project possible.

This work stands on the work of others. We thank Yuan, Lai and Liu, the authors of CoLLMLight, whose openly published code let us check our simulation pipeline against theirs; the developers of the CityFlow simulator; IIIT Hyderabad and Intel for the India Driving Dataset, on which our detector was trained and tested; and the developers of Ultralytics YOLOv8, Ollama, Streamlit and the open-source Python libraries listed in Chapter 3, all of which we used free of charge.

Finally, we thank our families and friends for their patience and support during the project.

C Bhargav Kumar
Achalla Akul Lakhan
Darsi Amit
Mylapalli Abhishek Raj

ABSTRACT

Signalised junctions in Indian cities carry a mixed stream of two-wheelers, auto-rickshaws, cars, buses and trucks, and many still run fixed signal plans. Large language models (LLMs) have recently been proposed as traffic-signal agents that can explain their choices and use extra information, such as an approaching ambulance. This project asks whether such agents, running a small model on a laptop GPU, can control signals as well as the best classical method, and how they cope with vehicle counts from a real camera detector instead of exact simulator counts.

We built TrafficSense on the CityFlow simulator and validated it against the CoLLMLight authors' own code, whose baseline results it reproduces to within the simulator's own run-to-run noise; along the way we found a bug in their fixed-time baseline. We built a 2x2 network with an Indian vehicle mix, calibrated its demand, and tuned MaxPressure and a Webster fixed-time plan before any LLM run. The LLM controller, its selection rule and the experiment plan were fixed in advance (pre-registration).

The selected controller, `llama3.2:3b` guided and guarded by MaxPressure, matched MaxPressure's average travel time to within 0.5–0.9% but did not improve on it. With ambulances listed in its prompt, their mean travel time fell from 130.1 s to 88.9 s, close to rule-based preemption (81.2 s), at no measurable cost to other traffic. A YOLOv8n detector trained on the India Driving Dataset reached mAP50 0.563 on held-out drives. Feeding its measured errors into the controllers' counts raised MaxPressure's travel time by 2.3–4.8%, mainly through false detections, while the fixed-time plan was unaffected. An exploratory study found that 3B models make the guided controller much worse, and larger models only bring it back to MaxPressure's level. An LLM agent is therefore best used as a safe layer that adds context MaxPressure cannot use, not as its replacement.

Keywords: traffic signal control, large language models, multi-agent systems, MaxPressure, emergency vehicle priority, YOLOv8, India Driving Dataset, CityFlow, pre-registration.

TABLE OF CONTENTS

Acknowledgements ..... **iii**

Abstract ..... **iv**

List of Tables ..... **viii**

List of Figures ..... **x**

1 Introduction **1**

1.1 Objectives 2

1.1.1 Contributions 2

1.1.2 Scope 3

1.2 Background and Literature Survey 3

1.2.1 Why Indian Roads 3

1.2.2 Fixed-Time Control and Webster's Method 4

1.2.3 Actuated and Adaptive Control 5

1.2.4 MaxPressure 5

1.2.5 Reinforcement Learning 6

1.2.6 LLM-Based Signal Control 6

1.2.7 Vision-Based Traffic Sensing 7

1.2.8 Indian Traffic Studies 7

1.2.9 Summary and Gap 8

1.3 Organization of the Report 8

1.4 Team Contributions 9

2 Proposed System and Methodology **12**

2.1 Proposed System 12

2.1.1 Simulation Layer 13

2.1.2 The Multi-Agent Decision Loop 14

2.1.3 Cooperation Through Neighbour Information 15

2.1.4 Controller Variants 15

2.1.5 Ambulance Handling 16

2.1.6 Camera-Noise Observation Layer 17

2.1.7 Dashboard 17

2.2 Working Methodology 18

2.2.1 Simulation 18

2.2.2 Scenario 21

2.2.3 Perception 25

2.2.4 Experimental Method 28

2.2.5 Pre-registration 30

2.3 Standards 32

2.4 System Details 33

2.4.1 Software Details....33
2.4.2 Hardware Details....34
3 Cost Analysis....35
3.1 List of Components and Their Cost....35
3.2 What a Real Deployment Would Additionally Need (Not Costed)....36
4 Results and Discussions....38
4.1 Baselines and Their Tuning....38
4.2 LLM Benchmark and the Design Rounds....40
4.2.1 Selection....43
4.3 Full Experiments....43
4.4 Ambulances....47
4.5 Model Size (Exploratory)....49
4.6 Camera Errors....51
4.7 Detector....55
4.8 Dashboard....57
4.9 Discussion....59
4.9.1 Why the LLM Agents Match MaxPressure but Do Not Beat It....59
4.9.2 Why the LLM Agents Help Ambulances....61
4.9.3 What the Camera Study Means....61
4.9.4 Relation to the Base Paper....62
4.10 Threats to Validity....62
5 Conclusion and Future Work....64
5.1 Conclusion....64
5.2 Future Work....65
6 Appendix....66
6.1 Configuration....66
6.1.1 Main Configuration....66
6.1.2 Indian 2x2 Scenario....68
6.2 Prompts....69
6.2.1 Prompt v1 (First Round)....69
6.2.2 Prompt v2....69
6.2.3 Prompt v3 (v2 with Two Lines Changed)....69
6.2.4 Guided Prompt (Hybrid Arm)....70
6.2.5 Guided Prompt v3 (hybrid_v3 and the Pre-Registered Guarded Arm)....70
6.2.6 An Example User Message....70
6.2.7 A Logged Decision....71
6.3 Reproducibility....72
6.3.1 How the Results Are Kept Honest....72
6.3.2 Steps....72
6.3.3 A Check from a Fresh Clone....74

6.3.4 Data Not in the Repository....75
6.4 Full Tables....75
6.4.1 Every Baseline Run on the Indian Network....75
6.4.2 MaxPressure Decision-Interval Sweep....77
6.4.3 Validation Against CoLLMLight....78
6.4.4 What the Models Chose in the Benchmark....79
6.4.5 LLM Arm Under Camera Errors....80
6.4.6 Hosted Model Runs (Exploratory)....80
6.5 Abbreviations....81
6.6 Source Code....81
6.6.1 src/trafficsense/control/base.py....82
6.6.2 src/trafficsense/control/maxpressure.py....82
6.6.3 src/trafficsense/control/llm.py....84
6.6.4 src/trafficsense/control/guided.py....87
6.6.5 src/trafficsense/control/preemption.py....90
6.6.6 src/trafficsense/llm/client.py....91
6.6.7 src/trafficsense/llm/prompt.py....94
6.6.8 src/trafficsense/sim/env.py....100
6.6.9 src/trafficsense/sim/observation.py....104
6.6.10 src/trafficsense/perception/errors.py....108
6.6.11 Other source files....112
References....115
Biodata....118

LIST OF TABLES

Table 1.1 Families of signal controllers compared....8
Table 1.2 Team contributions....10
Table 2.1 The LLM controller designs....16
Table 2.2 Our environment against CoLLMLight's code....20
Table 2.3 CoLLMLight's FixedTime against a correct 4-phase cycle....21
Table 2.4 Network and signal parameters of the Indian 2x2 scenario....22
Table 2.5 Vehicle types of the Indian scenario....23
Table 2.6 Capacity of the Indian network, measured in the simulator....24
Table 2.7 Our IDD splits after the class mapping....26
Table 2.8 Camera error parameters per detector class....28
Table 2.9 What was fixed in advance for each study....31
Table 2.10 Standards, formats and conventions followed....32
Table 2.11 Software used, with versions....33
Table 2.12 Hardware....34
Table 3.1 List of components and their cost....35
Table 4.1 Pre-registered selection of MaxPressure's minimum green....39
Table 4.2 All baselines on the Indian 2x2 network....40
Table 4.3 LLM benchmark, all rounds....42
Table 4.4 What the pre-registered rule selected in each round....43
Table 4.5 Full experiments, Indian 2x2 off-peak....44
Table 4.6 Full experiments, Indian 2x2 peak....44
Table 4.7 Full experiments, Jinan 3x4....45
Table 4.8 Paired comparison with MaxPressure....46
Table 4.9 LLM decision statistics....47
Table 4.10 Emergency scenario, normal traffic....48
Table 4.11 Signal-relevant ambulances, pooled over seeds 0–4....48
Table 4.12 Exploratory model-size study....49
Table 4.13 Exploratory: ambulances under each model....51
Table 4.14 Camera errors, Indian 2x2 off-peak, 3 seeds....52
Table 4.15 Camera errors, Indian 2x2 peak, 3 seeds....52
Table 4.16 MaxPressure with uniform detection and no false detections....54
Table 4.17 Post-hoc decomposition of the junction-camera errors....54
Table 4.18 Emergency scenario under junction-camera counts....55

Table 4.19 Detector on the test split, per class.....56
Table 4.20 Miss rate by object size.....57
Table 6.1 Every baseline run on the Indian 2x2 network.....75
Table 6.2 First MaxPressure tuning sweep (decision interval).....77
Table 6.3 Validation runs scored with CoLLMLight's metrics.....78
Table 6.4 Benchmark decisions replayed from the logs.....79
Table 6.5 The pre-registered LLM arm's decisions at each camera level.....80
Table 6.6 Every run of the hosted model (exploratory).....80
Table 6.7 Source files not listed in full, in commit 1d0cfca4a5.....112

LIST OF FIGURES

Figure 1 System architecture....13
Figure 2 One decision of one agent....14
Figure 3 The Indian 2x2 network....22
Figure 4 Demand calibration under MaxPressure....24
Figure 5 Validation accuracy during training....27
Figure 6 LLM benchmark: mean queue of every arm and model....41
Figure 7 Average travel time of every controller....44
Figure 8 Paired ATT difference of the LLM arms from MaxPressure....45
Figure 9 Travel time of the signal-relevant ambulances....49
Figure 10 Exploratory: one hybrid controller with five models....50
Figure 11 Paired ATT change under camera errors....53
Figure 12 Post-hoc decomposition of MaxPressure's ATT change....55
Figure 13 Miss rate by object size on the test split....57
Figure 14 Dashboard, Overview page....58
Figure 15 Dashboard, Live Detection page....58
Figure 16 Dashboard, Network Replay page....59
Figure 17 Dashboard, Results page....59

CHAPTER 1
INTRODUCTION

A traffic signal decides, many times an hour, which movements at a junction may go. Each decision has a cost: a green given to one approach is a red for the others, and every change of phase loses a few seconds to the amber and all-red interval in which no conflicting traffic moves. Over a network of junctions these small decisions add up to the delay that millions of people experience every day. A large part of the urban traffic-engineering literature is therefore about one question: *which phase should be green next, and for how long?*

The classical answers fall into three groups. **Fixed-time** control repeats a timing plan computed in advance from traffic counts; Webster's method [1] is still the textbook way to compute such a plan. **Actuated and adaptive** control changes the plan using detectors, from simple gap-out logic to city-wide systems such as SCATS and SCOOT [2], [3]. **MaxPressure** control [4] gives green, at each decision, to the phase whose waiting vehicles exceed the vehicles queued downstream of them by the most. It needs no training and no demand forecast, and it has a proof of maximum stability. In the last decade, reinforcement learning (RL) has been applied to the same problem [5], and since 2023 large language models (LLMs) have been tried as signal agents [6], [7], [8].

The appeal of an LLM agent is not raw optimisation power. It is that the agent works with information in the form people use: it can be told, in words, what its neighbours are doing, that an ambulance is approaching, or that a road is closed, and it can say why it chose a phase. The base paper of this project, CoLLMLight [8], places one LLM agent at each intersection and lets the agents cooperate by sharing their neighbours' states in the prompt.

1.1 Objectives

The objectives of this project are:

1. Build a multi-agent signal-control system in which each intersection is controlled by a local LLM agent that observes its own state and its neighbours', reasons, and acts, adapted from CoLLMLight to run on a 4 GB laptop GPU.
2. Validate the simulation pipeline against an independent implementation before trusting any result from it.
3. Build an Indian-style test network with a documented source for every parameter, and tune strong classical baselines (MaxPressure, a Webster fixed-time plan) on it **before** the LLM controllers are compared with them.
4. Compare the LLM agents with these baselines under a plan fixed in advance, on normal traffic and on traffic with ambulances.
5. Train a vehicle detector on Indian street images, measure its errors, and test how much those errors hurt each controller when they are injected into its observations.
6. Present the results in a dashboard that reads only the result files.

1.1.1 Contributions

- **A validated simulation pipeline.** Driven the way CoLLMLight's own code drives CityFlow, our environment reproduces their FixedTime and MaxPressure throughput and queue length exactly on all three of their datasets, and their waiting time to within CityFlow's own run-to-run noise (Section 2.2.1). We also found that CoLLMLight's FixedTime baseline never gives north–south traffic a green (Section 2.2.1).
- **A local, guarded LLM controller with honest results.** Six controller designs and four local models were compared under a pre-registered rule; the selected design reaches MaxPressure's performance but not better (Section 4.3). We report this negative result as it is, together with the analysis of why small models misread pressure (Section 4.9.1).

- **A clear benefit for ambulances.** Listing approaching ambulances in the prompt cuts their mean travel time by 31.7% compared with MaxPressure, close to a hand-written preemption rule (Section 4.4).
- **A measured camera-error study.** The detector's own error rates, measured on held-out drives, are injected into the controllers' counts while the simulator stays exact. A post-hoc decomposition separates misses, false detections and class confusion (Section 4.6).
- **Reproducibility by construction.** Every run records its provenance; every table and number in this report is generated from, or checked against, the result files by a script (Section 6.3).

1.1.2 Scope

This is a simulation study. No signal in the field was controlled, no footage from a fixed junction camera was used (the detector is trained and tested on dash-camera images), and the demand on the Indian network is synthetic, calibrated against measured capacity rather than counted at a real junction. CoLLMLight's fine-tuning stages (reasoning refinement and policy optimisation) are not reproduced: all models are used as released, without training, because the project's question is what an off-the-shelf local model can do.

1.2 Background and Literature Survey

This section first explains what makes Indian urban traffic different. It then follows the history of signal control in the order the methods are used later as baselines and as the controller under test: fixed-time plans, actuated and adaptive systems, MaxPressure, reinforcement learning, and LLM agents. It closes with vision-based traffic sensing and studies of Indian traffic, which motivate the perception part and the scenario.

1.2.1 Why Indian Roads

Indian urban traffic differs from the traffic for which most signal-control methods were developed in three ways that matter here.

1. **The vehicle mix is heterogeneous.** Two-wheelers and three-wheelers make up most of the stream. In the modal-share data we use for the simulation (Chennai, as reported in [9]), motorised two-wheelers are 45.1% of vehicles after normalisation, and auto-rickshaws 17.6%. Vehicles differ widely in length, acceleration and the space they occupy, which changes how fast a queue discharges [10], [11].
2. **Lane discipline is weak.** Two-wheelers filter between larger vehicles and queue several abreast at the stop line [12]. Loop detectors that count vehicles per lane work poorly; cameras are the natural sensor.
3. **Many junctions still run fixed plans or manual control**, and emergency vehicles are delayed at signals with no priority system.

These points shape the project. The controllers are tested on a network with an Indian vehicle mix; the counts they receive are, in one experiment, degraded with the errors of a real detector trained on Indian street images; and emergency vehicles are part of the evaluation. They also limit it: the simulator we use, CityFlow [13], is lane-based and cannot reproduce lane-free driving. This is stated wherever it matters (Section 2.2.1, Section 4.10).

1.2.2 Fixed-Time Control and Webster's Method

A fixed-time controller repeats a cycle of phases with fixed green times. It is cheap, needs no detectors and is predictable, which is why it remains common. Its weakness is equally simple: the plan cannot react to the traffic actually present, so green is wasted on empty approaches while queues grow elsewhere.

Webster [1] gave the standard way to compute a good fixed plan. For each phase i the flow ratio $y_i = q_i / s_i$ compares the demand q_i on its critical lane with the lane's saturation flow s_i (the rate at which a queue discharges during green). With $Y = \sum_i y_i$ and L the total lost time per cycle, the delay-minimising cycle length is

$$C_0 = \frac{1.5\,L + 5}{1 - Y},$$

and the effective green is shared between phases in proportion to y_i. The formula assumes steady, random arrivals; when demand changes during the period, a plan computed from average counts is no longer optimal. We use Webster's method as a strong fixed-time baseline and give it an unfair advantage: its plan is computed from the exact demand of the simulated hour (Section 4.1).

1.2.3 Actuated and Adaptive Control

Actuated controllers extend or end a green using detectors at the stop line, within minimum and maximum green limits [14]. They respond to the current queue but decide each junction on its own. Adaptive systems coordinate many junctions. SCATS [2] selects and adjusts plans from detector data at the stop line; SCOOT [3] models the traffic in each link from upstream detectors and adjusts splits, cycle times and offsets continuously. Both depend on dense, well-maintained detector networks and on lane-based traffic models. These assumptions are hard to meet on Indian roads, where loop detectors are rare and lanes are not followed (Section 1.2.8).

1.2.4 MaxPressure

MaxPressure comes from the theory of scheduling in queueing networks. Tassiulas and Ephremides [15] showed that serving, at every step, the set of queues with the largest "back-pressure" (the difference between a queue and the queue downstream of it) keeps a network stable whenever any policy can. Varaiya [4] carried this result to traffic signals. For a phase p that serves movements (l, m) from incoming lane l to outgoing lane m, the pressure is

$$P(p) = \sum_{(l,m) \in p} (x_l - x_m),$$

where x is the number of queued vehicles. At each decision the controller gives green to the phase with the highest pressure. MaxPressure is local (each junction needs only its own and its immediate downstream queues), needs no demand forecast and no training, and maximises the throughput the network can sustain. It does not by itself choose how often to decide or how short a green may be; those timing parameters matter

in practice, and we tune them (Section 4.1). MaxPressure is the main baseline of this project, and the hybrid LLM controllers use its ranking of phases.

1.2.5 Reinforcement Learning

RL learns a policy that maps an observed traffic state to a phase by trial and error in a simulator, rewarded for short queues or delays. Early work used tabular methods on single junctions [16]; deep RL followed [17]. IntelliLight [18] tested deep RL on real traffic data. PressLight [19] used MaxPressure's pressure as the reward and coordinated an arterial; CoLight [20] used graph attention so that each agent weighs information from its neighbours; MPLight [21] scaled pressure-based RL to thousands of signals by sharing parameters. These methods were developed on CityFlow [13], the simulator used here, and on the same Jinan and Hangzhou datasets. The survey by Wei et al. [5] summarises the field.

RL controllers can outperform hand-designed rules in simulation, but they need long training per network, their decisions are hard to explain, and they may not transfer to traffic unlike the training traffic. These limits motivated the move to language models.

1.2.6 LLM-Based Signal Control

LLMs bring general knowledge and the ability to follow instructions written in natural language. LLMLight [6] described the traffic state of one intersection in a prompt and asked an LLM to choose a phase with step-by-step reasoning; it also fine-tuned a model for the task. LA-Light [7] placed an LLM over a set of tools (perception, decision support, evaluation) so that the model could handle rare situations such as emergencies or sensor faults, while conventional algorithms handled normal operation.

CoLLMLight [8], the base paper of this project, argues that independent LLM agents ignore the interaction between neighbouring intersections. It gives each intersection an LLM agent with a spatiotemporal-aware cooperative reasoning module that analyses the neighbours' states and produces cooperative suggestions, runs this reasoning in an asynchronous decision architecture so that multi-step reasoning does not delay real-time decisions, and trains the model with a cost-aware cooperation optimisation: adaptive reasoning optimisation teaches the model to reason concisely, and

policy refinement rewards both good decisions and short reasoning. Their code (which we use for validation, Section 2.2.1) runs on CityFlow with the Jinan, Hangzhou, New York and Synthetic networks.

What we take from CoLLMLight is the multi-agent structure (one agent per intersection), the cooperation through neighbour information in each agent's prompt, and its datasets and simulator. What we do not take is the trained model and the fine-tuning: our agents use off-the-shelf models of 3–4 B parameters through Ollama [22] on a 4 GB GPU, because the question here is what a deployable local model can do without training. A second difference is in evaluation: we compare with tuned baselines under a plan fixed in advance (Section 2.2.5).

1.2.7 Vision-Based Traffic Sensing

Cameras can count, classify and track vehicles without cutting the road surface, which makes them the usual sensor for adaptive control in mixed traffic [23]. One-stage detectors of the YOLO family [24] are fast enough for real-time use on modest hardware; YOLOv8 [25] is the version used here. Detectors are evaluated with mean average precision (mAP) over intersection-over-union thresholds, and their accuracy falls sharply for small objects, which COCO [26] measures with its small / medium / large size buckets. For signal control the relevant quantity is not mAP but the error in the **counts** a controller receives: missed vehicles, invented vehicles and wrong classes. Visual sensing has also been proposed to detect emergency vehicles and give them priority [27]. Our perception study measures these count errors on Indian images and then measures their effect on control, a link that the detection literature usually does not test.

1.2.8 Indian Traffic Studies

The India Driving Dataset (IDD) [28] was collected to study autonomous driving in unstructured Indian traffic. It contains classes absent from most Western datasets, such as auto-rickshaws, and many partly occluded, densely packed road users. Traffic-engineering studies of Indian roads show that heterogeneous vehicles change capacity and passenger-car equivalents with road width [10], that microsimulation models must be calibrated for non-lane-based traffic at signals [12], and the Indian Highway Capacity Manual [11] provides capacity and level-of-service methods developed for these

conditions. The vehicle mix and dimensions of our scenario come from Indian sources [9], [29] (Section 2.2.2).

1.2.9 Summary and Gap

Table 1.1 Families of signal controllers compared

How the families of signal controllers compare on the properties this project cares about.

Method	Reacts to traffic	Coordinates neighbours	Needs training	Explains decisions	Uses extra context, e.g. an ambulance
Fixed time, Webster	no	through offsets only	no	the plan is fixed	no
Actuated, SCATS, SCOOT	yes	SCATS and SCOOT do	calibration	partly	through special modules
MaxPressure	yes	implicitly, through downstream queues	no	the pressures	no
RL: PressLight, CoLight, MPLight	yes	CoLight does	yes, per network	no	only if in the state
LLM agents: LLMLight, CoLLMLight	yes	CoLLMLight does	optional fine-tuning	yes, in words	yes, in the prompt

Published LLM controllers are usually evaluated with large hosted or fine-tuned models, against baselines whose settings are not tuned for the test network, and with exact simulator counts as input. Three questions are therefore open, and this project addresses them: whether a small local model, without fine-tuning, matches a tuned MaxPressure under a fixed evaluation plan; whether the LLM's ability to use extra context helps in a concrete case (ambulances); and how realistic camera errors change the comparison.

1.3 Organization of the Report

The report follows the college's capstone thesis format.

- **Chapter 1** states the objectives, reviews the background and literature, from fixed-time plans to LLM agents and vision-based traffic sensing, and lists each team member's contribution.

- **Chapter 2** describes the proposed system (architecture, the multi-agent decision loop, the controller designs, ambulance handling, the camera-noise layer and the dashboard) and the working methodology: the simulator and its validation, the Indian scenario, the perception part, the experimental method and the pre-registration. It closes with the standards followed and the software and hardware used.
- **Chapter 3** lists every tool and component with its licence and cost.
- **Chapter 4** presents the results (baselines, the LLM benchmark and design rounds, the full experiments, ambulances, the exploratory model-size study, camera errors, the detector and the dashboard) and discusses them, including the threats to validity.
- **Chapter 5** concludes and lists future work.
- **Chapter 6**, the appendix, gives the configuration, the prompts, the reproduction steps, the full tables, the abbreviations and listings of the key source files.

The references, the team's biodata and the note on the accompanying CD follow.

1.4 Team Contributions

Table 1.2 gives each member's role and the parts of the project each was responsible for. Every entry names a file or feature that exists in the project repository; the numbers and results of the project are given only in the later chapters, from the result files.

Table 1.2 Team contributions

Paths are under `src/trafficsense/` unless they start with `scripts/`, `tests/`, `configs/` or `outputs/`.

Name	Register No.	Role	Contribution
C Bhargav Kumar	23BCE8195	Computer vision and machine learning lead	• Chose the detector, YOLOv8n, and set up its training on the 4 GB GPU (batch size from a probe run, mixed precision off): scripts/train_detector.py. • Defined the mapping of IDD's labels to our seven classes (perception/idd.py) and built the inference pipeline used by the evaluation and the Live Detection page (perception/detector.py). • Ran the detector evaluation whose measured errors are the inputs of the camera error model: scripts/eval_detector.py, outputs/perception/.
Achalla Akul Lakhan	23BCE8015	System architect and multi-agent lead	• Overall architecture: simulator, observation layer, controllers, result files, dashboard (Figure 1). • The cooperative LLM controller and its guard: control/llm.py, control/guided.py. • The prompts (v1 to v3, guided) and the JSON-schema structured output: llm/prompt.py, llm/client.py. • CityFlow integration and the Indian 2x2 network and flows: sim/env.py, sim/roadnet.py, scripts/generate_indian_roadnet.py, scripts/generate_indian_flow.py. • Ambulance priority in the LLM arms and the preemption baseline: sim/emergency.py, control/preemption.py.
Darsi Amit	23BCE8288	Frontend and dashboard lead	• The Streamlit dashboard and its four pages: dashboard/app.py, dashboard/screens/. • Theme and charts: dashboard/theme.py, dashboard/ui.py, dashboard/views/results.py. • The Live Detection and Network Replay views: dashboard/views/detection.py, dashboard/views/replay.py. • Reading only from outputs/, with a source chip on every number: dashboard/data.py, tests/test_dashboard.py.
Mylapalli Abhishek Raj	23BCE9470	Data engineering and integration lead	• Dataset research and acquisition: CoLLMLight's road networks and flows (scripts/fetch_collmlight_data.sh) and IDD. • Conversion of IDD into YOLO format and the drive-level splits: scripts/prepare_idd.py, scripts/resize_idd.py. • Integration and configuration across modules: configs/default.yaml, config.py, experiment.py, scripts/run_experiment.py. • The pytest suite, with test doubles for LLM calls that exist only inside tests/. • README, git and GitHub maintenance, environment and requirements: pyproject.toml, requirements-lock.txt.

The project was developed with the help of an AI coding assistant (Claude Code), working under the team's direction: the team decided what to build and how to evaluate it, reviewed the code and the text, ran the experiments and checked the results, and is responsible for all of the work presented in this report.

CHAPTER 2
PROPOSED SYSTEM AND METHODOLOGY

This chapter describes what TrafficSense is made of and how a decision is made at one intersection (Section 2.1), how the work was carried out and checked (Section 2.2), the standards followed (Section 2.3) and the software and hardware used (Section 2.4). The design follows from three rules the project set itself before any code was written, because the results will be examined and must be trusted:

1. **No invented data in the results path.** Every number in a result file is measured from the simulator or from a real model's answer. Test doubles exist only inside the test suite.
2. **Fail loudly.** If the LLM server is down, an answer cannot be parsed, a file is missing or a setting is invalid, the run stops with an error or, where a fallback is part of the design, the fallback is counted and reported. Nothing silently falls back to a default.
3. **Every result records where it came from**: controller, model as reported by the server, seed, scenario, run length, git commit and time, and for LLM runs the number of calls, failures and the latency.

2.1 Proposed System

Figure 1 shows the components. The **CityFlow engine** simulates every vehicle and is the only place where metrics are measured. An **environment wrapper** around it parses the road network, enforces signal timing and computes metrics. Controllers do not read the engine directly; they read lane counts through an **observation layer**, which returns either the exact counts or, in the camera experiment, counts degraded by the detector's measured errors (Section 2.1.6). A **controller** decides the next phase for each intersection. The LLM controllers are built on top of **MaxPressure's ranking** of phases, ask an **LLM agent** per intersection, and pass its answer through a **guard** and the timing rules. Each run writes its results to `outputs/`, from which the dashboard, the documentation and this report are generated.

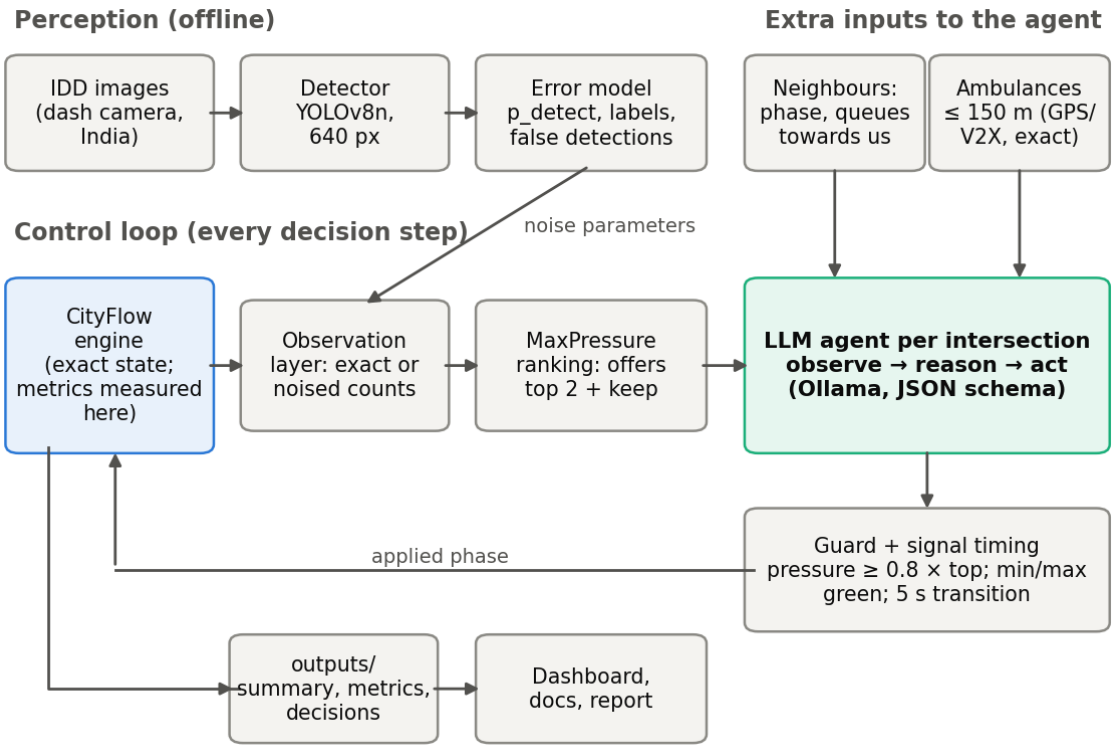


Figure 1 System architecture

The simulator holds the exact state; controllers see it only through the observation layer, which can add camera errors measured offline. Ambulance positions reach the agent through a separate channel and are never noised.

The code is a Python package, `trafficsense`, with sub-packages for the simulator wrapper (`sim`), the controllers (`control`), the LLM clients and prompts (`llm`), perception (`perception`), the dashboard (`dashboard`) and the report tooling (`report`). Settings and paths come from YAML files in `configs/`; no path is hard-coded.

2.1.1 Simulation Layer

Road network and phases. The wrapper reads a CityFlow road network and builds, for each intersection, its incoming lanes, the movements (straight, left, right) and which movements each phase serves. In the CoLLMLight networks and in ours, phase 0 is a 5 s transition phase in which only the kerb-side free turn may move, and phases 1–8 are green phases. The transition phase is detected from the network, not assumed.

Signal timing. CityFlow switches a signal instantly when told to, so the wrapper inserts the transition phase on every change and enforces a minimum green of 10 s and a

maximum green of 60 s for every controller. The tuned MaxPressure and all LLM controllers use their own, longer minimum green of 15 s (Section 4.1).

Metrics are measured from the engine at every second (Section 2.2.4).

2.1.2 The Multi-Agent Decision Loop

Each signalised intersection is controlled by its own agent. Every 5 s of simulated time, every intersection whose current green has lasted at least the minimum green is asked for its next phase. One decision goes through five steps (Figure 2):

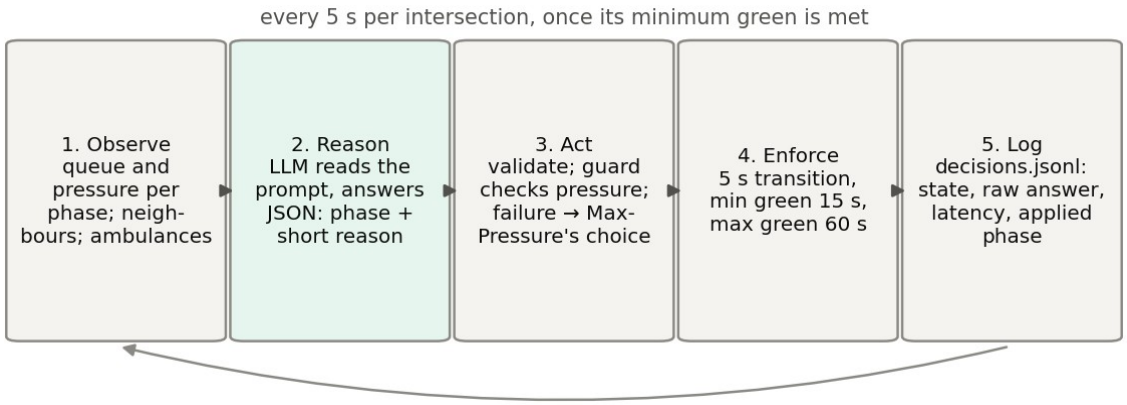


Figure 2 One decision of one agent

The simulator is paused during the LLM call, so slow answers cost wall-clock time, not simulated traffic.

1. **Observe.** The agent receives a short text describing its own intersection and its neighbours (Section 2.1.3). For each phase it may choose, it is given the phase's **pressure** (stopped vehicles the phase would serve minus stopped vehicles on their exit roads) and its **queue** (stopped vehicles it would serve), so that it needs no arithmetic. The pre-registered controller's prompts averaged 277 tokens in the benchmark.
2. **Reason.** The prompt is sent to a local model through Ollama's native `/api/chat` endpoint with a JSON schema that restricts the answer to the allowed phase numbers and a short reason. Sampling uses temperature 0 and a fixed seed, a context of 2048 tokens, at most 128 answer tokens and a timeout of 60 s; the model's "thinking" mode is switched off.
3. **Act.** The answer is parsed and checked: the phase must be one of those offered and the reason must be at most 15 words. A failure (timeout, invalid JSON, a

phase not offered, a reason that is too long) is counted by type. In the hybrid controllers, a failed call applies MaxPressure's choice for that decision and is reported as such.

4. **Enforce.** The chosen phase is requested from the wrapper, which inserts the transition and applies the green limits.
5. **Log.** The decision is written to the run's `decisions.jsonl`: the state the agent saw, MaxPressure's ranking and pressures, the raw answer, its latency and token counts, whether the guard intervened, and the phase applied.

All agents share one GPU, so the calls of one decision round run one after another. The pre-registered selection rule therefore sets the decision interval to the shortest multiple of the transition time that is at least the model's 95th-percentile latency times the number of intersections (Section 4.2.1). The simulator waits during calls; slow models make a run take longer in wall-clock time but do not change the simulated traffic.

2.1.3 Cooperation Through Neighbour Information

Following CoLLMLight, cooperation is through the prompt. For each neighbouring intersection, the agent is told the neighbour's current phase, how many vehicles are stopped on the neighbour's approaches heading towards this intersection and how many of those have green, and how many vehicles are stopped on this intersection's road into the neighbour. The agent can thus see a platoon about to be released towards it, or a neighbour whose approach from this intersection is full. An example prompt, rendered by the controller's own code from a real simulator state, is in Section 6.2.

2.1.4 Controller Variants

The project compared six LLM controller designs ("arms"). They differ in how much the LLM decides and in the prompt version (Section 6.2).

Table 2.1 The LLM controller designs

The pre-registered final design round compared the last three.

Arm	Controller	What the LLM chooses	Prompt
v1	`llm_v1`	any green phase	v1: per-movement counts, vehicle types, neighbours
v2	`llm_cooperative`	any green phase	v2: shorter; pressure and queue per phase
hybrid	`llm_guided_mp`	MaxPressure's top 2 or keep current	guided
v3	`llm_v3`	any green phase	v3 = v2 with two lines fixed
hybrid_v3	`llm_guided_mp_v3`	MaxPressure's top 2 or keep current	guided_v3
guarded_v3	`llm_guarded_mp`	as hybrid_v3, then the guard	guided_v3

Plain arms (v1, v2, v3) let the LLM choose any green phase. **Hybrid arms** let MaxPressure propose and the LLM decide: MaxPressure ranks the phases by pressure, and the LLM is offered MaxPressure's two best phases and keeping the current phase. The **guarded arm** adds a check: the LLM's choice is applied only if its pressure is close enough to the best phase's,

$$P(\text{LLM's phase}) \geq P_{\text{top}} - (1 - r)\,|P_{\text{top}}|,$$

with $r = 0.8$ in the configuration, i.e. within 20% of the top pressure. The absolute value keeps the top phase acceptable when every pressure is negative. Otherwise MaxPressure's choice is applied and the override is counted. The guard was added because the small models in the earlier rounds preferred low-pressure phases (Section 4.2); it limits how much harm such a choice can do.

2.1.5 Ambulance Handling

In the emergency scenarios, ambulances are part of the traffic (Section 2.2.2). An ambulance on an incoming lane within 150 m of the stop line is detected. For the hybrid and guarded arms:

- the prompt lists each detected ambulance with its movement, its distance to the stop line and the phases that serve it, and the system prompt adds one sentence: "Ambulances, if listed, must get through quickly: give green to a phase that serves them.";

- if no offered phase serves the ambulance, the best-ranked phase that does is added to the options;
- in the guarded arm, a phase that serves a listed ambulance is exempt from the guard, so the guard cannot block priority.

The non-LLM competitor, `preemption_mp`, is MaxPressure with a rule: when an ambulance is detected in the same way, the phase that serves it with the highest pressure is given green, within the same minimum and maximum green. Ambulance positions are assumed to come from a separate channel (GPS or V2X) and are not passed through the camera-noise layer.

2.1.6 Camera-Noise Observation Layer

To test controllers on realistic counts without changing the traffic, the observation layer can replace the engine's lane counts with noisy ones. Every time a controller observes, a fresh snapshot is drawn for the whole network: each vehicle is detected with a class-dependent probability, a detected vehicle may be given another class (a vehicle labelled as a pedestrian is lost from the count), and each road gains a random number of phantom stopped vehicles. All the rates come from the detector's measured errors (Section 2.2.3). The simulator itself stays exact, so every metric is still measured from the true traffic.

2.1.7 Dashboard

A Streamlit dashboard presents the results. It reads only files in `outputs/` (and the detector weights), and every chart and number carries a source chip naming the file it came from. Its four pages show the headline result, live detection on video with the trained detector, a side-by-side replay of two controllers on the same traffic, and the result tables; screenshots of all four are in Section 4.8. A test renders every page and fails if any number shown does not appear in the files the page read.

2.2 Working Methodology

The work proceeded in steps, and each step was checked before the next one relied on it:

1. **Simulation** (Section 2.2.1). The simulation pipeline was validated against the CoLLMLight authors' own code before any result was trusted.
2. **Scenario** (Section 2.2.2). An Indian-style network was built, every parameter with its source, and its demand levels were calibrated by a rule fixed in advance. The classical baselines were then tuned on it (Section 4.1).
3. **Perception** (Section 2.2.3). A detector was trained on Indian street images and its errors were measured on held-out drives and turned into an error model.
4. **Experimental method** (Section 2.2.4). Scenarios, seeds, metrics and the statistical treatment are the same for every controller.
5. **Pre-registration** (Section 2.2.5). The plan and decision rule of every study were committed before its runs; the few additions made later are stated and labelled.

2.2.1 Simulation

A controller comparison is only as good as the simulator and the scenario behind it. This section checks that our simulation pipeline gives the same answers as an independent implementation; Section 2.2.2 then describes the Indian test network and where each of its parameters comes from. How the classical baselines were tuned on it, before the LLM controllers were tested, is reported with the results (Section 4.1).

CityFlow and what it can represent

CityFlow [13] is a microscopic traffic simulator designed for signal-control research: it moves every vehicle every second with a car-following model, is fast enough for hour-long runs on a laptop, and is the simulator of CoLLMLight and most RL work. It is **lane-based**: every vehicle drives in one lane and follows the vehicle in front. It cannot represent vehicles straddling lanes, two-wheelers filtering between cars or queuing several abreast, and it ignores vehicle width in its dynamics. Our Indian scenario therefore models the Indian **vehicle mix** (sizes, headways, accelerations) but not Indian **driving behaviour**. Its results describe "a lane-based simulation with an Indian vehicle

mix", and capacities measured in it are probably lower in vehicles per hour than on a real road with the same mix.

We run CityFlow with lane changing switched off. With lane changing on, CityFlow is not exactly repeatable: the same run, repeated with identical inputs, gives slightly different travel times. With it off, the same phase sequence always gives exactly the same run, which we verified by re-running every deterministic experiment from a fresh copy of the repository (Section 6.3).

Validation against CoLLMLight's code

Before trusting any comparison, we checked our environment wrapper, signal timing and metrics against an independent implementation: the CoLLMLight authors' own code (commit `ce204a9`).

Method. Their FixedTime and MaxPressure baselines were run unmodified in a separate Python environment with their pinned library versions. Their configuration imports every agent class, including the neural and LLM ones, so those imports were replaced by inert stubs; FixedTime and MaxPressure never call them. Our own environment was then driven exactly as theirs: the same 4 phases, one decision every 30 s, a 5 s transition before every switch, no minimum or maximum green, lane changing on, and **the same CityFlow seed** their run drew. Their metric definitions differ from ours (they measure travel time per intersection approach rather than per trip, sum queues over the network, and sample at their decision times), so every matched run was scored twice: with our metrics and with a line-by-line reimplementation of theirs.

Result. Table 2.2 compares one run of each implementation under their metric definitions. Throughput, queue length and waiting time are identical floating-point numbers for both controllers on all three datasets. Average travel time under their definition is identical in all but one of the six runs; there, and in CityFlow's own average travel time, the differences are in the sixth significant digit or beyond. These comparisons need lane changing on, and with it CityFlow is not exactly repeatable: repeating the same run, in their code or ours, moves the average travel time by similar or larger amounts, while throughput and their queue length never changed (Section 6.3). The remaining differences are therefore within the simulator's own noise. (An earlier draft attributed

them to extra read-only queries; the fresh-clone check showed that test could not separate the two.) We therefore consider the pipeline validated.

Table 2.2 Our environment against CoLLMLight's code

Our environment against CoLLMLight's code, both driven the same way and scored with their metric definitions. "identical" means equal floating-point numbers; otherwise the relative difference is shown. "CityFlow ATT" is the engine's own average travel time.

Dataset	Controller	ATT	Throughput	Queue	Waiting	CityFlow ATT
synthetic_4x4	Fixedtime	identical	identical	identical	identical	3.1e-07 rel.
synthetic_4x4	MaxPressure	3.7e-06 rel.	identical	identical	identical	9.9e-07 rel.
Jinan 3x4	Fixedtime	identical	identical	identical	identical	1.8e-07 rel.
Jinan 3x4	MaxPressure	identical	identical	identical	identical	1.4e-06 rel.
hangzhou_4x4	Fixedtime	identical	identical	identical	identical	identical
hangzhou_4x4	MaxPressure	identical	identical	identical	identical	4.6e-06 rel.

The validation also corrected one of our own earlier claims. We had written that CityFlow's average travel time leaves out vehicles that never entered the network. It does not: it averages over every generated vehicle, timed from its scheduled start, including vehicles still waiting to enter. A unit test now reproduces the engine's number from the flow file, and every metric description was corrected.

A bug in CoLLMLight's FixedTime baseline

CoLLMLight's fixed-time agent computes its next action as `(cur_phase + 1) % 4`, where `cur_phase` is CityFlow's phase index (1–4). Actions, however, are numbered from 0 and the environment adds 1 again. Phase 1 therefore leads to phase 3 and phase 3 back to phase 1: the controller alternates between the two east–west phases, and **north–south traffic never gets a green**. Their own action log confirms it for every intersection over the whole hour. Table 2.3 shows what this does to the numbers: a correct 4-phase cycle on the same 30 s grid has a far lower travel time on the two real-world datasets. Any comparison with CoLLMLight's published fixed-time numbers must take this into account; we use only our own, correct fixed-time controllers.

Table 2.3 CoLLMLight's FixedTime against a correct 4-phase cycle

CoLLMLight's FixedTime (which serves east–west only) against a correct 4-phase cycle with the same 30 s decisions, and their MaxPressure for reference. One run per dataset, CoLLMLight's settings.

Dataset	Controller	CityFlow ATT (s)	ATT, CoLLMLight's definition (s)	Throughput, CoLLMLight's definition
Synthetic 4x4	CoLLMLight's FixedTime (bug: E-W only)	1612.0	1192.5	751
Synthetic 4x4	CoLLMLight's MaxPressure	1263.3	683.1	2505
Synthetic 4x4	Correct 4-phase FixedTime, same 30 s grid	1420.6	875.6	1724
Jinan 3x4	CoLLMLight's FixedTime (bug: E-W only)	1258.0	1239.3	1786
Jinan 3x4	CoLLMLight's MaxPressure	317.8	273.2	4187
Jinan 3x4	Correct 4-phase FixedTime, same 30 s grid	479.6	441.2	4027
Hangzhou 4x4	CoLLMLight's FixedTime (bug: E-W only)	1054.7	1002.8	1526
Hangzhou 4x4	CoLLMLight's MaxPressure	365.7	325.3	2751
Hangzhou 4x4	Correct 4-phase FixedTime, same 30 s grid	667.9	616.0	2450

The validation also showed why the Synthetic 4x4 dataset is unusual. One of its entry roads carries 2660 vehicles per hour that all need a single straight lane, whose stop line discharges at most 1440 vehicles per hour even with a permanent green. No signal plan can clear it, so a large entry queue is a property of the dataset. For this reason the comparisons in this report use Jinan 3x4 (real flow) and our Indian network, and use Synthetic 4x4 only for validation.

2.2.2 Scenario

Network

The test network (Figure 3, Table 2.4) is a 2 × 2 grid generated with CityFlow's own grid generator. India drives on the left and the generator builds right-hand traffic, but a square grid is mirror-symmetric: a right-hand network is the mirror image of a left-hand one, and the simulation is the same with the turn labels swapped. CityFlow's right turn (kerb side, allowed in every phase) plays the role of India's **free left turn**, and its left turn (across opposing traffic, protected phase) plays the role of India's **right turn**.

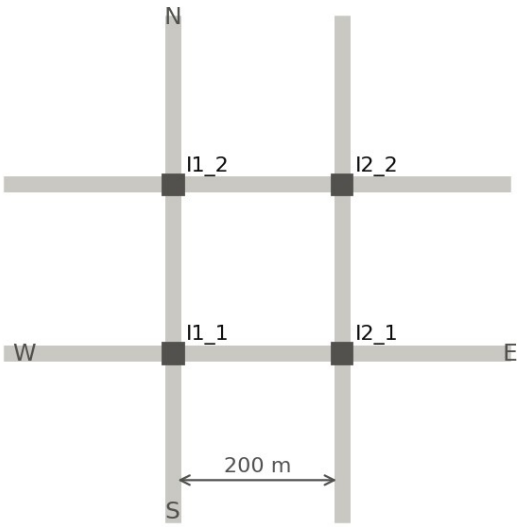


Figure 3 The Indian 2x2 network

The Indian 2x2 network, drawn from its CityFlow road-network file. East–west is the major road.

Table 2.4 Network and signal parameters of the Indian 2x2 scenario

Network and signal parameters of the Indian 2x2 scenario (`configs/scenarios/indian_2x2.yaml`).

Parameter	Value	Source
Grid	2 × 2 signalised intersections	project specification
Spacing between intersections	200 m	project specification
Entry and exit roads	200 m	assumption (same as spacing)
Lanes per direction	3 (one per turn)	CityFlow generator layout
Lane width	3.0 m	project specification
Speed limit	12.5 m/s (45 km/h)	assumption within the 11–13 m/s range
Centre to stop line	12 m	assumption (three lanes plus margin)
Phases	0 = 5 s transition (free turns only); 1–8 green	CityFlow generator
Minimum / maximum green	10 s / 60 s (MaxPressure and LLM arms: minimum 15 s)	configs/default.yaml

Vehicles

Each vehicle type has its own length, gap, headway, speed and acceleration (Table 2.5). The shares are the 2006 modal share of Chennai traffic [9], with bicycles removed (not modelled) and buses and trucks split equally (an assumption). Lengths and widths are the average Indian dimensions given by Umapathi and Khatawkar [29]. The car's gap, headway and accelerations are CityFlow's defaults; the other types' values are

assumptions chosen to be plausible (small vehicles follow closer and accelerate faster) and were not tuned to favour any controller.

Table 2.5 Vehicle types of the Indian scenario

Share = normalised share of all vehicles.

Type	Share	Length (m)	Width (m)	minGap (m)	headwayTime (s)	maxSpeed (m/s)
motorcycle	45.1%	1.67	0.7	1.0	1.0	12.5
auto_rickshaw	17.6%	3.2	1.4	1.5	1.5	11.1
car	30.8%	3.72	1.44	2.5	1.5	12.5
bus	1.6%	10.1	2.43	2.5	2.0	11.1
truck	4.9%	7.5	2.35	2.5	2.0	11.1

Demand

Vehicles arrive as a Poisson process on each of the 8 entry roads. East–west is the major road: each east or west entry has weight 1.0 and each north or south entry 0.6. At every intersection a vehicle goes straight with probability 0.60, takes the free (left) turn with 0.25 and the right turn with 0.15; these shares are assumptions. Each seed produces a different flow file, so seeds give genuinely different traffic.

Demand calibration

The demand levels were not chosen by eye. The goal was an off-peak level that is clearly below capacity and a peak level near capacity that still drains. A three-step procedure, fixed before it was run, set them:

1. **Measure capacity.** Saturation flows were measured in the simulator with the vehicle mix by letting a queue discharge after a long red (Table 2.6).
2. **Sweep demand under MaxPressure.** For total demands from 1000 to 14000 vehicles per hour and three seeds, MaxPressure ran for an hour and then until the network emptied (Figure 4).
3. **Apply the rule.** The network “drains” if at most 2% of the vehicles are still waiting to enter at the end of the hour and it empties within 900 s after arrivals stop, on every seed. The lowest swept demand that does not drain is R^* = 5500 veh/h. Peak is the largest swept demand at or below 0.9 × R^* that drains, 4750 veh/h; off-peak is 0.5 × peak, 2375 veh/h.

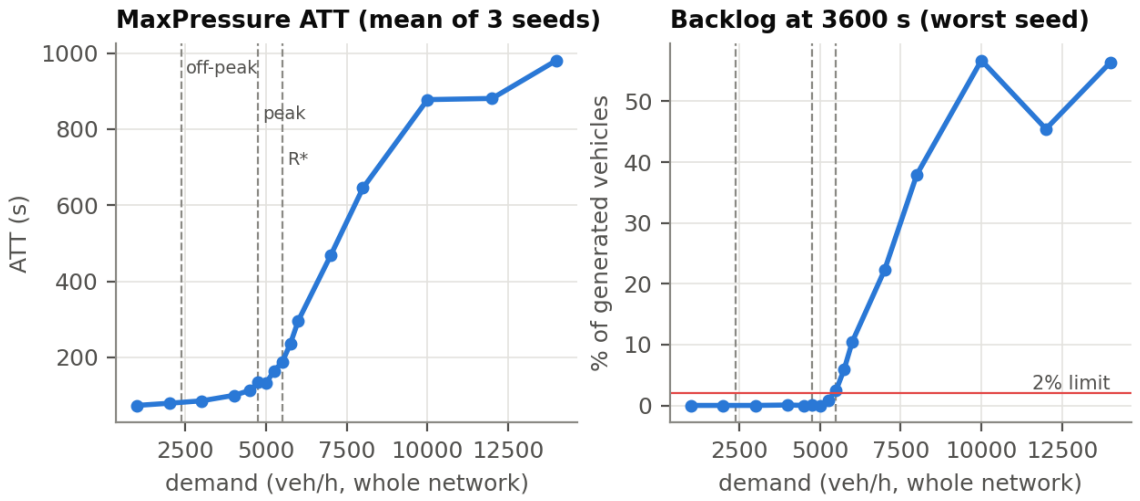


Figure 4 Demand calibration under MaxPressure

Demand calibration under MaxPressure: average travel time and the backlog left at the end of the hour, against total demand. The dashed lines mark the chosen off-peak and peak levels and R^*.

Table 2.6 Capacity of the Indian network, measured in the simulator

Capacity of the Indian network with its vehicle mix, measured in the simulator.

Quantity (mixed traffic, measured)	**Value**
Saturation flow, straight lane	2103 veh/h
Saturation flow, protected-turn lane (Indian right turn)	2024 veh/h
Start-up lost time (mean of both lanes)	1.88 s
Entry admission limit (CityFlow insertion)	1341 veh/h

"Near capacity" here means near the capacity **of MaxPressure as configured**, not of the intersections. A controller-independent check with Webster's critical flow ratio shows that the peak level uses only about 0.55 of Webster's capacity limit. MaxPressure breaks down earlier because it switches often and every switch costs 5 s of transition: under the earlier 10 s minimum green, a third of each intersection's time was transition (Section 4.1). This is why a well-timed fixed plan can keep up with MaxPressure at peak.

Emergency scenario

The emergency scenarios add ambulances to the normal flow files of the peak level without changing any other vehicle. Ambulances arrive at 6 per hour over the whole network (an assumption), with routes drawn like the normal traffic, from a random generator seeded separately so that the normal traffic does not depend on them. An ambulance is a 5 m van at the 12.5 m/s lane limit. Over seeds 0–4 there are 37 ambulances, of which 29 are **signal-relevant**: their route crosses at least one movement a

signal controls (the others only make free turns, which no signal affects). CityFlow has no notion of yielding: other vehicles do not pull over and, with lane changing off, the ambulance cannot overtake. Only the signal can help it, which is exactly what the controllers are tested on.

2.2.3 Perception

A deployed version of this system would not get exact vehicle counts from a simulator; it would count vehicles with a camera and a detector. The perception part of the project has a narrow, measurable purpose: train a detector on real Indian street images, measure the errors it makes in the terms a signal controller cares about (missed vehicles, invented vehicles, wrong classes), and turn those measurements into an error model for the simulation experiment of Section 4.6. The detector is not used to control signals directly.

Dataset

We use the detection part of the **India Driving Dataset** (IDD) [28], collected by IIIT Hyderabad and Intel from vehicles driving in Indian cities. It has bounding boxes for road users including auto-rickshaws, which most Western datasets lack. IDD is filmed from a moving vehicle, not from a pole at a junction; we return to this limitation below.

Classes. We keep 7 classes, mapped one-to-one from IDD: car, motorcycle, autorickshaw, bus, truck, bicycle and person. IDD's `rider` boxes are dropped: a rider sits on a two-wheeler that has its own box, and counting riders would double-count road users, while labelling them `person` would make every two-wheeler look like a pedestrian. The catch-all `vehicle fallback`, the rare `caravan`, `trailer` and `train` classes, animals, traffic signs and traffic lights are dropped as well; regions that contain them become background. These rules were written down before any conversion code existed.

Splits. IDD's official test labels are withheld, so the official test split cannot measure anything. Our **test** split is the official validation split; our **validation** split is 10% of the official training *drives*, chosen per camera group with a fixed seed. Splitting by drive rather than by frame matters: consecutive frames of one drive are near-duplicates, and a frame-level split would leak them into training and flatter the validation scores. The

test split was evaluated once, after every choice (including the confidence threshold) had been frozen on the validation split. Table 2.7 gives the sizes.

Table 2.7 Our IDD splits after the class mapping

Drives = recorded sequences.

Our split	From	Used for	Images	Drives	Boxes
train	official train minus held-out drives	training	28,384	393	247,051
val	held-out official-train drives	early stopping, threshold	3,185	45	28,722
test	official val	final numbers, used once	10,225	119	88,755

Image size. Training and evaluation use images of 640 pixels on the long side, which is what fits the GTX 1650. A full-HD frame is shrunk three times, so a distant motorcycle keeps only a third of its height in pixels. We therefore report errors by object size, so that this cost is visible rather than hidden in an average. The dataset itself stays outside the repository; a resized copy was made once to cut image-reading time.

Training

The detector is YOLOv8n [25] (about 3 million parameters), starting from weights pre-trained on COCO, whose classes already include cars, buses, trucks, motorcycles, bicycles and people. A short probe run decided the batch size: twice the batch needed more memory than the 4 GB card has and was slower, so training used batch 8. Mixed-precision training was switched off because GTX 16-series cards produce invalid (NaN) losses with it; no invalid loss occurred. Training ran for 50 epochs with early stopping after 10 epochs without improvement on our validation split.

The full run took 13.3 hours. Early stopping did not trigger: validation accuracy was still rising very slowly at the end (Figure 5), and the best epoch was 49. An earlier engineering problem is worth recording because it cost more time than the training itself: reading images from the external hard disk through WSL was limited to a few dozen images per second by a fixed cost per file, most of it Windows Defender’s on-access scan. Excluding the dataset folder from the scan removed the bottleneck, and the GPU, not the disk, became the limit.

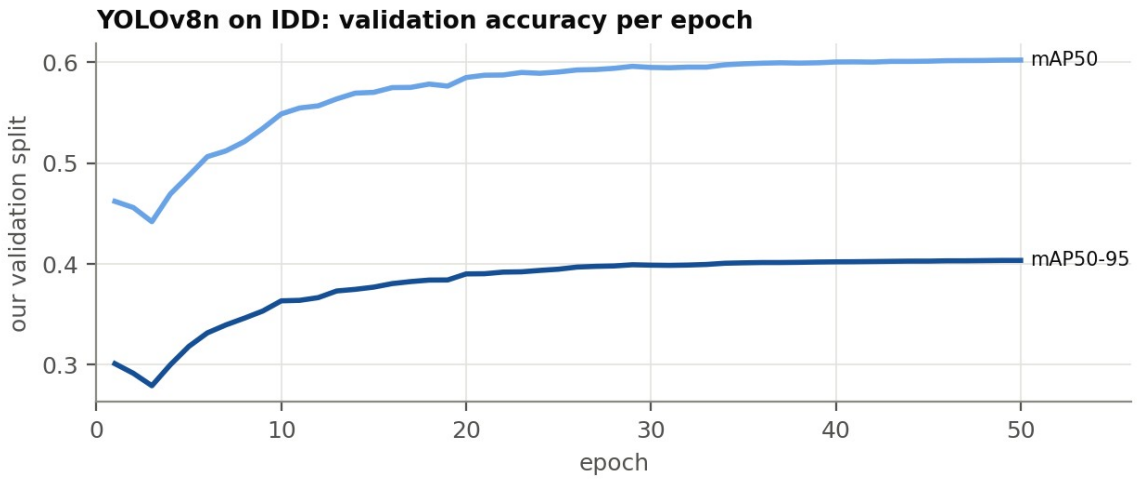


Figure 5 Validation accuracy during training

Accuracy on our validation split after each training epoch.

Camera error model

The measurements are turned into the parameters of the observation-noise layer (Section 2.1.6), stored in `outputs/perception/error_model.json` and read from there by the simulation, never typed into code:

- **Detection probability** of each class, $p_{\text{detect}} = 1 - \text{miss rate}$.
- **Label probabilities**: the probability of each predicted class given the true class and that the object was detected (the normalised rows of the confusion matrix).
- **False detections**: a Poisson count per road and snapshot whose mean is the measured number of false detections per image of each class.

Two camera levels are defined (Table 2.8). The **junction** level uses detection rates measured on medium and large objects, because queued vehicles near a stop line are close to a camera on a pole. The **crowded / far** level uses the rates over all objects, dominated by distant ones. Both use the measured false-detection rates and label probabilities.

Table 2.8 Camera error parameters per detector class

Camera error parameters used in the simulation, per detector class.

detector class	junction p_detect	crowded p_detect	false det. per road and snapshot
car	0.936	0.568	0.222
motorcycle	0.913	0.567	0.271
autorickshaw	0.944	0.598	0.070
bus	0.907	0.632	0.039
truck	0.832	0.537	0.079
bicycle (label only)	–	–	0.006

Limitation. These are measured errors of our detector on dash-camera images. A camera on a pole at a junction sees from a different height and angle, with different occlusion. The error model is therefore a measured *proxy* for a junction camera, not its true error, and the two levels bracket a range rather than predict one value.

2.2.4 Experimental Method

A comparison between a new controller and established ones is easy to bias without meaning to: by tuning the new method until it wins, by choosing the scenario or the metric after seeing the results, or by comparing against a weak baseline. This section describes the safeguards used and defines every metric; the pre-registration of every study is in Section 2.2.5.

Scenarios, seeds and run length

Every run simulates 3600 s. The Indian off-peak and peak scenarios and Jinan 3x4 use seeds 0, 1 and 2; the emergency scenario uses seeds 0–4 because ambulances are few. On the Indian network each seed is a different flow file, so seeds differ in traffic. Jinan has one fixed flow file, so the deterministic baselines give identical results on every seed; only the LLM runs vary there. LLM runs use the run seed as the sampling seed at temperature 0. Even so, Ollama on a GPU is not bit-for-bit deterministic: a repeated run gives slightly different answers in a few calls, after which the traffic diverges. Every decision is logged, so each LLM run can be replayed exactly from its log, and the dashboard's replays are checked to reproduce their runs' metrics.

The full experiments comprise 67 runs (Indian off-peak and peak: all baselines and both LLM arms; Jinan: baselines and the main arm; emergency: five controllers on five seeds). The camera-error experiment comprises 130 runs.

Metrics

All traffic metrics are measured from the simulator, never estimated.

- **Average travel time (ATT)**: CityFlow's `get_average_travel_time()` at the end of the run. It averages over every vehicle generated so far, from its scheduled start to its arrival or to the end of the run, *including time spent waiting to enter the network*. A controller that blocks an entry therefore cannot hide the delay it causes. ATT is the primary metric.
- **Queue length**: at each second, the number of waiting vehicles (speed below 0.1 m/s) on an intersection's incoming lanes; averaged over seconds and intersections.
- **Waiting time**: per vehicle, the seconds spent with speed below 0.1 m/s, averaged over vehicles.
- **Throughput**: vehicles that finished their trip within the hour.
- **Ambulance travel time**: finish time minus scheduled start, per signal-relevant ambulance (the same convention as ATT), pooled over seeds; also its median and the time it spent stopped. Each controller is also compared with preemption ambulance by ambulance.
- **LLM decision statistics**: calls, failures by type, mean and 95th-percentile latency, how often the LLM's answer differs from MaxPressure's choice, how often the guard overrode it, and how often the *applied* phase differs from MaxPressure's. These describe decisions; they are not traffic metrics.
- **Detector metrics**: mAP50 and mAP50-95 (Ultralytics), and precision, recall, miss rate and false detections per image from our own matching (Section 4.7).

Results are given as mean ± sample standard deviation over seeds. Because every controller sees the same flow file on the same seed, the most informative comparison is **paired by seed**: each arm's ATT minus MaxPressure's ATT on the same seed.

Fairness of the comparison

- **Same timing.** MaxPressure, preemption and all LLM arms decide on the same 5 s grid with the same 15 s minimum green and 60 s maximum green, so they differ only in how they choose a phase.
- **Same traffic.** All controllers see the same flow file on the same seed, and ambulances are appended to the normal flow without changing any other vehicle.
- **Baselines tuned first.** MaxPressure and Webster were tuned (Section 4.1) before any LLM run, and Webster's plan uses a perfect count of the simulated demand.
- **Clean runs.** Batch runs refuse to start from a git tree with uncommitted changes, so every result can be traced to a commit. In the LLM benchmark, every loaded model was unloaded and the GPU had to be idle before each model was tested.
- **Repeatability of baselines.** Every baseline run of the full experiments that repeats an earlier run (same scenario, controller, settings, seed and flow file) was compared with it: 27 of 27 are identical in every metric.

Statistical treatment

With three seeds per scenario (five with ambulances) a significance test has little power, and none was pre-registered. Differences are therefore reported as measured, with their spread and, for paired comparisons, the result on each seed. A difference is called a result only when it has the same sign on every seed and is large compared with the seed-to-seed spread; smaller differences are described as "within seed noise".

2.2.5 Pre-registration

Pre-registration means writing down the design of an experiment and the rule that decides its outcome *before* running it, and committing that to version control so the order can be checked [30]. It separates confirmatory results, which test a fixed hypothesis, from exploratory ones, which suggest new hypotheses. Every major step of this project was pre-registered, and each study's document records the commit at which its plan was fixed (Table 2.9).

Table 2.9 What was fixed in advance for each study

Commit hashes refer to the project's git history.

Study	Fixed before any of its runs
Demand calibration	the "drains" criterion, the rule for R^*, peak and off-peak
MaxPressure tuning	the swept settings and the rank-based selection rule with its tie-break (commits `8414f9a`, `c7a6e34`)
LLM benchmark	the setup and the model-selection rule; reused unchanged in every round
Final LLM design round	the three arms, and that its pick goes to the full experiments even if worse than MaxPressure
Full experiments	arms, scenarios, seeds, run length and analysis (commit `694dc0c`)
Emergency experiment	ambulance rate, seeds, controllers and the primary metric
Detector	class mapping, splits, image size, training settings; threshold frozen on validation (`876076d`) before the single test evaluation
Camera-error experiment	noise model, levels, runs, analysis and checks (`91aa214`)
Model-size study	models, runs and tables, committed before its runs, but the study itself was added after the main results (exploratory)

The model-selection rule. Each LLM benchmark run is a 600 s run of the Indian peak scenario, seed 0. A model is **eligible** if fewer than 2% of its calls failed and its 95th-percentile latency is below 3 s; among eligible models the one with the lowest mean queue per intersection is chosen, ties broken by travel time. The decision interval is then the shortest multiple of the 5 s transition that is at least the chosen model's 95th-percentile latency times the number of intersections, so that a full round of decisions fits in real time on one GPU.

Deviations and additions, stated openly. Four things were not part of the original plans and are labelled wherever they appear:

1. **Prompt v3 is a post-hoc fix.** After the first rounds showed that the prompts never said which direction of pressure is better, two lines were changed (Section 4.2). The final design round, which used v3, was itself pre-registered and declared final.
2. **The `gemma4:e4b` hybrid arm is exploratory.** It failed the selection rule only on latency and was added to the full experiments, at the user's request, after their plan was fixed. It is labelled "exploratory, NOT selected by the rule" in every table.
3. **The model-size study** was planned after the main results and is exploratory.

4. **The camera-error decomposition** (one error source at a time) was added after the camera results had been read, to test an explanation offered for them; it is post-hoc.

2.3 Standards

TrafficSense is a simulation and software study: it controls no signal in the field, so no standard for traffic-signal equipment applies to it. Table 2.10 lists the formats, definitions and practices it does follow. Each is used by the code itself, so a departure from it would break a run, a test or a check.

Table 2.10 Standards, formats and conventions followed

Standards, formats and conventions the project follows.

Area	Standard or convention	Where and how it is used
Road network and demand	CityFlow's road-network and flow file formats (JSON)	The Indian 2x2 network and its flows are generated in these formats and run by the unmodified engine; lane ids are `road_lane`; signals are under external control (`rlTrafficLight`)
Data interchange	JSON (RFC 8259); JSON Lines for logs	Every result file (`summary.json`), every decision log (`decisions.jsonl`, one object per line) and every LLM answer
LLM output	JSON Schema, through Ollama's structured-output `format` field	The answer must be an object with a phase from the allowed list and a short reason; our code parses and checks it again (Section 2.1.2)
Detector data	IDD annotations in Pascal VOC XML; the YOLO label format	`perception/idd.py` converts the one into the other (Section 2.2.3)
Detector metrics	COCO-style evaluation: mAP at IoU 0.5 and averaged over IoU 0.5 to 0.95; COCO's object-size buckets	Section 4.7
Traffic metrics	CityFlow's average travel time; queue, waiting time and throughput as defined in Section 2.2.4	Every result table
Experiment design	Pre-registration [30]	Every study (Section 2.2.5)
Referencing	IEEE citation style	The reference list, through the IEEE citation style file of the report build
Licences	The open-source licences and terms of use of every tool, dataset and model	Chapter 3; the dataset and the model weights are not redistributed

2.4 System Details

This section lists the software the system is built from and the hardware it ran on.

2.4.1 Software Details

Table 2.11 lists the software. Python package versions are those pinned in the repository's lock file (`requirements-lock.txt`), the environment every committed result was checked with; the Ollama version is the one every local LLM run recorded; the report tools are those of the last report build. The table is generated from those files. The controllers, the observation layer, the experiment scripts and the dashboard are the project's own Python code.

Table 2.11 Software used, with versions

Component	Software	Version	Used for
Operating system	Ubuntu in WSL2 on Windows 11	22.04.5	everything below
Language	Python	3.10.12	all project code
Simulator	CityFlow, built from source	commit 81ee0f4	traffic simulation
LLM server	Ollama, native /api/chat with a JSON schema	0.40.3	local LLM agents
Local LLMs	llama3.2:3b (selected), qwen2.5:3b, gemma4:e4b; gemma3:4b (round 1 only); llama3.1:8b (model-size study)	4-bit quantised Ollama builds	signal-control agents
Hosted LLM	nvidia/nemotron-3-ultra-550b-a55b, NVIDIA's API	as served	exploratory model-size study
Detector	Ultralytics (YOLOv8n)	8.4.183	training and inference
Deep learning	PyTorch, torchvision (CUDA build)	2.14.1+cu126, 0.29.1+cu126	detector
Images and video	OpenCV, Pillow	5.0.0.93, 12.3.0	detector input, dashboard
Numerics	NumPy	2.2.6	metrics, statistics
Configuration	PyYAML	6.0.3	configs/*.yaml
HTTP	requests	2.34.2	Ollama and NVIDIA API calls
Charts	Matplotlib, Plotly	3.10.9, 7.1.0	report figures, dashboard charts
Dashboard	Streamlit	1.65.0	the four dashboard pages
Tests	pytest, Playwright (Chromium)	9.1.1, 1.64.1	test suite, dashboard layout tests
Report	pandoc, LibreOffice, pypdfium2	3.12.1, 25.8.6.2, 5.14.0	this report (Word file and PDF)

2.4.2 Hardware Details

All experiments, the detector training and this report ran on one laptop, the project laptop of Table 2.12. A second laptop was used only to check that a stranger could reproduce the setup from the README (Section 6.3); its figures are as reported by the project owner.

Table 2.12 Hardware

The project laptop's values are from the fresh-clone record; the second laptop's are as reported by the project owner.

	Project laptop	**Second laptop**
Used for	every experiment, the detector training and this report	a reproduction check only (tests, detector weights, dashboard, one short LLM trial)
CPU	AMD Ryzen 5 4600H	AMD Ryzen 9 5900HS
Memory	16 GB	16 GB
GPU	NVIDIA GTX 1650, 4 GB	NVIDIA RTX 3050 Ti, 4 GB
Operating system	Windows 11 Home, WSL2, Ubuntu 22.04.5 LTS	Windows 11, WSL2, Ubuntu 22.04

There is no field hardware. No camera, signal controller or roadside computer was used: every experiment is a simulation, and the detector was trained and tested on recorded images.

The GTX 1650 limits the work in two ways that shape the results: `gemma3:4b` does not fit in its memory and runs partly on the CPU, and the stronger `gemma4:e4b` needs several seconds per call, too slow for real-time control of four intersections. The LLM and the detector never ran on the GPU at the same time.

CHAPTER 3
COST ANALYSIS

This chapter lists every tool, dataset, model and piece of equipment the project used, with its licence and its cost to the project. Nothing was bought. All the software is free and open source, or free of charge. The dataset and the model weights are free for research use under their own terms. The laptops were the team members' own and are not costed. No price in this chapter is estimated.

3.1 List of Components and Their Cost

Table 3.1 List of components and their cost

Every tool, dataset, model and piece of equipment used, its licence and its cost to the project. Licences of the Python packages are those their installed package metadata state.

Component	Used for	Licence or terms	Cost to the project
CityFlow	traffic simulation	Apache-2.0	free (open source)
Python	all project code	PSF License	free (open source)
NumPy	metrics, statistics	BSD-3-Clause	free (open source)
PyYAML	configuration files	MIT	free (open source)
requests	calls to Ollama and the NVIDIA API	Apache-2.0	free (open source)
OpenCV, Pillow	images and video	Apache-2.0; MIT-CMU	free (open source)
Matplotlib, Plotly	report figures, dashboard charts	Matplotlib License (PSF-based); MIT	free (open source)
PyTorch, torchvision	detector training and inference	BSD-3-Clause with bundled Apache-2.0, BSD-2-Clause, BSL-1.0 and MIT parts; BSD	free (open source)
Ultralytics YOLOv8	the detector	AGPL-3.0	free (open source)
Ollama	local LLM server	MIT	free (open source)
Streamlit	dashboard	Apache-2.0	free (open source)
pytest, Playwright	tests, dashboard layout tests and screenshots	MIT; Apache-2.0	free (open source)
pandoc, LibreOffice, pypdfium2	this report (Word file and PDF)	GPL-2.0-or-later; MPL-2.0; Apache-2.0 or BSD-3-Clause	free (open source)
Ubuntu in WSL2	operating system for all the work	open-source licences of the distribution	free
NVIDIA driver and CUDA	GPU computing	NVIDIA's licence	free of charge
llama3.2:3b, llama3.1:8b	LLM agents	Llama Community License	free download, not redistributed
gemma4:e4b, gemma3:4b	LLM agents	Gemma terms of use	free download, not redistributed
qwen2.5:3b	LLM agent (benchmark)	Qwen licence	free download, not redistributed

Component	Used for	Licence or terms	Cost to the project
nvidia/nemotron-3-ultra-550b-a55b	exploratory model-size study only	NVIDIA's API terms	free tier (no charge incurred, as confirmed by the team); usage and credit amounts were not recorded by the project
India Driving Dataset (IDD)	detector training and testing	IDD terms of use, after registration	free for research, not redistributed
CoLLMLight code and data	validation of the simulator	no licence file (checked at commit ce204a9)	free download; run unmodified, not copied
Geist and Geist Mono fonts	dashboard	SIL Open Font License 1.1	free (open source)
Times New Roman, Consolas	fonts of this report	licensed with Windows	not costed (came with the laptops)
Project laptop, second laptop	all computing (Table 2.12)	the team's own	not costed
TrafficSense code	the system itself (own work)	no licence file in the repository	not applicable

Two of these licences carry obligations worth knowing before any deployment. Ultralytics YOLOv8 is under AGPL-3.0, which requires the source of a networked application that uses it to be made available to its users. The model weights and IDD may be used only under their own terms, which is why neither is redistributed with the project.

3.2 What a Real Deployment Would Additionally Need (Not Costed)

The project is a simulation study, so a deployment at real junctions would need components the project never had. **None of them was priced**: their cost depends on vendors, on the number of junctions and on the city's existing equipment, and an estimate made here would be invented.

- **Cameras at every junction approach**, with poles, mounting and power. The detector was trained and tested on dash-camera images, so it would also need images from these cameras to be retrained and its errors measured again (Section 2.2.3).
- **Computing at the junction or in a control centre**, with a GPU for the detector and the LLM agents.
- **An interface to the existing signal controllers**, approved by the traffic authority, with a safe fallback to the controller's own fixed plan on any fault.

- **Network links** between neighbouring junctions (the agents share their neighbours' states) and to a control centre.
- **Ambulance positions from GPS or V2X**, which the simulation assumes (Section 2.1.5).
- **Field trials and calibration** against counted traffic, maintenance, power backup, and protection of the camera footage.

CHAPTER 4
RESULTS AND DISCUSSIONS

This chapter reports the results in the order the experiments were run: the tuning of the classical baselines, the LLM benchmark that selected the controller, the full experiments, the ambulance experiment, the exploratory model-size study and the camera-error experiment. It then gives the detector's evaluation and the dashboard, and discusses what the results mean and what limits them. Every table in this chapter is generated from the result files.

4.1 Baselines and Their Tuning

An LLM controller should be compared with baselines that are as good as they can reasonably be, and those must be fixed before the comparison, not after. Three classical controllers were used.

- **FixedTime**: the 4 paired phases (each pair of opposing approaches together), 30 s of green each. This is the naive plan.
- **Webster fixed time**: the same 4 phases with a cycle and splits computed per intersection by Webster's method. Saturation flows and start-up lost times are measured on the network with the run's vehicle mix (mean of 30 random-order queues), and lane demand is read from the run's own flow file, i.e. a perfect count of the simulated hour. This favours Webster: a real plan is built from older counts.
- **MaxPressure**: all 8 green phases, ties keep the current phase.

MaxPressure's timing was tuned in two sweeps on four scenarios (Jinan, Hangzhou, Indian off-peak and peak; 3 seeds each), with selection rules committed before the runs: rank the settings by mean travel time in each scenario, choose the lowest mean rank, break ties by the lowest mean ratio to the best. The first sweep varied the decision interval and showed that what really changed was the shortest possible green. The second sweep therefore fixed the interval at 5 s and varied the minimum green (Table 4.1). A minimum green of 15 s was chosen, through the tie-break: the 10 s setting is best on three scenarios but last at Indian peak.

Table 4.1 Pre-registered selection of MaxPressure's minimum green

Pre-registered selection of MaxPressure's minimum green (decision interval 5 s): rank of each setting by mean travel time per scenario, mean rank, and the tie-break.

Min green	Jinan	Hangzhou	Off-peak	Peak	Mean rank	Mean ATT / best	Chosen
10 s	1	1	1	4	1.75	1.058	
15 s	2	2	2	1	1.75	1.026	chosen
20 s	3	3	3	2	2.75	1.072	
25 s	4	4	4	3	3.75	1.139	

The minimum green matters because of the transition. With a 10 s minimum green MaxPressure switches so often that 32.4% of each intersection's time at peak is transition, against 24.7% with 15 s. In light traffic reacting quickly wins; near capacity the green lost to transitions costs more. No setting is best at both levels, and the pre-registered rule chose the compromise.

Table 4.2 gives all baselines on the Indian network. Off-peak, MaxPressure is clearly best. At peak, the tuned MaxPressure and the Webster plan are effectively tied (115.5 s and 116.3 s), and the 30 s FixedTime no longer drains: about 308 vehicles are still waiting to enter at the end of the hour. This tie matters for reading the LLM results: at peak, "as good as MaxPressure" is also "as good as a well-timed fixed plan".

Table 4.2 All baselines on the Indian 2x2 network

All baselines on the Indian 2x2 network, mean ± std over seeds 0–2, 3600 s. The default MaxPressure is the tuned one (5 s interval, 15 s minimum green).

Level	Controller	ATT (s)	Queue / int. (veh)	Finished trips	Waiting to enter at end	Share of time in transition
off-peak	FixedTime, 4 paired phases, 30 s	121.5 ± 0.2	10.58 ± 0.17	2316 ± 28	0.0 ± 0.0	0.142 ± 0.000
off-peak	FixedTime, all 8 phases, 30 s	129.3 ± 1.3	11.88 ± 0.42	2298 ± 28	0.0 ± 0.0	0.142 ± 0.000
off-peak	FixedTime, 4 paired phases, Webster plan	100.4 ± 4.1	6.82 ± 0.52	2320 ± 36	0.0 ± 0.0	0.326 ± 0.006
off-peak	MaxPressure (8 phases, 5 s, min 15 / max 60)	88.1 ± 1.0	4.94 ± 0.23	2332 ± 36	0.0 ± 0.0	0.246 ± 0.001
off-peak	MaxPressure (8 phases, 15 s, min 10 / max 60)	81.3 ± 0.3	3.80 ± 0.07	2332 ± 38	0.0 ± 0.0	0.314 ± 0.003
off-peak	MaxPressure (8 phases, 5 s, min 10 / max 60)	82.0 ± 0.8	3.91 ± 0.17	2332 ± 37	0.0 ± 0.0	0.319 ± 0.003
peak	FixedTime, 4 paired phases, 30 s	279.9 ± 9.9	44.44 ± 0.75	4166 ± 93	307.7 ± 20.5	0.142 ± 0.000
peak	FixedTime, all 8 phases, 30 s	291.9 ± 7.3	47.67 ± 0.62	4113 ± 88	331.0 ± 18.3	0.142 ± 0.000
peak	FixedTime, 4 paired phases, Webster plan	116.3 ± 4.6	18.27 ± 1.28	4577 ± 87	0.3 ± 0.6	0.227 ± 0.004
peak	MaxPressure (8 phases, 5 s, min 15 / max 60)	115.5 ± 0.9	17.83 ± 0.32	4576 ± 68	0.3 ± 0.6	0.247 ± 0.001
peak	MaxPressure (8 phases, 15 s, min 10 / max 60)	133.8 ± 15.9	23.24 ± 4.60	4540 ± 116	0.3 ± 0.6	0.321 ± 0.002
peak	MaxPressure (8 phases, 5 s, min 10 / max 60)	142.4 ± 18.3	25.84 ± 5.22	4516 ± 131	0.3 ± 0.6	0.324 ± 0.001

4.2 LLM Benchmark and the Design Rounds

Each LLM arm was benchmarked on the same 600 s of Indian peak traffic (seed 0) with up to four local models (`llama3.2:3b` from the Llama 3 family [31], `qwen2.5:3b` [32], and Google's `gemma3:4b` and `gemma4:e4b`), and the pre-registered rule (Section 2.2.5) was applied. Figure 6 and Table 4.3 summarise all rounds; Table 4.4 gives what the rule chose.

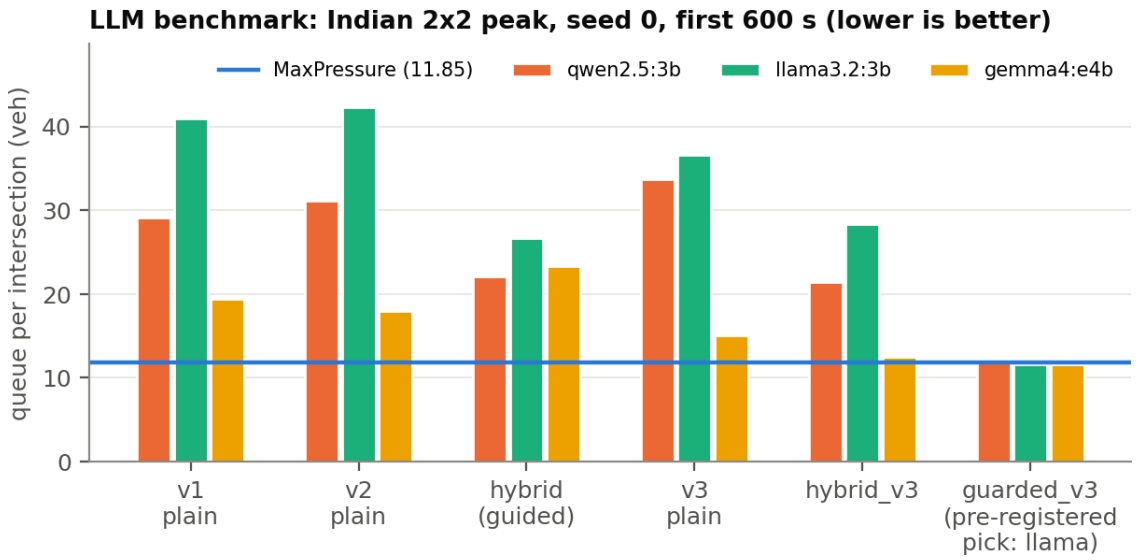


Figure 6 LLM benchmark: mean queue of every arm and model

Mean queue per intersection over the first 600 s of Indian peak traffic for every arm and model, against MaxPressure (blue line). gemma3:4b, run only in v1, is in Table 4.3.

Round 1 (prompt v1). No model met the rule. `qwen2.5:3b` was just too slow (95th-percentile latency 3.34 s against the 3 s limit) and `llama3.2:3b` failed 6.2% of its calls with over-long reasons; both Gemma models were slower still. Every model controlled traffic much worse than MaxPressure: queues of 19–46 vehicles per intersection against 11.85. `gemma3:4b` did not fit in the 4 GB card and was dropped after this round. Latency was mostly the time to read the long prompt, so the next prompt was made much shorter.

Round 2 (prompt v2 and the hybrid). Prompt v2 gave each phase its pressure and queue directly, in about 285 tokens instead of about 654. The rule then chose `llama3.2:3b`, the only eligible model, which was also the worst controller of the round (queue 42.26). The decision logs showed why: given pressures, the small models **preferred low pressure**. In the hybrid arm, which offered only MaxPressure's two best phases or keeping the current one, every model chose the lowest-pressure option in 77–86% of the calls where the options differed. Their reasons said so directly, for example "Low pressure, minimal queue impact". The prompts defined pressure but never said which direction is better.

Final round (prompt v3, pre-registered as final). Two lines of the prompts were changed: the definition of pressure now ends "HIGHER pressure = the phase needs

green MORE", and the reason format no longer contains a placeholder that one model had copied into its answers. Three arms were tested with this prompt: plain v3, hybrid_v3, and guarded_v3, which adds the pressure guard (Section 2.1.4). The fix changed `gemma4:e4b`, which in hybrid_v3 chose the highest-pressure option in 78% of its calls. It did not change the two 3B models, which still kept the current phase in most calls.

Table 4.3 LLM benchmark, all rounds

LLM benchmark, all rounds: Indian 2x2 peak, seed 0, first 600 s. MaxPressure on the same traffic: queue 11.85 vehicles per intersection, ATT 92.5 s. Eligible = failure rate and latency within the pre-registered limits.

Arm	Model	Failures	Latency p95 (s)	Queue / int. (veh)	ATT at 600 s (s)	Eligible
v1	gemma3:4b	0.0% (none)	5.43	46.35	197.4	no
v1	gemma4:e4b	0.6% (reason_too_long 1)	8.36	19.44	112.2	no
v1	llama3.2:3b	6.2% (reason_too_long 18)	3.35	40.94	181.1	no
v1	qwen2.5:3b	0.0% (none)	3.34	29.15	141.2	no
v2	gemma4:e4b	0.0% (none)	5.32	17.95	111.1	no
v2	llama3.2:3b	0.3% (reason_too_long 1)	1.52	42.26	181.5	yes
v2	qwen2.5:3b	29.3% (reason_too_long 104)	1.65	31.11	146.8	no
hybrid	gemma4:e4b	0.4% (reason_too_long 1)	4.84	23.38	125.8	no
hybrid	llama3.2:3b	0.3% (reason_too_long 1)	1.13	26.64	135.3	yes
hybrid	qwen2.5:3b	11.2% (reason_too_long 28)	1.27	22.05	123.3	no
v3	gemma4:e4b	0.0% (none)	5.22	15.12	103.1	no
v3	llama3.2:3b	0.0% (none)	1.40	36.62	163.5	yes
v3	qwen2.5:3b	1.4% (reason_too_long 5)	1.53	33.67	156.1	yes
hybrid_v3	gemma4:e4b	0.0% (none)	4.80	12.46	93.6	no
hybrid_v3	llama3.2:3b	0.0% (none)	1.19	28.34	140.8	yes
hybrid_v3	qwen2.5:3b	7.7% (reason_too_long 21)	1.32	21.44	120.0	no
guarded_v3	gemma4:e4b	0.0% (none)	4.81	11.56	91.6	no
guarded_v3	llama3.2:3b	0.0% (none)	1.18	11.62	92.2	yes
guarded_v3	qwen2.5:3b	10.2% (reason_too_long 13)	1.23	11.85	92.5	no

4.2.1 Selection

By the pre-registered rule the final round selected **`llm_guarded_mp` with `llama3.2:3b`** and a decision interval of 5 s (95th-percentile latency times four intersections: 4.7 s). Its queue in the benchmark, 11.62, was slightly below MaxPressure's, but the guard had replaced most of the model's answers: the applied phase differed from MaxPressure's in only 15 of 121 decisions. As pre-registered, this controller went to the full experiments whatever its benchmark result, and design work on the LLM controller stopped. The best-performing LLM-led controller, `gemma4:e4b`, was not eligible because of its latency (95th percentile 4.80 s).

Table 4.4 What the pre-registered rule selected in each round

What the pre-registered rule selected in each round and in the final decision.

Applied to	Eligible	Chosen	Decision interval
v1	none	none	–
v2	llama3.2:3b	llama3.2:3b	10 s (p95 × intersections = 6.1 s)
hybrid	llama3.2:3b	llama3.2:3b	5 s (p95 × intersections = 4.5 s)
v3	llama3.2:3b, qwen2.5:3b	qwen2.5:3b	10 s (p95 × intersections = 6.1 s)
hybrid_v3	llama3.2:3b	llama3.2:3b	5 s (p95 × intersections = 4.8 s)
guarded_v3	llama3.2:3b	llama3.2:3b	5 s (p95 × intersections = 4.7 s)
Phase 3b: all of v1, v2, hybrid	hybrid llama3.2:3b, v2 llama3.2:3b	hybrid llama3.2:3b	5 s (p95 × intersections = 4.5 s)
Final decision (final-round arms)	guarded_v3 llama3.2:3b, hybrid_v3 llama3.2:3b, v3 llama3.2:3b, v3 qwen2.5:3b	guarded_v3 llama3.2:3b (`llm_guarded_mp`); queue 11.62 vs MaxPressure 11.85	5 s (p95 × intersections = 4.7 s)

4.3 Full Experiments

Table 4.5, Table 4.6 and Table 4.7 give the results of the full experiments, and Figure 7 shows the travel times side by side.

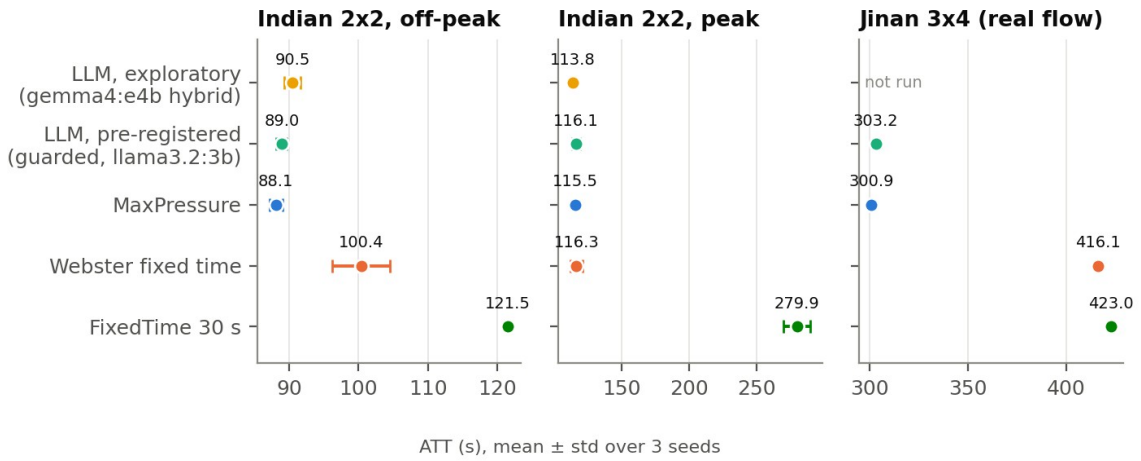


Figure 7 Average travel time of every controller

Average travel time of every controller in the three scenarios (mean ± std over three seeds). The two LLM arms are close to MaxPressure everywhere; FixedTime fails at peak.

Table 4.5 Full experiments, Indian 2x2 off-peak

Indian 2x2, off-peak (2375 veh/h), 3600 s, mean ± std over seeds 0–2.

Controller	Seeds	ATT (s)	Queue / int. (veh)	Waiting / vehicle (s)	Finished trips	ATT vs MaxPressure
MaxPressure	3	88.1 ± 1.0	4.94 ± 0.23	29.8 ± 0.9	2332 ± 36	+0.0%
Webster fixed time	3	100.4 ± 4.1	6.82 ± 0.52	41.1 ± 3.9	2320 ± 36	+13.9%
FixedTime (4 phases, 30 s)	3	121.5 ± 0.2	10.58 ± 0.17	63.7 ± 0.5	2316 ± 28	+37.8%
LLM, pre-registered (guarded, llama3.2:3b)	3	89.0 ± 0.8	5.08 ± 0.18	30.6 ± 0.6	2327 ± 41	+0.9%
LLM, exploratory (hybrid_v3, gemma4:e4b)	3	90.5 ± 1.2	5.37 ± 0.28	32.3 ± 1.1	2323 ± 34	+2.7%

Table 4.6 Full experiments, Indian 2x2 peak

Indian 2x2, peak (4750 veh/h), 3600 s, mean ± std over seeds 0–2.

Controller	Seeds	ATT (s)	Queue / int. (veh)	Waiting / vehicle (s)	Finished trips	ATT vs MaxPressure
MaxPressure	3	115.5 ± 0.9	17.83 ± 0.32	54.1 ± 0.5	4576 ± 68	+0.0%
Webster fixed time	3	116.3 ± 4.6	18.27 ± 1.28	55.4 ± 4.0	4577 ± 87	+0.7%
FixedTime (4 phases, 30 s)	3	279.9 ± 9.9	44.44 ± 0.75	144.0 ± 1.6	4166 ± 93	+142.3%
LLM, pre-registered (guarded, llama3.2:3b)	3	116.1 ± 2.9	17.97 ± 0.85	54.5 ± 2.5	4582 ± 77	+0.5%
LLM, exploratory (hybrid_v3, gemma4:e4b)	3	113.7 ± 0.3	17.33 ± 0.44	52.6 ± 0.5	4577 ± 77	-1.5%

Table 4.7 Full experiments, Jinan 3x4

Jinan 3x4 (CoLLMLight's real-world flow), 3600 s, seeds 0–2. The flow file is fixed, so the deterministic baselines are identical on every seed.

Controller	Seeds	ATT (s)	Queue / int. (veh)	Waiting / vehicle (s)	Finished trips	ATT vs MaxPressure
MaxPressure	3	300.9 ± 0.0	5.62 ± 0.00	55.7 ± 0.0	4130 ± 0	+0.0%
Webster fixed time	3	416.1 ± 0.0	16.42 ± 0.00	162.6 ± 0.0	3965 ± 0	+38.3%
FixedTime (4 phases, 30 s)	3	423.0 ± 0.0	17.76 ± 0.00	176.2 ± 0.0	3968 ± 0	+40.6%
LLM, pre-registered (guarded, llama3.2:3b)	3	303.2 ± 1.1	5.81 ± 0.12	57.7 ± 1.2	4134 ± 5	+0.8%

The pre-registered controller matches MaxPressure but does not beat it. Its ATT is +0.9% off-peak, +0.5% at peak and +0.8% on Jinan relative to MaxPressure. Paired by seed (Figure 8, Table 4.8), it is behind MaxPressure on all 3 off-peak seeds, all 3 Jinan seeds and 2 of the 3 peak seeds, by 0.2–3.5 s.

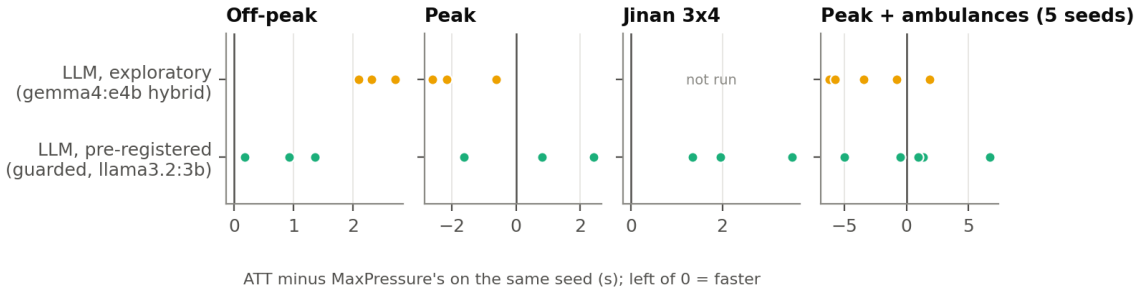


Figure 8 Paired ATT difference of the LLM arms from MaxPressure

ATT minus MaxPressure's ATT on the same seed, for the two LLM arms. Each dot is one seed; left of zero is faster than MaxPressure.

Table 4.8 Paired comparison with MaxPressure

Paired comparison: each LLM arm's ATT minus MaxPressure's on the same seed.

Scenario	Controller	ATT minus MaxPressure, per seed (s)	Seeds lower / higher than MaxPressure
Indian 2x2 off-peak	LLM, pre-registered (guarded, `llama3.2:3b`)	+0.92 / +1.36 / +0.18	0 lower, 3 higher of 3
Indian 2x2 off-peak	LLM, exploratory (hybrid_v3, `gemma4:e4b`)	+2.30 / +2.09 / +2.70	0 lower, 3 higher of 3
Indian 2x2 peak	LLM, pre-registered (guarded, `llama3.2:3b`)	+0.81 / +2.42 / -1.63	1 lower, 2 higher of 3
Indian 2x2 peak	LLM, exploratory (hybrid_v3, `gemma4:e4b`)	-2.61 / -2.15 / -0.61	3 lower, 0 higher of 3
Jinan 3x4	LLM, pre-registered (guarded, `llama3.2:3b`)	+3.52 / +1.96 / +1.35	0 lower, 3 higher of 3
Peak + ambulances	Preemption (rule-based)	+1.33 / +0.01 / -1.71 / +3.18 / -1.28	2 lower, 3 higher of 5
Peak + ambulances	LLM, pre-registered (guarded, `llama3.2:3b`)	+6.72 / +1.31 / -0.50 / +0.92 / -5.03	2 lower, 3 higher of 5
Peak + ambulances	LLM, exploratory (hybrid_v3, `gemma4:e4b`)	+1.85 / -3.43 / -6.23 / -0.80 / -5.73	4 lower, 1 higher of 5

Why so close: the guard does most of the work (Table 4.9). The model's own answer differs from MaxPressure's choice in 87–93% of calls, but the guard replaces most of these answers (76–83% of calls), so the applied phase differs from MaxPressure's in only 10–11% of calls. Every remaining difference is a decision the guard judged "close enough", and on average those cost a little. The arm never failed a call off-peak or at peak and runs in real time: mean latency 1.02–1.10 s per call for a 5 s decision interval.

The exploratory `gemma4:e4b` arm is the only LLM arm that beats MaxPressure anywhere, and only under heavy traffic. At peak its ATT is lower on all 3 seeds (paired differences −0.6 to −2.6 s (−1.5% on the mean); off-peak it is worse on every seed (+2.7%). It departs from MaxPressure more off-peak (32.0% of calls) than at peak (20.9%), and off-peak those departures cost time. With three seeds and no significance test this is a lead, not a demonstrated improvement. The arm is also not real-time on this

GPU: 4.54 s per call on average, so an hour of traffic took 3.2 hours over three seeds, against 0.6 hours for the main arm.

Table 4.9 LLM decision statistics

LLM decision statistics, pooled over the seeds of each scenario. “Answer differs” is over successful calls; “applied phase differs” over all calls.

Scenario	Arm	Calls	Failures	Latency mean (s)	p95 per run (s)	Answer differs from MaxPressure	Guard replaced it	Applied phase differs
Indian 2x2 off-peak	LLM, pre-registered (guarded, llama3.2:3b)	2177	0.0% (none)	1.05	1.21–1.22	91.5%	81.2%	10.4%
Indian 2x2 off-peak	LLM, exploratory (hybrid_v3, gemma4:e4b)	2802	0.1% (reason_too_long 4)	4.61	4.92–5.02	32.1%	– (no guard)	32.0%
Indian 2x2 peak	LLM, pre-registered (guarded, llama3.2:3b)	2167	0.0% (none)	1.02	1.19–1.19	92.8%	82.8%	10.0%
Indian 2x2 peak	LLM, exploratory (hybrid_v3, gemma4:e4b)	2541	0.1% (reason_too_long 3)	4.54	4.79–4.98	20.9%	– (no guard)	20.9%
Jinan 3x4	LLM, pre-registered (guarded, llama3.2:3b)	6665	0.0% (reason_too_long 1)	1.10	1.28–1.36	89.0%	77.8%	11.1%
Peak + ambulances	LLM, pre-registered (guarded, llama3.2:3b)	3632	0.0% (none)	1.06	1.24–1.24	86.9%	76.3%	10.6%
Peak + ambulances	LLM, exploratory (hybrid_v3, gemma4:e4b)	4146	0.0% (reason_too_long 2)	4.72	5.02–5.15	20.4%	– (no guard)	20.4%

4.4 Ambulances

Table 4.10 and Table 4.11 give the emergency experiment: peak traffic plus ambulances, seeds 0–4, with the primary metric the mean travel time of the 29 signal-relevant ambulances.

Table 4.10 Emergency scenario, normal traffic

Emergency scenario, normal traffic: mean ± std over seeds 0–4.

Controller	ATT (s)	Queue / int. (veh)	Finished trips
MaxPressure	115.7 ± 2.3	17.97 ± 0.86	4605 ± 66
Webster fixed time	115.2 ± 2.7	18.01 ± 0.64	4606 ± 73
MaxPressure + preemption	116.0 ± 0.7	18.08 ± 0.41	4606 ± 61
LLM, pre-registered (guarded, `llama3.2:3b`)	116.4 ± 3.1	18.16 ± 0.72	4604 ± 64
LLM, exploratory (hybrid_v3, `gemma4:e4b`)	112.9 ± 1.9	17.11 ± 0.30	4610 ± 70

Table 4.11 Signal-relevant ambulances, pooled over seeds 0–4

The last column compares each ambulance with the same ambulance under rule-based preemption.

Controller	Finished	Travel time (s)	Median (s)	Stopped (s)	Per ambulance vs preemption (s)
MaxPressure	29 / 29	130.1	126.0	62.3	48.9 (1 faster, 28 slower of 29)
Webster fixed time	29 / 29	123.5	123.0	55.0	42.3 (4 faster, 25 slower of 29)
MaxPressure + preemption	29 / 29	81.2	80.0	14.1	–
LLM, pre-registered (guarded, `llama3.2:3b`)	29 / 29	88.9	87.0	21.2	7.7 (12 faster, 15 slower of 29)
LLM, exploratory (hybrid_v3, `gemma4:e4b`)	29 / 29	82.2	78.0	15.3	1.1 (16 faster, 12 slower of 29)

Both LLM arms get most of the benefit of preemption (Figure 9). Mean ambulance travel time is 130.1 s under MaxPressure and 123.5 s under the Webster plan, which ignore ambulances; 88.9 s under the pre-registered LLM arm; and 81.2 s under rule-based preemption. The difference comes almost entirely from time spent stopped: 62.3 s under MaxPressure, 21.2 s under the main LLM arm. Ambulance by ambulance, the main arm is 7.7 s slower than preemption on average (12 faster, 15 slower); the exploratory `gemma4:e4b` arm is only 1.1 s slower. Part of the main arm's benefit is by design: a phase serving a listed ambulance is exempt from the guard, so when the model picks it, it is applied.

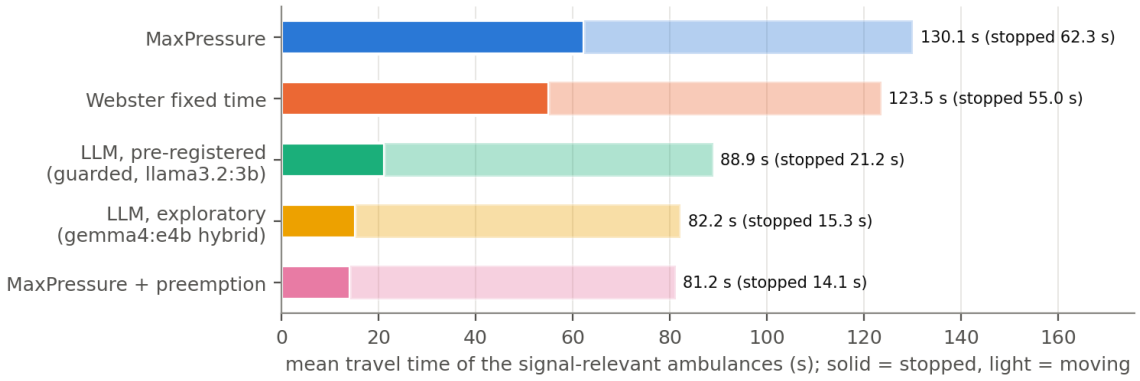


Figure 9 Travel time of the signal-relevant ambulances

Mean travel time of the signal-relevant ambulances, split into time stopped (solid) and time moving (light).

Normal traffic does not pay for the priority. Network ATT under the main arm is 116.4 s against MaxPressure's 115.7 s, a difference well inside the seed-to-seed spread (±2.3 s).

4.5 Model Size (Exploratory)

Exploratory, planned after the main results. With the controller fixed to the unguarded hybrid (hybrid_v3), only the model was changed, from 3B local models to a hosted 550 B mixture-of-experts model (Nemotron 3 Ultra [33], 55 B parameters active per token). Table 4.12 and Figure 10 give the results.

Table 4.12 Exploratory model-size study

Exploratory model-size study, Indian 2x2 peak, 3 seeds. "Follows pressure" = share of answers that are the highest-pressure option. Failed calls apply MaxPressure's choice.

Model	Total params (B)	ATT (s), mean ± std	ATT vs MaxPressure, mean (range over seeds)	Seeds worse than MaxPressure	Follows pressure	Answer differs from MaxPressure	Failed calls
qwen2.5:3b	3.1	174.4 ± 2.8	+51.0% (+47.6 to +53.5)	3 of 3	17%	83%	308 of 5229 (5.9%): reason_too_long 308
llama3.2:3b	3.2	209.6 ± 8.9	+81.4% (+74.1 to +86.1)	3 of 3	2%	99%	0 of 5393 (0.0%)
gemma4:e4b	7.5	113.7 ± 0.3	-1.5% (-2.2 to -0.5)	0 of 3	80%	21%	3 of 2541 (0.1%): reason_too_long 3
llama3.1:8b	8	117.1 ± 2.6	+1.4% (-1.4 to +4.3)	2 of 3	62%	43%	0 of 2207 (0.0%)
nemotron-3-ultra-550b	550	114.8 ± 3.6	-0.6% (-3.1 to +3.9)	1 of 3	99%	2%	39 of 2205 (1.8%): retries_exhausted 37, timeout 2
MaxPressure (baseline)	–	115.5 ± 0.9	+0.0%	–	–	–	–

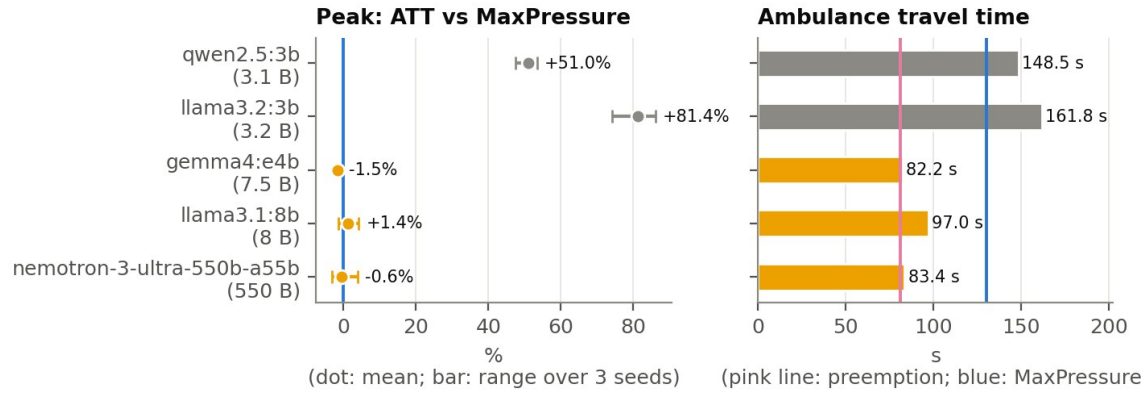


Figure 10 Exploratory: one hybrid controller with five models

Exploratory: the same hybrid controller with five models. Left: ATT relative to MaxPressure at peak (dot: mean; bar: range over seeds). Right: mean travel time of the 29 signal-relevant ambulances. Grey: models worse than MaxPressure on every seed.

1. **Below some size the LLM makes the hybrid controller worse.** With either 3B model the hybrid is far behind MaxPressure on every seed (ATT +51.0% and +81.4% at peak). Without the guard, their preference for keeping the current phase is applied.
2. **From `gemma4:e4b` upwards, ATT is MaxPressure's within seed noise.** `gemma4:e4b`, `llama3.1:8b` and the 550 B model are within a few percent of MaxPressure; only gemma4 is better on every peak seed.
3. **Following pressure grows with size, but not strictly with parameter count.** `gemma4:e4b` follows pressure in 80% of calls, `llama3.1:8b` in 62%, the hosted model in 99%: away from ambulances it nearly copies MaxPressure. Model family and training matter as much as size.
4. **Ambulances show the clearest effect of size** (Table 4.13). `gemma4:e4b` and the hosted model bring the signal-relevant ambulances through about as fast as preemption, without preempting: when an ambulance is in view they leave MaxPressure's choice and give it green.

Table 4.13 Exploratory: ambulances under each model

Exploratory: signal-relevant ambulances under each model (emergency scenario, 5 seeds), paired with MaxPressure and with preemption.

Controller / model	Signal-relevant ambulances finished	Travel time, mean (s)	Stopped time, mean (s)	vs MaxPressure, paired	vs MaxPressure + preemption, paired
qwen2.5:3b	29 of 29	148.5	77.5	+18.4 s (11 faster, 18 slower, of 29)	+67.3 s (3 faster, 26 slower, of 29)
llama3.2:3b	29 of 29	161.8	86.4	+31.7 s (13 faster, 16 slower, of 29)	+80.6 s (5 faster, 24 slower, of 29)
gemma4:e4b	29 of 29	82.2	15.3	-47.9 s (27 faster, 2 slower, of 29)	+1.1 s (16 faster, 12 slower, of 29)
llama3.1:8b	29 of 29	97.0	29.7	-33.1 s (23 faster, 6 slower, of 29)	+15.8 s (9 faster, 19 slower, of 29)
nemotron-3-ultra-550b	29 of 29	83.4	16.5	-46.7 s (28 faster, 1 slower, of 29)	+2.3 s (14 faster, 14 slower, of 29)
MaxPressure (baseline)	29 of 29	130.1	62.3	+0.0 s (0 faster, 0 slower, of 29)	+48.9 s (1 faster, 28 slower, of 29)
MaxPressure + preemption (baseline)	29 of 29	81.2	14.1	-48.9 s (28 faster, 1 slower, of 29)	+0.0 s (0 faster, 0 slower, of 29)

The hosted model's runs depended on an API that throttled the account heavily during the full runs: many calls were retried several times, and calls that ran out of retries were counted as failures and fell back to MaxPressure's choice (Section 6.4). Its prompts also leave the machine, so it is not an on-device option.

4.6 Camera Errors

The camera-error experiment repeated the main comparisons with the controllers' counts degraded by the detector's measured errors (Section 2.2.3), while the simulator stayed exact. Ambulance positions were not noised (GPS / V2X assumption). Table 4.14 and Table 4.15 give the main block and Figure 11 the paired changes.

Table 4.14 Camera errors, Indian 2x2 off-peak, 3 seeds

The paired change is against perfect counts on the same seed.

controller	level	ATT (s)	ATT vs perfect, paired (s; range; %)	queue / int. (veh)	waiting / veh (s)	finished trips
MaxPressure	perfect	88.1 ± 1.0	–	4.94 ± 0.23	29.8 ± 0.9	2332 ± 36
	junction	92.4 ± 0.5	+4.3 (+3.7 to +4.7); +4.8%	5.64 ± 0.16	33.9 ± 0.4	2323 ± 35
	crowded	95.4 ± 1.7	+7.3 (+6.4 to +8.8); +8.2%	6.15 ± 0.34	37.0 ± 1.5	2320 ± 38
Webster fixed-time	perfect	100.4 ± 4.1	–	6.82 ± 0.52	41.1 ± 3.9	2320 ± 36
	junction	100.4 ± 4.1	+0.0 (+0.0 to +0.0); +0.0%	6.82 ± 0.52	41.1 ± 3.9	2320 ± 36
	crowded	100.4 ± 4.1	+0.0 (+0.0 to +0.0); +0.0%	6.82 ± 0.52	41.1 ± 3.9	2320 ± 36
LLM arm (guarded llama3.2:3b)	perfect	89.0 ± 0.8	–	5.08 ± 0.18	30.6 ± 0.6	2327 ± 41
	junction	93.5 ± 0.6	+4.5 (+3.8 to +4.9); +5.1%	5.82 ± 0.21	35.0 ± 0.7	2321 ± 35
	crowded	95.3 ± 0.3	+6.4 (+5.5 to +7.5); +7.2%	6.13 ± 0.09	36.9 ± 0.4	2318 ± 35

Table 4.15 Camera errors, Indian 2x2 peak, 3 seeds

controller	level	ATT (s)	ATT vs perfect, paired (s; range; %)	queue / int. (veh)	waiting / veh (s)	finished trips
MaxPressure	perfect	115.5 ± 0.9	–	17.83 ± 0.32	54.1 ± 0.5	4576 ± 68
	junction	118.2 ± 1.8	+2.7 (+1.2 to +4.3); +2.3%	18.62 ± 0.21	56.5 ± 1.6	4584 ± 80
	crowded	120.6 ± 1.4	+5.1 (+3.5 to +7.7); +4.4%	19.37 ± 0.67	58.8 ± 1.4	4582 ± 84
Webster fixed-time	perfect	116.3 ± 4.6	–	18.27 ± 1.28	55.4 ± 4.0	4577 ± 87
	junction	116.3 ± 4.6	+0.0 (+0.0 to +0.0); +0.0%	18.27 ± 1.28	55.4 ± 4.0	4577 ± 87
	crowded	116.3 ± 4.6	+0.0 (+0.0 to +0.0); +0.0%	18.27 ± 1.28	55.4 ± 4.0	4577 ± 87
LLM arm (guarded llama3.2:3b)	perfect	116.1 ± 2.9	–	17.97 ± 0.85	54.5 ± 2.5	4582 ± 77
	junction	117.9 ± 1.7	+1.8 (-0.5 to +3.2); +1.6%	18.52 ± 0.28	56.2 ± 1.4	4571 ± 77
	crowded	121.7 ± 1.2	+5.6 (+3.6 to +9.6); +4.9%	19.69 ± 0.74	59.7 ± 1.3	4577 ± 81

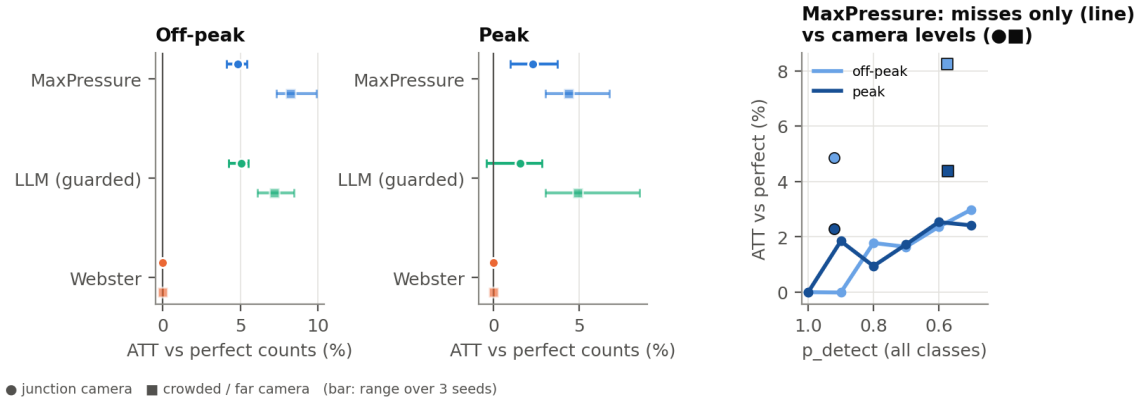


Figure 11 Paired ATT change under camera errors

Left and centre: paired change in ATT against perfect counts for each controller and camera level (bar: range over seeds). Right: MaxPressure with only missed vehicles (uniform detection probability, line) and with the camera levels (markers) at their effective detection rates.

- **Every count-based controller gets slower.** With the junction camera MaxPressure's ATT rises by 4.8% off-peak and 2.3% at peak, and with crowded / far detection by 8.2% and 4.4%; every seed is worse. The LLM arm degrades by about as much (junction +5.1% / +1.6%), which is expected: the guard's pressures come from the same noisy counts.
- **The Webster plan is unaffected**: it never reads counts, and its runs are identical at every level.
- **At peak the advantage of adaptive control disappears.** With perfect counts MaxPressure and Webster were level at peak; with junction-camera counts MaxPressure's mean ATT (118.2 s) falls behind Webster's (116.3 s). Off-peak MaxPressure stays ahead, with a smaller margin.

Missed vehicles alone cost little (Table 4.16). With half of all vehicles missed and no false detections, MaxPressure loses only 3.0% off-peak and 2.4% at peak. When every vehicle is missed with the same probability, every phase's expected pressure shrinks by the same factor, so the ranking is unchanged on average; only the added randomness costs. The junction camera is different: it detects 92% of vehicles, yet off-peak it costs +4.8%, whereas detecting 90% of vehicles with nothing else wrong costs 0.0%. What the camera adds are phantom vehicles: about 0.7 per road and snapshot, all counted as stopped.

Table 4.16 MaxPressure with uniform detection and no false detections

MaxPressure with every class detected with the same probability, no false detections and exact labels (mean ± std over 3 seeds, change against perfect counts in brackets).

p_detect (all classes, no false detections)	Off-peak: MaxPressure ATT (s)	Peak: MaxPressure ATT (s)
1.00	88.1 ± 1.0	115.5 ± 0.9
0.90	88.1 ± 0.8 (-0.0%)	117.7 ± 1.5 (+1.8%)
0.80	89.7 ± 0.3 (+1.8%)	116.6 ± 1.3 (+0.9%)
0.70	89.6 ± 0.7 (+1.6%)	117.5 ± 1.2 (+1.7%)
0.60	90.2 ± 1.5 (+2.4%)	118.5 ± 1.4 (+2.5%)
0.50	90.8 ± 0.8 (+3.0%)	118.3 ± 0.7 (+2.4%)

Decomposition (post-hoc). To test that explanation, the junction level was re-run with one error component at a time (Table 4.17, Figure 12). Off-peak, false detections alone cost +3.8% of the +4.8%, misses +0.8% and class confusion nearly nothing, and the three add up to the whole effect. At peak no single component explains the +2.3%: the components sum to only +0.4%, so either they interact near capacity or part of the peak effect is chance; three seeds cannot tell which. Class confusion alone has no measurable effect because these controllers read class-free counts.

Table 4.17 Post-hoc decomposition of the junction-camera errors

Post-hoc decomposition of the junction-camera errors (MaxPressure, 3 seeds): one component switched on at a time. Each component draws its own random numbers, so they need not add up.

junction-camera errors	Off-peak: ATT vs perfect, paired (s; range; %)	Peak: ATT vs perfect, paired (s; range; %)	realised detection rate	phantoms per road and snapshot
(a) misses only	+0.7 (-0.6 to +1.7); +0.8%	+0.7 (+0.3 to +1.2); +0.6%	0.920	0.000
(b) false detections only	+3.4 (+2.8 to +3.7); +3.8%	-0.3 (-1.8 to +1.0); -0.3%	1.000	0.688
(c) class confusion only	+0.1 (-1.1 to +1.0); +0.2%	+0.1 (-1.6 to +1.4); +0.1%	1.000	0.000
(d) all (main batch junction level)	+4.3 (+3.7 to +4.7); +4.8%	+2.7 (+1.2 to +4.3); +2.3%	0.921	0.690
sum of (a) + (b) + (c)	+4.3; +4.8%	+0.4; +0.4%		

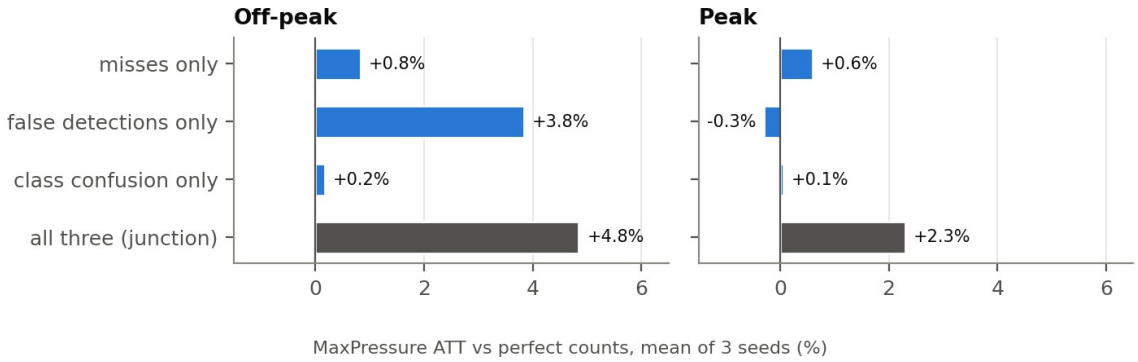


Figure 12 Post-hoc decomposition of MaxPressure's ATT change

Post-hoc decomposition: MaxPressure's ATT change with one junction-camera error component at a time.

Ambulances under camera errors (Table 4.18). Preemption keeps its advantage (81.2 s → 84.5 s) because it reads ambulance positions from the separate channel. The LLM arm's ambulances are 11.0 s slower on average with junction-camera counts, but individual ambulances move by tens of seconds in either direction between paired runs, so this is not distinguishable from zero with 29 ambulances.

Table 4.18 Emergency scenario under junction-camera counts

Emergency scenario under junction-camera counts (5 seeds). Ambulance positions are not noised.

controller	level	ATT (s)	ATT vs perfect, paired (s; range; %)	ambulances finished	ambulance travel time (s)	ambulance vs perfect, paired (s)
MaxPressure	perfect	115.7 ± 2.3	–	29 / 29	130.1	–
	junction	117.6 ± 2.4	+1.8 (-1.9 to +7.1); +1.6%	29 / 29	135.9	+5.8 (n 29; 14 slower, 13 faster)
Webster fixed-time	perfect	115.2 ± 2.7	–	29 / 29	123.5	–
	junction	115.2 ± 2.7	+0.0 (+0.0 to +0.0); +0.0%	29 / 29	123.5	+0.0 (n 29; 0 slower, 0 faster)
MaxPressure + preemption	perfect	116.0 ± 0.7	–	29 / 29	81.2	–
	junction	117.2 ± 2.3	+1.1 (-0.8 to +5.1); +1.0%	29 / 29	84.5	+3.3 (n 29; 16 slower, 12 faster)
LLM arm (guarded llama3.2:3b)	perfect	116.4 ± 3.1	–	29 / 29	88.9	–
	junction	120.2 ± 1.8	+3.8 (-0.3 to +9.6); +3.3%	29 / 29	99.9	+11.0 (n 29; 17 slower, 11 faster)

4.7 Detector

Operating point. A detector outputs boxes with confidence scores; a controller needs a yes/no decision per box. The confidence threshold was chosen on the validation

split as the one that maximises the mean F1 score over the 7 classes: 0.3 (mean F1 0.620). The F1 curve is flat around this value, so the exact choice matters little.

Matching. On each test image, detections (sorted by confidence) are matched greedily to the unmatched ground-truth box with the highest IoU of at least 0.5, ignoring the class. A match with the same class is a true positive; a match with another class is a **confusion**; an unmatched detection is a **false detection**; an unmatched ground-truth box is a **miss**. Confidence intervals come from 1,000 bootstrap resamples of whole drives, not frames, so that correlated frames do not make them look tighter than they are.

Results. On the test split the detector reaches mAP50 0.563 and mAP50-95 0.375 (validation: 0.602 and 0.403). Table 4.19 gives the per-class numbers. Precision is high (few false detections) and recall is moderate: the detector undercounts.

Table 4.19 Detector on the test split, per class

Detector on the test split (official IDD validation set), per class. mAP from Ultralytics; precision, recall, miss rate and false detections from our matching at confidence ≥ 0.3. Recall = 1 − miss rate − confusion rate.

Class	Boxes	mAP50	mAP50-95	Precision	Recall	Miss rate	False det. / image
car	24,843	0.637	0.442	83.9%	54.9%	43.2%	0.222
motorcycle	25,489	0.630	0.373	81.9%	56.2%	43.4%	0.271
autorickshaw	7,782	0.652	0.461	82.0%	56.0%	40.2%	0.070
bus	4,915	0.664	0.502	82.3%	56.9%	36.8%	0.039
truck	7,077	0.576	0.412	72.9%	48.3%	46.3%	0.079
bicycle	569	0.339	0.193	61.0%	24.8%	55.4%	0.006
person	18,078	0.444	0.241	79.1%	35.5%	63.6%	0.149
all	88,753	0.563	0.375				

Most misses are small, distant objects (Figure 13, Table 4.20). Small boxes are most of the ground truth, and the detector misses 59–76% of them, against 1–22% of large boxes. This is the cost of running at 640 pixels named above. One camera group dominates the test set: the `highquality_16k` camera supplies 3,633 images but 64,381 of the boxes, 74% of them small. The overall per-class rates therefore mostly describe crowded scenes with distant objects. Confusions are few and plausible: bicycles are labelled motorcycles, and buses, trucks and auto-rickshaws are confused with each other

and with cars. `person` has the highest miss rate, partly because pedestrians are small and possibly because the model was taught not to fire on riders.

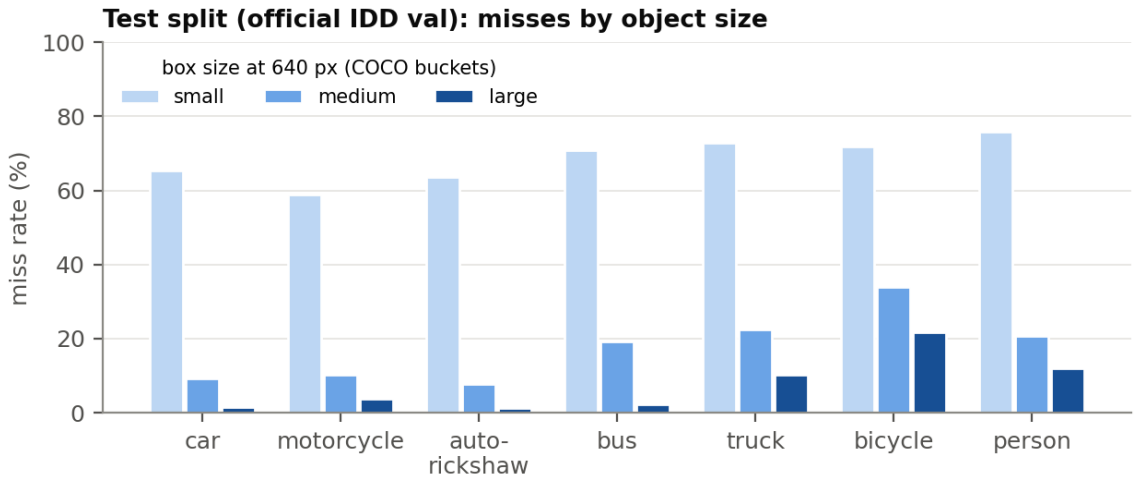


Figure 13 Miss rate by object size on the test split

Sizes are COCO's buckets measured in the 640-pixel image.

Table 4.20 Miss rate by object size

Miss rate by object size (number of ground-truth boxes in brackets), test split.

class	Small: miss rate (boxes)	Medium	Large
car	65.4% (15,489)	9.1% (6,065)	1.3% (3,289)
motorcycle	59.0% (17,565)	10.1% (6,250)	3.8% (1,674)
autorickshaw	63.6% (4,646)	7.8% (2,106)	1.1% (1,030)
bus	70.8% (2,199)	19.2% (1,141)	2.1% (1,575)
truck	73.0% (3,712)	22.4% (1,820)	10.2% (1,545)
bicycle	71.9% (334)	33.8% (198)	21.6% (37)
person	75.7% (14,180)	20.7% (3,288)	12.0% (610)

4.8 Dashboard

The dashboard (Section 2.1.7) presents these results; its four pages are shown in Figure 14, Figure 15, Figure 16 and Figure 17. The screenshots were taken by a script with the dashboard reading the committed result files (`docs/screenshots/`).

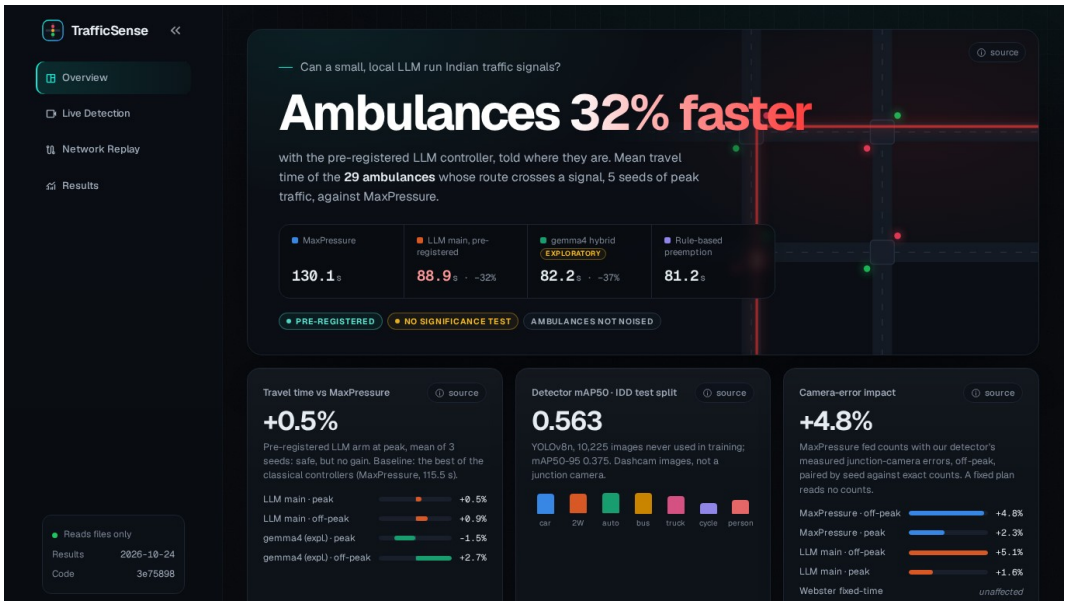


Figure 14 Dashboard, Overview page

Dashboard, Overview page. The headline is the pre-registered arm's ambulance result; exploratory numbers are labelled as such.

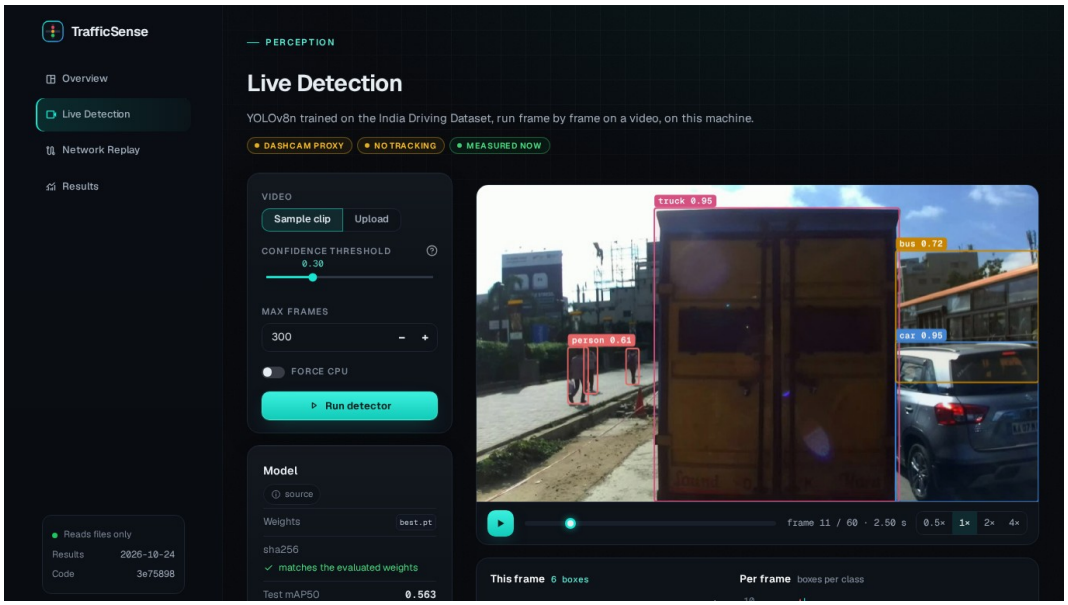


Figure 15 Dashboard, Live Detection page

Dashboard, Live Detection page: the trained detector on a clip of IDD test frames, with boxes, per-class counts and the measured speed on the GTX 1650.

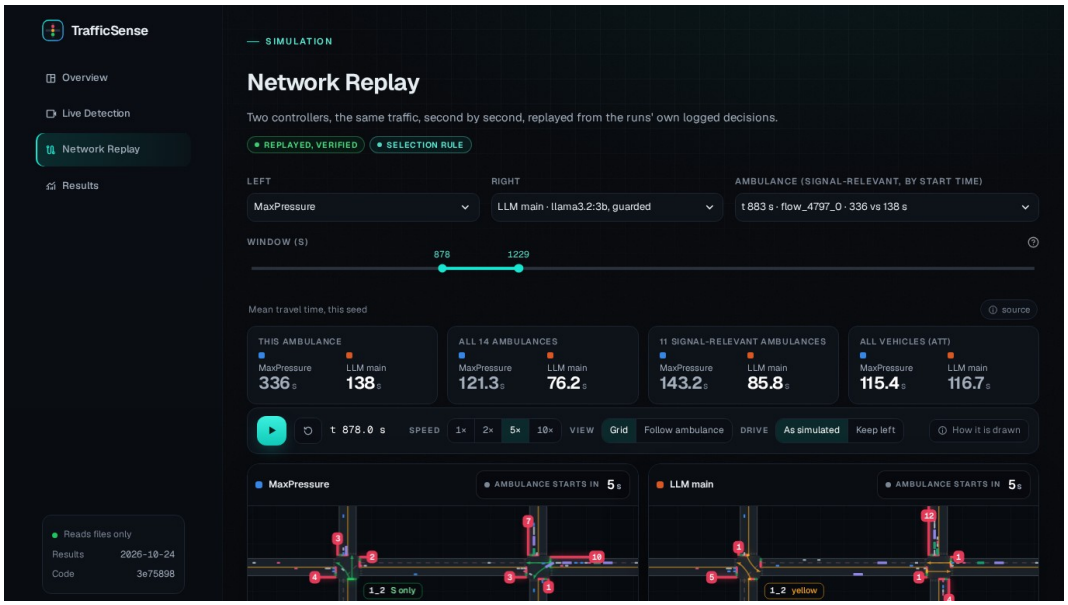


Figure 16 Dashboard, Network Replay page

Dashboard, Network Replay page: MaxPressure (left) and the pre-registered LLM arm (right) on the same traffic, one ambulance's trip. Replays re-drive the simulator with each run's logged decisions and are kept only if they reproduce the run's metrics exactly.

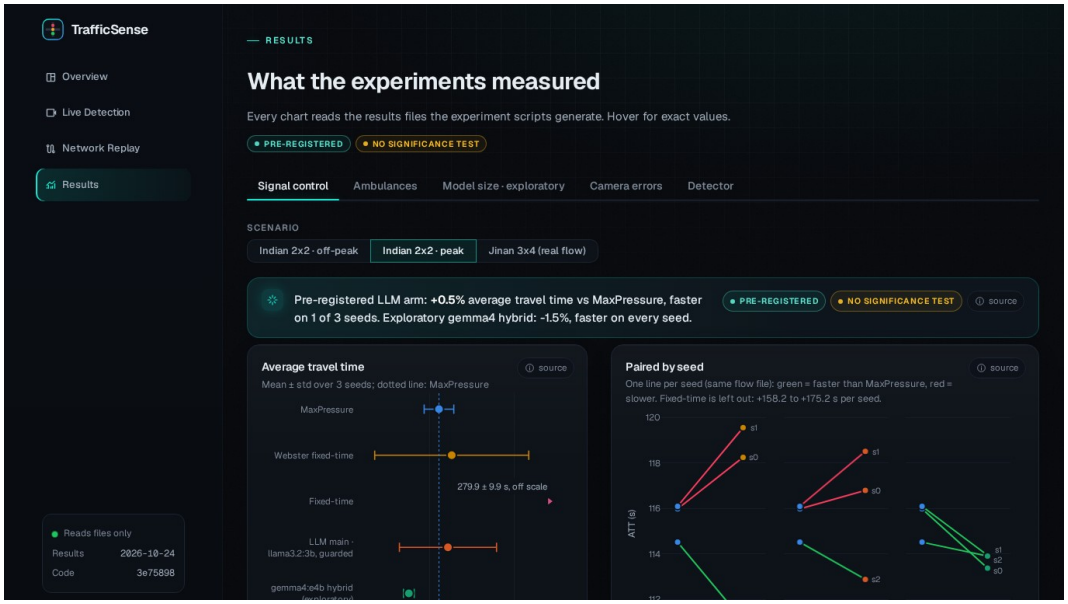


Figure 17 Dashboard, Results page

Dashboard, Results page, Signal control tab (Indian 2x2 peak). Average travel time of each controller and the paired comparison by seed, labelled as pre-registered and without a significance test; the other tabs show the ambulance, model-size (exploratory), camera-error and detector results.

4.9 Discussion

4.9.1 Why the LLM Agents Match MaxPressure but Do Not Beat It

Three observations together explain the main result.

First, an LLM that reads pressure correctly becomes MaxPressure. In the hybrid controllers the LLM is shown each option's pressure and queue, the same information MaxPressure uses. The largest model tested, the hosted Nemotron 3 Ultra, chose the highest-pressure option in 99% of its calls, and its ATT is within seed noise of MaxPressure's (Section 4.5). This is the expected end point: for steady demand on a small grid, serving the largest pressure is close to the best a one-step rule can do, and the extra information in the prompt, the neighbours' queues, rarely points to a better phase. To beat MaxPressure an agent must use information MaxPressure does not have, or plan over more than one decision. Neither was asked of the agents in the pre-registered design.

Second, small models misread the task. With prompts v2 and guided, the 3B models treated low pressure as good and kept the current phase most of the time, and their reasons said so in words (Section 4.2). The v3 fix, which states which direction of pressure is better, changed `gemma4:e4b` but not the 3B models. A model of this size follows the surface of the prompt ("keep", "low queue") more than its stated rule. Without a guard their choices are applied and the controller is far worse than MaxPressure (Table 4.12).

Third, the guard converts a poor model into MaxPressure with noise. The pre-registered controller's answers differ from MaxPressure's in about nine of ten calls, but the guard replaces most of them, and the applied phase differs in about one in ten. What remains are choices within 20% of the top pressure. Off-peak, when queues are short, such a choice gives green to a slightly smaller queue and costs a little; this is the small, consistent deficit seen on every off-peak seed. The guard therefore made the LLM safe to deploy, not useful.

The exploratory `gemma4:e4b` arm follows pressure in most calls and departs from MaxPressure in about one call in five at peak, where it was faster on every seed. We have not identified the mechanism. One hypothesis is that its departures more often keep a phase that is still discharging a long queue, avoiding a transition; this was not tested, and with three seeds the gain itself needs confirmation.

4.9.2 Why the LLM Agents Help Ambulances

The ambulance experiment is the one case where the agent has information MaxPressure lacks: the prompt lists each approaching ambulance, its distance and the phases that serve it. A one-sentence instruction was enough for the pre-registered arm to cut ambulance travel time by 31.7% compared with MaxPressure, mostly by removing time stopped at a red. Two design choices helped: the phase that serves the ambulance is always among the options, and it is exempt from the guard. Normal traffic was not measurably slowed, because ambulances are rare and each priority decision affects one intersection for a short time.

A hand-written preemption rule did slightly better. This is the honest comparison for a deployment decision: for a single, well-defined rule like "give green to the ambulance", a rule is simpler, faster and verifiable. The case for an LLM agent is flexibility, not performance. The same mechanism, a sentence in the system prompt and a line in the observation, can carry rules that are awkward to hard-code: a road closure, a school zone at certain hours, a police instruction, priority for a bus that is late. This project tested only the ambulance case.

4.9.3 What the Camera Study Means

Adaptive control is only as good as its counts. With counts degraded by the errors of a real detector, every count-based controller lost a few percent of travel time, and at peak its advantage over a well-timed fixed plan, already small with exact counts, disappeared. Three lessons follow for a deployment.

1. **False detections hurt more than misses.** Missing vehicles uniformly scales every pressure down and barely changes the ranking. A phantom vehicle adds pressure where there is none and changes the decision directly. For signal control a detector's precision matters more than its recall, the opposite of the usual emphasis, and temporal smoothing or tracking that rejects one-frame detections should help (each snapshot here is independent, the worst case).
2. **Camera placement matters.** The junction level, which assumes queued vehicles are close enough to be medium or large in the image, costs about half as much as the crowded / far level.

3. **A fixed plan is a robust fallback.** Webster's plan does not depend on counts at all. Where detection is unreliable, at night or in heavy rain, a controller should fall back to a good fixed plan rather than act on bad counts.

These conclusions rest on a proxy: the detector's errors on dash-camera images. The same detector on a pole-mounted camera could be better (closer, less occlusion) or worse (unfamiliar viewpoint).

4.9.4 Relation to the Base Paper

CoLLMLight reports that cooperative LLM agents outperform existing methods. Our results neither confirm nor contradict that claim, because the settings differ in ways that matter: CoLLMLight fine-tunes its model in several stages and uses an asynchronous reasoning architecture, while we used small models as released; we compared against MaxPressure and Webster tuned on the test networks; and we found that CoLLMLight's own fixed-time baseline does not serve north–south traffic. Its metric definitions also differ from CityFlow's, so numbers cannot be compared directly. What our results do show is that, without fine-tuning, a 3–4 B model is not enough to improve on a tuned MaxPressure, and that the clearest value of an LLM agent appears when it is given information a classical controller cannot use.

4.10 Threats to Validity

Internal validity. Three seeds per scenario (five with ambulances) are few, and no significance test was pre-registered; differences of one or two percent are within seed noise and are reported as such. LLM runs are not bit-for-bit repeatable on the GPU, although every decision is logged and replayable. Prompt v3 was a post-hoc fix, the `gemma4:e4b` arm and the model-size study were added after the plans were fixed, and the camera decomposition was post-hoc; all are labelled. The hosted model's runs were affected by heavy API throttling, which turned some decisions into MaxPressure fallbacks.

Construct validity. ATT counts time spent waiting to enter the network, which penalises controllers that block entries; this is intended but makes ATT differ from definitions used elsewhere, including CoLLMLight's (Section 2.2.1). The ambulance

metric assumes that nobody yields and that ambulances cannot overtake, so it measures only what the signal can do. The camera layer draws independent snapshots and does not model occlusion between lanes or ambulance detection.

External validity. CityFlow is lane-based and cannot represent lane-free Indian driving (Section 2.2.1). The Indian network is a 2 × 2 grid with synthetic, steady demand; its vehicle mix comes from one city in one year; and the turning shares are assumptions. Jinan provides one real-world flow. Latencies are specific to a GTX 1650. The results show what a small local model does on this network; they do not show how a deployed system would perform on a real junction.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project built and evaluated TrafficSense, a multi-agent traffic-signal controller in which each intersection is run by a local LLM agent that observes its own and its neighbours' state, reasons and acts, adapted from CoLLMLight to a 4 GB laptop GPU. The objectives set in Section 1.1 were met as follows.

1. **A validated pipeline.** Our CityFlow environment reproduces the CoLLMLight authors' own baseline results when driven the same way (throughput and queue exactly, waiting and travel times to within the simulator's run-to-run noise), and the validation exposed a bug in their fixed-time baseline.
2. **An Indian test network with documented parameters**, calibrated against measured capacity, and **baselines tuned before the comparison**: MaxPressure with a 15 s minimum green, and a Webster plan computed from perfect counts.
3. **A pre-registered comparison.** The selected LLM controller, `llama3.2:3b` guided and guarded by MaxPressure, performs within 0.5–0.9% of MaxPressure's travel time and runs in real time, but does not improve on it. The reason is clear from its decision logs: small models misread pressure, and the guard, which makes them safe, also makes them MaxPressure.
4. **A clear benefit for emergency vehicles.** Told about approaching ambulances, the agent cut their mean travel time from 130.1 s to 88.9 s, close to a hand-written preemption rule, without slowing other traffic.
5. **A measured camera-error study.** A YOLOv8n detector trained on IDD reaches mAP50 0.563 on held-out drives. Its measured errors cost count-based controllers a few percent of travel time, mostly through false detections, and remove adaptive control's small lead over a fixed plan at peak.
6. **A dashboard and a report generated from the data**, with every number checked against the result files.

The overall answer to the question in Chapter 1 is therefore modest and, we believe, useful: a small local LLM does not yet beat a well-tuned classical controller on routine traffic, but as a guarded layer on top of one it is safe, and it adds value where the situation is described in words rather than in queue counts.

5.2 Future Work

- **Memory and tool-using agents.** The agents here see one snapshot and answer once. An agent that remembers its recent decisions and their effects, and can call tools (MaxPressure's ranking, a short look-ahead simulation, the queue history of an approach), could plan over several decisions, which is where an agent might beat a one-step rule.
- **Fine-tuning small models.** CoLLMLight's training stages (reasoning refinement and policy refinement with simulation feedback) were not reproduced. Fine-tuning a 3B model on decisions scored by the simulator may fix the pressure misreading that prompting could not.
- **Real junction camera footage.** Collecting and labelling video from a fixed camera at an Indian junction would replace the dash-camera proxy with the true error, and would allow tracking-based counting that rejects one-frame false detections.
- **Larger networks and real demand.** Hangzhou and New York networks are available in the same format; counts from a real Indian corridor would replace the synthetic demand. A simulator with sub-lane movement would represent two-wheeler filtering.
- **Faster inference.** A larger GPU or a smaller, faster model would make the stronger `gemma4:e4b`-class controller real-time and allow more seeds for a confirmatory test of its peak gain.
- **Field testing.** A first deployment would run the agent in shadow mode beside an existing controller, logging what it would have done, before it is allowed to switch a real signal. Ambulance positions would come from a GPS or V2X feed, as assumed here.

CHAPTER 6
APPENDIX

The appendix gives what is needed to check or repeat the work: the configuration in force (Section 6.1), the prompts (Section 6.2), the reproduction steps (Section 6.3), the full result tables behind the summaries of Chapter 4 (Section 6.4), the abbreviations (Section 6.5) and listings of the key source files (Section 6.6).

6.1 Configuration

Every setting used in the experiments is read from YAML files in `configs/`; no setting is typed into code. The listings below are copied from the files by the report's build script, so they are the settings actually in force.

6.1.1 Main Configuration

Excerpt of `configs/default.yaml`:

```
sim:
  steps: 3600          # default run length (CLAUDE.md)
  interval: 1.0        # seconds per simulator step
  seed: 0
  threads: 1
  lane_change: false   # CityFlow laneChange; CoLLMLight runs with true
signal:
  min_green: 10        # s
  max_green: 60        # s
emergency:
  detection_range_m: 150   # ASSUMPTION: an ambulance on an incoming lane
within this distance of
                           # the stop line is detected (lanes are 176-188
m long; at 12.5 m/s an
                           # ambulance covers 150 m in 12 s, enough for a
5 s decision step plus
                           # the 5 s transition)
controllers:  # excerpt
  fixedtime:
    green_time: 30     # s per green phase
    phases: paired     # the 4 opposing-approach phases that serve every
non-right movement once
  fixedtime_webster:      # 4 paired phases; cycle and splits by Webster's
method (control/webster.py)
    cycle_min: 60          # s
    cycle_max: 180         # s
    demand_window_s: 3600  # lane demand = vehicles in the flow file
```

```
starting in [0, window)
    measurement_reps: 30   # random-order queues averaged for saturation
flow / start-up loss
    measurement_seed: 0    # seed of those queue orders (not the run seed)
  maxpressure:
    decision_interval: 5   # s between re-evaluations
    min_green: 15          # s, controller's own minimum green (>=
signal.min_green).
                           # Chosen by
scripts/tune_maxpressure_min_green.py (docs/baselines.md)
    phases: all            # all | paired (CoLLMLight's MaxPressure uses
the 4 paired phases)
    tie_break: keep_current  # keep_current | lowest_index (CoLLMLight:
np.argmax -> lowest)
  preemption_mp:           # MaxPressure + rule-based ambulance preemption
(control/preemption.py)
    decision_interval: 5   # same timing as the default MaxPressure and
the LLM benchmark
    min_green: 15
    tie_break: keep_current
  llm_guided_mp_v3:
    mode: cooperative
    prompt: guided_v3
    options: 2
    tie_break: keep_current
  llm_guarded_mp:          # llm_guided_mp_v3 + guard: the LLM's phase is
applied only if its pressure
    mode: cooperative      # >= 0.8 x the top phase's (control/guided.py),
else MaxPressure's choice
    prompt: guided_v3
    options: 2
    tie_break: keep_current
    guard_ratio: 0.8
llm:
  provider: ollama
  host: http://localhost:11434
  endpoint: /api/chat  # native endpoint, structured output via `format`
(JSON schema)
  model: null          # chosen after scripts/benchmark_llm.py; must be
set for LLM runs
  timeout_s: 60        # per call; a timeout counts as a failed decision
  temperature: 0
  seed: 0              # LLM sampling seed, fixed (recorded in every
summary.json)
  num_ctx: 2048        # context window (tokens). Prompts are ~650; at
4096 llama3.2:3b no longer fits
                       # fully in the GTX 1650's VRAM. A prompt that could
be truncated raises.
  num_predict: 128     # cap on generated tokens; the JSON answer needs
about 40
  keep_alive: 30m      # keep the model loaded between calls
  decision_interval: null  # s; decided after scripts/benchmark_llm.py
  min_green: null          # s; LLM controller's own min green (>=
signal.min_green)
```

```
  max_consecutive_failures: 20  # this many failed calls in a row stops
the run with an error
```

6.1.2 Indian 2x2 Scenario

Excerpt of `configs/scenarios/indian_2x2.yaml`; the vehicle-type parameters are given in Table 2.5.

```
roadnet:
  file: data/scenarios/indian_2x2/roadnet.json
  rows: 2
  cols: 2
  spacing_m: 200            # between intersections (Phase 2 spec)
  boundary_road_m: 200      # entry/exit roads to the network edge
(ASSUMPTION: same as spacing)
  lane_width_m: 3.0         # Phase 2 spec (~3.0 m)
  lanes:                    # per direction, one lane per turn (CityFlow
generator layout)
    left: 1
    straight: 1
    right: 1
  lane_max_speed_mps: 12.5  # 45 km/h; Phase 2 spec range 11-13 m/s
(ASSUMPTION within it)
  intersection_width_m: 12  # distance from the centre to the stop line
(ASSUMPTION, see doc)
  # Phases come from the generator: phase 0 = 5 s transition (right turns
only), phases 1-8 green.
turn_probabilities:       # ASSUMPTION
  go_straight: 0.60
  turn_right: 0.25        # Indian free left
  turn_left: 0.15         # Indian right turn
entry_weights:            # ASSUMPTION
  W: 1.0
  E: 1.0
  N: 0.6
  S: 0.6
demand_levels:
  offpeak:
    veh_per_hour: 2375
  peak:
    veh_per_hour: 4750
emergency:
  vehicle_type: ambulance
  veh_per_hour: 6           # ASSUMPTION (Phase 3b spec "~6/hour", whole
2x2 network, both levels)
  seed_offset: 1000         # ASSUMPTION: any fixed offset; ambulance RNG
seed = flow seed + 1000
```

6.2 Prompts

The system prompts below are rendered by the controllers' own code (`src/trafficsense/llm/prompt.py`) with the timing used in the experiments. The LLM must answer with JSON that matches a schema listing the allowed phase numbers; the reason is checked for length after parsing.

6.2.1 Prompt v1 (First Round)

```
You control the traffic signal of one intersection in a city road network
with mixed Indian traffic (two-wheelers, auto-rickshaws, cars, buses,
trucks).
At each decision you choose which green phase to show next. Goal: keep
vehicles moving and queues short across the whole network, so total travel
time is low.
Rules of the signal:
- Choosing the current phase keeps it green. Choosing another phase first
shows a 5 s transition in which only free turns move, so switching too
often wastes green time.
- A green phase cannot be ended before 15 s and is ended automatically at
60 s.
Data you get for each movement: queue = stopped vehicles waiting at the
stop line; moving = vehicles driving towards the stop line; downstream =
stopped vehicles on the exit road (a full exit road blocks the movement).
Answer with JSON only: {"phase": <one of the allowed phase ids>, "reason":
"<at most 25 words>"}.
```

The v1 user message listed, for every phase, each movement's queue, moving and downstream counts, the vehicle types per approach, and the neighbours.

6.2.2 Prompt v2

```
You pick the next green phase for one traffic signal in an Indian city.
Goal: short queues and low travel time network-wide.
Changing phase starts a 5 s transition in which only free turns move. A
green lasts 15 to 60 s.
queue = stopped vehicles a phase serves; pressure = queue minus stopped
vehicles on their exit roads.
Reply JSON: {"phase": <allowed id>, "reason": "<at most 15 words>"}
```

6.2.3 Prompt v3 (v2 with Two Lines Changed)

```
You pick the next green phase for one traffic signal in an Indian city.
Goal: short queues and low travel time network-wide.
Changing phase starts a 5 s transition in which only free turns move. A
green lasts 15 to 60 s.
queue = stopped vehicles a phase serves; pressure = vehicles waiting to go
minus vehicles already stopped on their exit roads. HIGHER pressure = the
phase needs green MORE.
```

```
Reply JSON: {"phase": <allowed id>, "reason": "<short reason>"}. The
reason must be 15 words or fewer.
```

6.2.4 Guided Prompt (Hybrid Arm)

```
You help a MaxPressure controller run one traffic signal in an Indian
city. Goal: short queues and low travel time network-wide.
MaxPressure ranks phases by pressure = stopped vehicles a phase serves
minus stopped vehicles on their exit roads; queue = stopped vehicles a
phase serves. Choose one of the offered phases; neighbour data may help.
Changing phase starts a 5 s transition in which only free turns move. A
green lasts 15 to 60 s.
Reply JSON: {"phase": <offered id>, "reason": "<at most 15 words>"}
```

6.2.5 Guided Prompt v3 (hybrid_v3 and the Pre-Registered Guarded Arm)

```
You help a MaxPressure controller run one traffic signal in an Indian
city. Goal: short queues and low travel time network-wide.
MaxPressure ranks phases by pressure = vehicles waiting to go minus
vehicles already stopped on their exit roads; HIGHER pressure = the phase
needs green MORE; queue = stopped vehicles a phase serves. Choose one of
the offered phases; neighbour data may help.
Changing phase starts a 5 s transition in which only free turns move. A
green lasts 15 to 60 s.
Reply JSON: {"phase": <offered id>, "reason": "<short reason>"}. The
reason must be 15 words or fewer.
```

In the emergency scenarios one sentence is added:

```
You help a MaxPressure controller run one traffic signal in an Indian
city. Goal: short queues and low travel time network-wide.
MaxPressure ranks phases by pressure = vehicles waiting to go minus
vehicles already stopped on their exit roads; HIGHER pressure = the phase
needs green MORE; queue = stopped vehicles a phase serves. Choose one of
the offered phases; neighbour data may help.
Changing phase starts a 5 s transition in which only free turns move. A
green lasts 15 to 60 s.
Ambulances, if listed, must get through quickly: give green to a phase
that serves them.
Reply JSON: {"phase": <offered id>, "reason": "<short reason>"}. The
reason must be 15 words or fewer.
```

6.2.6 An Example User Message

The user message of the guided prompts at a real simulator state: Indian 2x2 peak with ambulances, seed 2, `intersection_2_2` at t = 265 s, under MaxPressure control up to that moment. MaxPressure ranks phase 2 (S+N straight) first, but an ambulance 70 m from the stop line is served by phases 6, 1, so a phase that serves it is among the options.

```
Phase 1 (W+E straight) green for 15 s.
Options, best MaxPressure rank first (phase: pressure, queue):
```

```
2 S+N straight: 3, 4
3 W+E cross-turn: 3, 4
6 E only: -2, 3
1 W+E straight (keep current): -4, 0
Ambulances within 150 m:
E straight, 70 m from the stop line; served by phase 6, 1
Neighbours:
W: phase 1 (W+E straight); 9 stopped toward us, 7 on green; 4 stopped on
our road to it
S: phase 5 (W only); 0 stopped toward us, 0 on green; 1 stopped on our
road to it
```

6.2.7 A Logged Decision

One decision of the pre-registered arm, as logged in `decisions.jsonl` (selected keys): the first decision of its emergency run (seed 0) in which the LLM's choice differed from MaxPressure's and served an ambulance. The model's reason still mentions "low pressure", but its choice gives the ambulance green; because that phase serves a listed ambulance, the guard does not apply. The guard's pressures are those shown in the prompt, which leave out the free turns.


```
{
 "step": 1980,
 "intersection": "intersection_1_2",
 "current_phase": 5,
 "time_in_phase": 15,
 "allowed": [1, 2, 5, 8],
 "mp_ranking": [1, 2, 8, 4, 6, 7, 5, 3],
 "mp_choice": 1,
 "ambulances": [{"vehicle": "flow_4668_0", "approach": "N", "distance_m":
44.2, "movement": "N cross-turn", "phases": [8, 4]}],
 "latency_s": 1.42,
 "prompt_tokens": 340,
 "server_model": "llama3.2:3b",
 "raw": "{\"phase\": 8, \"reason\": \"Low pressure, serving ambulances
within 150m\"}",
 "llm_phase": 8,
 "guard_threshold": 2.4,
 "guard_llm_pressure": 3,
 "guard_override": false,
 "guard_ambulance_exempt": true,
 "phase": 8
}
```

6.3 Reproducibility

6.3.1 How the Results Are Kept Honest

- **Provenance in every run.** Each run writes a folder `<time>_<scenario>_<controller>_s<seed>/` with `summary.json` (metrics plus controller, parameters, model as reported by the server, provider, seed, scenario, run length, git commit and whether the tree was clean, time, and for LLM runs the calls, failures by type and latency), `metrics.csv` (per-second metrics) and, for LLM runs, `decisions.jsonl` (every decision with the raw answer).
- **Generated documents.** Every table in the project's documents is generated from the result files by a script, and each script has a `--check` mode that fails if the document and the data disagree.
- **This report.** Every table in this report is generated from the result files or copied from a checked document by `scripts/make_report.py`. Every other number taken from the data carries an invisible tag naming its source (a value in a result file, or a cell of a checked table), and `make_report.py --check` fails if any shown number differs from its source at the precision shown, or if the text contains an untagged number that is not a year, a reference, a small count or a product name. The figures are drawn from the result files, and the values each one plots are stored beside it and checked. `scripts/build_report.py` refuses to build the Word and PDF files unless that check passes; the PDF is LibreOffice's conversion of the Word file itself, and `scripts/check_report_format.py` then checks the Word file against the college format.

6.3.2 Steps

The project runs in WSL2 Ubuntu 22.04 with Python 3.10. The commands below are run from the repository root in its virtual environment; the batch scripts are resumable and skip finished runs.

```
# environment (exact versions: requirements-lock.txt)
python3.10 -m venv .venv && source .venv/bin/activate
git clone https://github.com/cityflow-project/CityFlow.git ../CityFlow
git -C ../CityFlow checkout 81ee0f4 && pip install ../CityFlow
pip install torch==2.14.1 torchvision==0.29.1 --index-url
```

```
https://download.pytorch.org/whl/cu126
pip install -e ".[dev,dashboard,perception]" -c requirements-lock.txt
scripts/fetch_collmlight_data.sh                      # CoLLMLight
roadnets and flows
curl -fsSL https://ollama.com/install.sh | sh         # or Ollama for
Windows (README)
ollama pull llama3.2:3b && ollama pull gemma4:e4b && ollama pull
qwen2.5:3b

# Sections 2.2.1, 2.2.2 and 4.1: validation, scenario, baselines
python scripts/validation/run_collmlight_baseline.py --model Fixedtime --
dataset synthetic_4x4 --seed 0
python scripts/validation/run_matched.py --scenario synthetic_4x4 --
profile replica_fixedtime --engine-seed 47
python scripts/validation/make_tables.py
python scripts/generate_indian_roadnet.py
python scripts/calibrate_indian_demand.py --rates 1000 2000 3000 4000 4500
4750 5000 5250 5500 5750 6000 7000 8000 10000 12000 14000
python scripts/generate_indian_flow.py --level all --seed 0 1 2
python scripts/tune_maxpressure.py && python
scripts/tune_maxpressure_min_green.py
python scripts/run_indian_experiments.py && python
scripts/make_indian_tables.py

# Sections 4.2 to 4.4: LLM benchmark, full experiments, ambulances
python scripts/benchmark_llm.py --arm v3 && python
scripts/benchmark_llm.py --arm hybrid_v3
python scripts/benchmark_llm.py --arm guarded_v3
python scripts/generate_indian_flow.py --level all --seed 0 1 2 --
emergency
python scripts/generate_indian_flow.py --level peak --seed 3 4 && python
scripts/generate_indian_flow.py --level peak --seed 3 4 --emergency
python scripts/run_full_experiments.py && python scripts/make_results.py

# Sections 2.2.3 and 4.7: detector (IDD downloaded separately, linked at
data/external/idd_detection)
python scripts/prepare_idd.py && python scripts/resize_idd.py
python scripts/train_detector.py --full
python scripts/eval_detector.py --stage val && python
scripts/eval_detector.py --stage test
python scripts/make_perception_results.py

# Sections 4.5 and 4.6: camera errors and the exploratory model-size study
python scripts/run_camera_error.py && python
scripts/run_camera_decomposition.py
python scripts/make_camera_error.py
python scripts/run_model_size.py --bench && python
scripts/run_model_size.py --full
python scripts/make_model_size.py

# Dashboard replays, from the committed runs
python scripts/build_replays.py

# This report
python scripts/make_report.py && python scripts/build_report.py
```

```
# Checks
pytest -q
for s in validation/make_tables make_indian_tables make_baseline_tables
make_results \
         make_camera_error make_model_size make_perception_results
make_reproducibility make_report; do
  python scripts/$s.py --check
done
python scripts/benchmark_llm.py --check
```

6.3.3 A Check from a Fresh Clone

To test these steps, the repository was cloned from GitHub into an empty folder and set up from the README alone, and every deterministic result was run again (`docs/reproducibility.md`, generated from `outputs/reproducibility/fresh_clone.json`). All 42 of the 42 runs of the fixed-time, Webster, MaxPressure and preemption controllers came out identical in every metric, as did the detector's test-split evaluation (mAP50 0.563, mAP50-95 0.375, and the whole camera error model), the IDD label conversion and the dashboard's replays. A 600 s run of the pre-registered LLM arm repeated the committed run exactly, in all 121 answers. The check found gaps in the setup instructions, not in the results: tests and checks that failed in a clean install, an evaluation script that read an absolute path, a PyTorch build the GPU driver could not use, and no pinned versions. Each was fixed, and the affected steps were run again in the same clone.

The one part that did not repeat exactly was CityFlow with lane changing on, which only the validation's matched runs use: the same run, repeated, gives a slightly different average travel time, in CoLLMLight's code as in ours, while throughput and their queue length never changed (Section 2.2.1).

LLM runs are not guaranteed to be bit-for-bit repeatable on a GPU (Section 2.2.4): a re-run can give slightly different numbers. The logged decisions reproduce each run exactly, and the dashboard's replays are kept only when they do. The hosted model of the model-size study needs an API key in a git-ignored `.env` file and is subject to the provider's rate limits.

6.3.4 Data Not in the Repository

IDD is downloaded from its authors after registration and is not redistributed; the detector weights, training runs, the dashboard's replays and its sample clip are kept outside the repository and reached through git-ignored links set in `configs/default.yaml`. CoLLMLight's roadnets and flows are downloaded by `scripts/fetch_collmlight_data.sh`.

6.4 Full Tables

6.4.1 Every Baseline Run on the Indian Network

Table 6.1 Every baseline run on the Indian 2x2 network

Every baseline run on the Indian 2x2 network (seeds 0–2), including the two MaxPressure settings not chosen by the tuning.

Level	Controller	Seed	ATT (s)	Queue / int.	Waiting (s)	Throughput	Waiting to enter
offpeak	FixedTime, 4 paired phases, 30 s	0	121.6	10.44	64.1	2285	0
offpeak	FixedTime, 4 paired phases, 30 s	1	121.2	10.53	63.1	2325	0
offpeak	FixedTime, 4 paired phases, 30 s	2	121.6	10.77	63.9	2338	0
offpeak	FixedTime, all 8 phases, 30 s	0	128.3	11.49	70.6	2267	0
offpeak	FixedTime, all 8 phases, 30 s	1	128.8	11.82	70.9	2304	0
offpeak	FixedTime, all 8 phases, 30 s	2	130.7	12.33	73.1	2322	0
offpeak	FixedTime, 4 paired phases, Webster plan	0	104.4	7.33	45.0	2280	0
offpeak	FixedTime, 4 paired phases, Webster plan	1	100.7	6.85	41.1	2329	0
offpeak	FixedTime, 4 paired phases, Webster plan	2	96.1	6.28	37.2	2350	0
offpeak	MaxPressure (8 phases, 5 s, min 15 / max 60)	0	87.2	4.72	29.0	2291	0
offpeak	MaxPressure (8 phases, 5 s, min 15 / max 60)	1	88.2	4.94	29.6	2347	0
offpeak	MaxPressure (8 phases, 5 s, min 15 / max 60)	2	89.1	5.17	30.7	2358	0

Level	Controller	Seed	ATT (s)	Queue / int.	Waiting (s)	Throughput	Waiting to enter
offpeak	MaxPressure (8 phases, 15 s, min 10 / max 60)	0	81.0	3.73	22.9	2289	0
offpeak	MaxPressure (8 phases, 15 s, min 10 / max 60)	1	81.6	3.82	22.9	2345	0
offpeak	MaxPressure (8 phases, 15 s, min 10 / max 60)	2	81.3	3.86	22.9	2362	0
offpeak	MaxPressure (8 phases, 5 s, min 10 / max 60)	0	81.2	3.72	22.9	2290	0
offpeak	MaxPressure (8 phases, 5 s, min 10 / max 60)	1	82.2	3.92	23.5	2345	0
offpeak	MaxPressure (8 phases, 5 s, min 10 / max 60)	2	82.7	4.07	24.1	2361	0
peak	FixedTime, 4 paired phases, 30 s	0	291.2	43.64	144.8	4061	325
peak	FixedTime, 4 paired phases, 30 s	1	275.8	44.54	142.3	4238	285
peak	FixedTime, 4 paired phases, 30 s	2	272.7	45.14	145.1	4198	313
peak	FixedTime, all 8 phases, 30 s	0	300.3	46.98	156.2	4016	335
peak	FixedTime, all 8 phases, 30 s	1	288.7	47.83	153.6	4188	311
peak	FixedTime, all 8 phases, 30 s	2	286.7	48.19	156.1	4135	347
peak	FixedTime, 4 paired phases, Webster plan	0	118.3	18.47	57.0	4477	1
peak	FixedTime, 4 paired phases, Webster plan	1	119.6	19.44	58.4	4637	0
peak	FixedTime, 4 paired phases, Webster plan	2	111.1	16.91	50.8	4617	0
peak	MaxPressure (8 phases, 5 s, min 15 / max 60)	0	116.0	17.53	54.1	4501	1
peak	MaxPressure (8 phases, 5 s, min 15 / max 60)	1	116.1	18.16	54.6	4634	0
peak	MaxPressure (8 phases, 5 s, min 15 / max 60)	2	114.5	17.80	53.5	4594	0
peak	MaxPressure (8 phases, 15 s, min 10 / max 60)	0	152.0	28.47	87.9	4410	1

Level	Controller	Seed	ATT (s)	Queue / int.	Waiting (s)	Throughput	Waiting to enter
peak	MaxPressure (8 phases, 15 s, min 10 / max 60)	1	122.2	19.78	59.4	4634	0
peak	MaxPressure (8 phases, 15 s, min 10 / max 60)	2	127.4	21.48	64.6	4575	0
peak	MaxPressure (8 phases, 5 s, min 10 / max 60)	0	162.9	31.69	97.8	4366	1
peak	MaxPressure (8 phases, 5 s, min 10 / max 60)	1	136.3	24.18	72.6	4610	0
peak	MaxPressure (8 phases, 5 s, min 10 / max 60)	2	127.9	21.65	65.1	4572	0

6.4.2 MaxPressure Decision-Interval Sweep

Table 6.2 First MaxPressure tuning sweep (decision interval)

First tuning sweep (decision interval, minimum green 10 s): rank of each interval per scenario. The 5 s and 15 s settings tie on mean rank; the tie-break chose 15 s, which the minimum-green sweep (Table 4.1) later replaced.

Interval	Rank: jinan_3x4	Rank: hangzhou_4x4	Rank: indian_2x2_offpeak	Rank: indian_2x2_peak	Mean rank	Mean ATT / best
5 s	1	1	2	5	2.25	1.059
10 s	3	3	3	2	2.75	1.028
15 s	2	2	1	4	2.25	1.042
20 s	4	4	4	1	3.25	1.045
30 s	5	5	5	3	4.50	1.158

6.4.3 Validation Against CoLLMLight

Table 6.3 Validation runs scored with CoLLMLight's metrics

Validation runs scored with CoLLMLight's metric definitions: their code against ours, same settings and CityFlow seed.

Dataset	Run	ATT (their def.)	Throughput (their def.)	Queue (their def.)	Waiting (their def.)	CityFlow ATT	Engine seed
Synthetic 4x4	CoLLMLight code: FixedTime	1192.5	751	467.3	1215.6	1612.0	47
Synthetic 4x4	Ours, replaying their FixedTime (phases 1,3 on a 30 s grid)	1192.5	751	467.3	1215.6	1612.0	47
Synthetic 4x4	CoLLMLight code: MaxPressure	683.1	2505	468.4	84.8	1263.3	47
Synthetic 4x4	Ours, MaxPressure matched to theirs	683.1	2505	468.4	84.8	1263.3	47
Jinan 3x4	CoLLMLight code: FixedTime	1239.3	1786	1338.6	920.3	1258.0	47
Jinan 3x4	Ours, replaying their FixedTime (phases 1,3 on a 30 s grid)	1239.3	1786	1338.6	920.3	1258.0	47
Jinan 3x4	CoLLMLight code: MaxPressure	273.2	4187	106.6	38.2	317.8	47
Jinan 3x4	Ours, MaxPressure matched to theirs	273.2	4187	106.6	38.2	317.8	47
Hangzhou 4x4	CoLLMLight code: FixedTime	1002.8	1526	694.5	827.5	1054.7	47
Hangzhou 4x4	Ours, replaying their FixedTime (phases 1,3 on a 30 s grid)	1002.8	1526	694.5	827.5	1054.7	47
Hangzhou 4x4	CoLLMLight code: MaxPressure	325.3	2751	69.0	49.6	365.7	47
Hangzhou 4x4	Ours, MaxPressure matched to theirs	325.3	2751	69.0	49.6	365.7	47

6.4.4 What the Models Chose in the Benchmark

Table 6.4 Benchmark decisions replayed from the logs

Benchmark decisions replayed from the logs: how often each model kept the current phase and chose the highest- or lowest-pressure option. Pressures are MaxPressure's at the moment of each decision.

Arm (prompt)	Model	Successful calls	Kept current phase	Chose highest-pressure phase	Chose lowest-pressure phase	Kept current although another had higher pressure
v1 (v1)	gemma3:4b	333	86%	4%	51%	95%
v1 (v1)	gemma4:e4b	159	35%	9%	29%	34%
v1 (v1)	llama3.2:3b	270	77%	2%	34%	84%
v1 (v1)	qwen2.5:3b	159	34%	18%	25%	34%
v2 (v2)	gemma4:e4b	120	0%	40%	9%	0%
v2 (v2)	llama3.2:3b	350	89%	6%	48%	99%
v2 (v2)	qwen2.5:3b	251	88%	6%	49%	100%
hybrid (guided)	gemma4:e4b	283	78%	18%	83%	84%
hybrid (guided)	llama3.2:3b	286	79%	4%	86%	86%
hybrid (guided)	qwen2.5:3b	221	78%	25%	77%	80%
v3 (v3)	gemma4:e4b	122	4%	57%	9%	3%
v3 (v3)	llama3.2:3b	350	88%	9%	50%	99%
v3 (v3)	qwen2.5:3b	355	89%	5%	48%	100%
hybrid_v3 (guided_v3)	gemma4:e4b	151	28%	78%	22%	23%
hybrid_v3 (guided_v3)	llama3.2:3b	299	81%	5%	90%	90%
hybrid_v3 (guided_v3)	qwen2.5:3b	252	82%	20%	81%	86%
guarded_v3 (guided_v3)	gemma4:e4b	127	17%	88%	9%	9%
guarded_v3 (guided_v3)	llama3.2:3b	121	66%	13%	71%	70%
guarded_v3 (guided_v3)	qwen2.5:3b	114	75%	26%	72%	72%

6.4.5 LLM Arm Under Camera Errors

Table 6.5 The pre-registered LLM arm's decisions at each camera level

scenario	level	LLM calls	failures	guard overrides	disagrees with MP	mean latency (s)
Off-peak	perfect	2177	0	81.2%	91.5%	1.02
	junction	2195	0	85.2%	93.1%	1.00
	crowded	2189	0	82.5%	92.9%	1.00
Peak	perfect	2167	0	82.8%	92.8%	0.99
	junction	2174	0	86.1%	92.4%	0.99
	crowded	2175	0	85.1%	92.6%	0.99
Emergency	perfect	3632	0	76.3%	86.9%	1.03
	junction	3646	0	78.8%	87.8%	1.03

6.4.6 Hosted Model Runs (Exploratory)

Table 6.6 Every run of the hosted model (exploratory)

Every run of the hosted model: calls that needed retries after rate limiting, and calls that ran out of retries (MaxPressure's choice applied).

Started (IST)	Run	Calls	Calls that needed retries	Failed calls → MaxPressure's choice applied	Wall time (min)
2026-10-22 12:33	indian_2x2_peak s0	128	0 (0%)	0 (0.0%)	5
2026-10-23 10:23	indian_2x2_peak s0	736	286 (39%)	11 (1.5%): retries_exhausted 10, timeout 1	121
2026-10-23 12:22	indian_2x2_peak s1	734	351 (48%)	12 (1.6%): retries_exhausted 11, timeout 1	135
2026-10-23 14:36	indian_2x2_peak s2	735	417 (57%)	16 (2.2%): retries_exhausted 16	150
2026-10-23 17:07	indian_2x2_peak_amb s0	733	415 (57%)	15 (2.0%): retries_exhausted 15	160
2026-10-23 19:46	indian_2x2_peak_amb s1	749	425 (57%)	20 (2.7%): retries_exhausted 17, timeout 3	170
2026-10-23 22:34	indian_2x2_peak_amb s2	746	413 (55%)	17 (2.3%): retries_exhausted 15, timeout 2	159
2026-10-24 01:13	indian_2x2_peak_amb s3	731	405 (55%)	24 (3.3%): retries_exhausted 21, timeout 3	150
2026-10-24 03:43	indian_2x2_peak_amb s4	732	392 (54%)	17 (2.3%): retries_exhausted 16, timeout 1	150

6.5 Abbreviations

Abbreviation	Meaning
ATT	Average travel time (CityFlow's definition, Section 2.2.4)
CoLLMLight	Cooperative LLM agents for network-wide traffic signal control (the base paper)
COCO	Common Objects in Context, the dataset whose object-size buckets we use
CVD	Colour vision deficiency
FT	Fixed-time signal control
GPU / VRAM	Graphics processing unit / its memory
IDD	India Driving Dataset (IIIT Hyderabad and Intel)
IoU	Intersection over union of two bounding boxes
JSON	JavaScript Object Notation, the format of the LLM's answer
LLM	Large language model
mAP50, mAP50-95	Mean average precision at IoU ≥ 0.5, and averaged over IoU thresholds from 0.5 to 0.95
MP	MaxPressure signal control
NIM	NVIDIA Inference Microservices (hosted model API)
NMS	Non-maximum suppression
PCU	Passenger car unit
RL	Reinforcement learning
SCATS / SCOOT	Sydney Coordinated Adaptive Traffic System / Split Cycle Offset Optimisation Technique
TSC	Traffic signal control
V2X	Vehicle-to-everything communication
YOLO	You Only Look Once, a family of one-stage object detectors

6.6 Source Code

Every listing below is the file as it is in commit `1d0cfca4a5` of the project repository, printed by the report build (line numbers on the left); none is retyped. Long lines wrap.

6.6.1 src/trafficsense/control/base.py

Controller interface. `act(env)` is called once before every env.step(). 26 lines.

```
"""Controller interface. `act(env)` is called once before every env.step()."""
from __future__ import annotations

from abc import ABC, abstractmethod
from pathlib import Path
from typing import Any

from trafficsense.sim.env import TrafficEnv


class Controller(ABC):
    name: str

    @abstractmethod
    def act(self, env: TrafficEnv) -> None: ...

    def info(self) -> dict[str, Any]:
        """Parameters recorded in the run summary."""
        return {"name": self.name}

    def provenance(self) -> dict[str, Any]:
        """Extra provenance for summary.json (LLM controllers: model, provider, llm stats)."""
        return {}

    def write_artifacts(self, out_dir: Path) -> None:
        """Extra files in the run folder (LLM controllers: decisions.jsonl)."""
```

6.6.2 src/trafficsense/control/maxpressure.py

Max-pressure control (Varaiya, 2013). 100 lines.

```
"""Max-pressure control (Varaiya, 2013).

Pressure of a phase = sum over the movements it serves of
(waiting vehicles on the movement's incoming lanes - waiting vehicles on its outgoing lanes).
Every `decision_interval` seconds, each intersection that has met min_green switches to the
phase with the highest pressure. Decisions fall on a fixed grid (simulation time a multiple of
the interval), as in CoLLMLight, so the shortest green actually possible is the first grid point
at least min_green after the transition: 10 s for a 5 s or 15 s interval, 15 s for 10 s and 20 s,
25 s for 30 s (with 5 s transitions). `phases` picks the candidate set (all green phases, or the 4
paired phases). With `tie_break: keep_current` a tie keeps the current phase, otherwise the
lowest index wins; `lowest_index` always takes the lowest index (as CoLLMLight's np.argmax).

`min_green` (optional, default signal.min_green) is the controller's own minimum green; it may
be longer than the env's signal.min_green, never shorter. It decouples the shortest green from
the decision grid: with a 5 s interval, the shortest green equals `min_green` exactly.

The env forces a switch when a green reaches max_green. To keep the choice pressure-based,
the current phase is excluded when keeping it would reach max_green before the next decision;
with a 5 s interval and 60 s max_green this caps a max-pressure green at 55 s.
"""
from __future__ import annotations

from typing import Any

from trafficsense.control.base import Controller
```

```
from trafficsense.control.fixedtime import PHASE_SETS, phase_cycle
from trafficsense.sim.env import TrafficEnv
from trafficsense.sim.roadnet import Intersection


TIE_BREAKS = ("keep_current", "lowest_index")


def phase_pressures(inter: Intersection, state: dict[str, Any],
                    candidates: list[int] | None = None) -> dict[int, float]:
    by_index = {m["index"]: m for m in state["movements"]}
    phases = [p for p in inter.green_phases if candidates is None or p.index in candidates]
    return {
        p.index: sum(by_index[k]["queue"] - by_index[k]["out_queue"] for k in p.movements)
        for p in phases
    }


def choose_phase(pressures: dict[int, float], current: int, exclude_current: bool,
                 tie_break: str = "keep_current") -> int:
    cand = {p: v for p, v in pressures.items() if not (exclude_current and p == current)}
    best = max(cand.values())
    if tie_break == "keep_current" and current in cand and cand[current] == best:
        return current
    return min(p for p, v in cand.items() if v == best)


class MaxPressureController(Controller):
    name = "maxpressure"

    def __init__(self, cfg: dict[str, Any], name: str = "maxpressure"):
        self.name = name
        if name not in cfg["controllers"]:
            raise KeyError(f"Config is missing controllers.{name}")
        c = cfg["controllers"][name]
        if c.get("decision_interval") is None:
            raise KeyError(f"Config is missing controllers.{name}.decision_interval")
        self.interval = int(c["decision_interval"])
        if self.interval <= 0:
            raise ValueError(f"decision_interval must be positive, got {self.interval}")
        self.phases = c.get("phases", "all")
        if self.phases not in PHASE_SETS:
            raise ValueError(f"{name}.phases must be one of {PHASE_SETS}, got {self.phases!
    r}")
        self.tie_break = c.get("tie_break", "keep_current")
        if self.tie_break not in TIE_BREAKS:
            raise ValueError(f"{name}.tie_break must be one of {TIE_BREAKS}, got
    {self.tie_break!r}")
        self.max_green = int(cfg["signal"]["max_green"])
        env_min = int(cfg["signal"]["min_green"])
        self.min_green = int(c.get("min_green", env_min))
        if not env_min <= self.min_green < self.max_green:
            raise ValueError(f"{name}.min_green must be in [signal.min_green {env_min},
    max_green "
                             f"{self.max_green}), got {self.min_green}")
        self._candidates: dict[str, list[int]] = {}

   def act(self, env: TrafficEnv) -> None:
        if int(round(env.time)) % self.interval != 0:
            return
        for iid in env.intersection_ids:
            if not env.can_switch(iid) or env.time_in_phase(iid) < self.min_green:
                continue
            env.request_phase(iid, self._choose(env, iid))

    def _pressures(self, env: TrafficEnv, iid: str) -> dict[int, float]:
        inter = env.intersection(iid)
        if iid not in self._candidates:
            self._candidates[iid] = phase_cycle(inter, self.phases)
        return phase_pressures(inter, env.get_state(iid), self._candidates[iid])

```

```
94      def _choose(self, env: TrafficEnv, iid: str) -> int:
95          near_max = env.time_in_phase(iid) + self.interval >= self.max_green
96          return choose_phase(self._pressures(env, iid), env.signals[iid].green_phase,
      near_max, self.tie_break)
97
98      def info(self) -> dict[str, Any]:
99          return {"name": self.name, "decision_interval": self.interval, "min_green":
      self.min_green,
100                 "phases": self.phases, "tie_break": self.tie_break}
```

6.6.3 `src/trafficsense/control/llm.py`

LLM signal control: one LLM call per intersection decision (adapted from CoLLMLight). 207 lines.

```
1   """LLM signal control: one LLM call per intersection decision (adapted from CoLLMLight).
2
3   Modes: `cooperative` (the prompt includes a summary of each neighbouring intersection) and
4   `isolated` (own intersection only). `prompt` picks the prompt version (v1: Phase 3a, kept as
5   controller llm_v1; v2: llm_cooperative / llm_isolated); see trafficsense.llm.prompt for
      exactly
6   what is sent.
7
8   Timing, from llm.* in the config, follows MaxPressure so the two differ only in the decision
9   rule: decisions fall on a grid of `decision_interval` seconds; an intersection is asked only
10  when it may switch (not in transition, green shown for at least `min_green`, which must be
11  >= signal.min_green). The env inserts the transition phase and enforces min/max green. When
12  keeping the current phase would reach max_green before the next decision, the current phase
13  is not offered.
14
15  Failures: a timeout, server error or an invalid answer (see prompt.parse_response) keeps the
16  current phase and is counted by type and logged. If the server is unreachable
17  (LLMUnavailable), or `max_consecutive_failures` calls in a row fail, the run stops with an
18  error. Nothing is filled in silently.
19
20  Outputs: provenance() for summary.json (model and digest, call/failure counts, latency
21  mean/p95) and decisions.jsonl (one line per call) via write_artifacts().
22  """
23  from __future__ import annotations
24
25  import json
26  import logging
27  import time
28  from pathlib import Path
29  from typing import Any
30
31  import numpy as np
32
33  from trafficsense.control.base import Controller
34  from trafficsense.llm import prompt
35  from trafficsense.llm.client import LLMError, LLMUnavailable
36  from trafficsense.llm.hosted import make_client
37  from trafficsense.sim.env import TrafficEnv
38
39  log = logging.getLogger(__name__)
40
41  REQUIRED = ("decision_interval", "min_green", "max_consecutive_failures")
42
43
44  class LLMControllerBase(Controller):
45      """Shared by the LLM controllers: config, client, failure counting, stats and provenance.
46      Subclasses implement act() and call self._ask() for each decision."""
47
48      def __init__(self, cfg: dict[str, Any], name: str, client: Any = None):
49          self.name = name
50          if name not in cfg["controllers"]:
51              raise KeyError(f"Config is missing controllers.{name}")
```

```
52          c = cfg["controllers"][name]
53          self.mode = c.get("mode")
54          if self.mode not in prompt.MODES:
55              raise ValueError(f"controllers.{name}.mode must be one of {prompt.MODES}, got
    {self.mode!r}")
56          self.prompt = c.get("prompt")
57          if self.prompt not in self.PROMPTS:
58              raise ValueError(f"controllers.{name}.prompt must be one of {self.PROMPTS}, got
    {self.prompt!r}")
59          llm = cfg["llm"]
60          missing = [k for k in REQUIRED if llm.get(k) is None]
61          if missing:
62              raise KeyError(f"Config is missing llm.{missing} (decided after
    scripts/benchmark_llm.py)")
63          self.interval = int(llm["decision_interval"])
64          self.min_green = int(llm["min_green"])
65          self.max_green = int(cfg["signal"]["max_green"])
66          self.max_consecutive = int(llm["max_consecutive_failures"])
67          env_min = int(cfg["signal"]["min_green"])
68          if self.interval <= 0:
69              raise ValueError(f"llm.decision_interval must be positive, got {self.interval}")
70          if not env_min <= self.min_green < self.max_green:
71              raise ValueError(f"llm.min_green must be in [signal.min_green {env_min},
    max_green {self.max_green}), "
72                               f"got {self.min_green}")
73          self.client = client if client is not None else make_client(llm)
74          self.model_info = self.client.model_info()          # raises if the server or model
    is missing
75          self.server_version = self.client.server_version()
76          self.decisions: list[dict[str, Any]] = []
77          self._consecutive = 0
78
79      PROMPTS: tuple[str, ...] = ()
80
81      def _due(self, env: TrafficEnv) -> list[str]:
82          """Intersections to decide for now: on the decision grid, switch allowed, min green
    met."""
83          if int(round(env.time)) % self.interval != 0:
84              return []
85          return [i for i in env.intersection_ids if env.can_switch(i) and env.time_in_phase(i)
    >= self.min_green]
86
87      def _ask(self, obs: dict[str, Any], allowed: list[int], rec: dict[str, Any]) ->
    tuple[int, str] | None:
88          """One LLM call. Returns (phase, reason), or None after a counted failure (recorded
    in
89          `rec`, which the caller completes and appends). LLMUnavailable propagates; too many
90          failures in a row raise."""
91          t0 = time.perf_counter()
92          try:
93              res = self.client.chat(prompt.messages(obs, self.prompt),
    prompt.response_schema(allowed, self.prompt))
94              rec.update(latency_s=res.latency_s, prompt_tokens=res.prompt_tokens,
95                         response_tokens=res.response_tokens, server_model=res.model,
    raw=res.content,
96                         reasoning_chars=res.reasoning_chars, retries=list(res.retries),
    wait_s=res.wait_s,
97                         call_wall_s=time.perf_counter() - t0)
98              out = prompt.parse_response(res.content, allowed,
    prompt.MAX_REASON_WORDS[self.prompt])
99          except LLMUnavailable:
100             raise
101         except (LLMError, prompt.InvalidResponse) as e:
102             if isinstance(e, LLMError):
103                 rec.update(retries=list(e.retries), call_wall_s=time.perf_counter() - t0)
104             self._consecutive += 1
105             rec.update(ok=False, failure_type=e.failure_type, error=str(e), reason=None)
106             n = sum(1 for d in self.decisions if not d["ok"]) + 1
107             log.warning("LLM failure #%d (%s) at t=%s %s: %s", n, e.failure_type,
    rec["step"], rec["intersection"], e)
108             if self._consecutive >= self.max_consecutive:
```

```
109                 raise RuntimeError(f"{self._consecutive} LLM calls in a row failed (last:
    {e.failure_type}: {e}); "
110                                    "stopping the run") from e
111             return None
112         self._consecutive = 0
113         rec.update(ok=True, failure_type=None, reason=out[1])
114         return out
115 
116     # ---- reporting ---------------------------------------------------------------
117     def stats(self) -> dict[str, Any]:
118         d = self.decisions
119         fails: dict[str, int] = {}
120         for x in d:
121             if not x["ok"]:
122                 fails[x["failure_type"]] = fails.get(x["failure_type"], 0) + 1
123         lat = [x["latency_s"] for x in d if "latency_s" in x]   # every call that got an
    answer
124         pt = [x["prompt_tokens"] for x in d if x.get("prompt_tokens") is not None]
125         rt = [x["response_tokens"] for x in d if x.get("response_tokens") is not None]
126         served = sorted({x["server_model"] for x in d if "server_model" in x})
127         retries: dict[str, int] = {}
128         for x in d:
129             for r in x.get("retries", []):
130                 retries[r] = retries.get(r, 0) + 1
131         wall = [x["call_wall_s"] for x in d if "call_wall_s" in x]
132         return {
133             "calls": len(d),
134             "ok": sum(1 for x in d if x["ok"]),
135             "failures": sum(fails.values()),
136             "failures_by_type": dict(sorted(fails.items())),
137             "failure_rate": sum(fails.values()) / len(d) if d else None,
138             "switches_requested": sum(1 for x in d if x.get("action") == "switch"),
139             "latency_n": len(lat),
140             "latency_mean_s": float(np.mean(lat)) if lat else None,
141             "latency_p95_s": float(np.percentile(lat, 95)) if lat else None,   # numpy linear
    interpolation
142             "latency_max_s": float(np.max(lat)) if lat else None,
143             "prompt_tokens_mean": float(np.mean(pt)) if pt else None,
144             "response_tokens_mean": float(np.mean(rt)) if rt else None,
145             "server_reported_models": served,
146             "calls_with_reasoning_output": sum(1 for x in d if x.get("reasoning_chars")),
147             "retries": sum(retries.values()),
148             "retries_by_reason": dict(sorted(retries.items())),
149             "calls_with_retries": sum(1 for x in d if x.get("retries")),
150             "wait_s_total": float(sum(x.get("wait_s", 0.0) for x in d)),
151             "call_wall_mean_s": float(np.mean(wall)) if wall else None,
152             "call_wall_p95_s": float(np.percentile(wall, 95)) if wall else None,
153             "retry_note": "retries (HTTP 429/5xx, dropped connection) are not failures; a
    call fails only "
154                           "when it ends without a valid answer. call_wall = whole call incl.
    rate-limit "
155                           "waits, backoff and retried attempts",
156             "latency_note": "wall clock per call, over calls that returned a response
    (timeouts excluded, counted as failures)",
157         }
158 
159     def info(self) -> dict[str, Any]:
160         return {"name": self.name, "mode": self.mode, "prompt": self.prompt,
    "decision_interval": self.interval,
161                 "min_green": self.min_green, "phases": "all",
162                 "client": {"provider": self.client.provider, "model": self.client.model,
163                            "host": self.client.host, "endpoint": self.client.endpoint,
164                            "timeout_s": self.client.timeout_s, "options":
    self.client.options,
165                            "keep_alive": self.client.keep_alive},
166                 "max_consecutive_failures": self.max_consecutive}
167 
168     def provenance(self) -> dict[str, Any]:
169         st = self.stats()
170         served = st["server_reported_models"]
171         return {"model": served[0] if len(served) == 1 else (served or
```

```
      self.model_info["name"]),
172                 "provider": self.client.provider,
173                 "llm": {**st, "model_digest": self.model_info["digest"], "model_details":
      self.model_info["details"],
174                         "server_version": self.server_version, "decisions_log":
      "decisions.jsonl"}}
175
176     def write_artifacts(self, out_dir: Path) -> None:
177         with (out_dir / "decisions.jsonl").open("w", encoding="utf-8") as f:
178             for rec in self.decisions:
179                 f.write(json.dumps(rec) + "\n")
180
181
182 class LLMController(LLMControllerBase):
183     """The LLM chooses any green phase (the current one is left out near max green)."""
184     PROMPTS = ("v1", "v2", "v3")
185
186     def __init__(self, cfg: dict[str, Any], name: str = "llm_cooperative", client: Any =
      None):
187         super().__init__(cfg, name, client)
188
189     def act(self, env: TrafficEnv) -> None:
190         for iid in self._due(env):
191             self._decide(env, iid)
192
193     def _decide(self, env: TrafficEnv, iid: str) -> None:
194         cur = env.signals[iid].green_phase
195         allowed = [p.index for p in env.intersection(iid).green_phases]
196         if env.time_in_phase(iid) + self.interval >= self.max_green:
197             allowed.remove(cur)
198         obs = prompt.observe(env, iid, allowed, self.mode, self.min_green)
199         rec: dict[str, Any] = {"step": int(env.time), "intersection": iid, "current_phase":
      cur,
200                                "time_in_phase": env.time_in_phase(iid), "allowed": allowed}
201         out = self._ask(obs, allowed, rec)
202         if out is None:
203             rec.update(phase=cur, action="kept current phase (failure)")
204         else:
205             switched = env.request_phase(iid, out[0])
206             rec.update(phase=out[0], action="switch" if switched else "keep")
207         self.decisions.append(rec)
```

6.6.4 `src/trafficsense/control/guided.py`

LLM-guided MaxPressure (`llm_guided_mp`): MaxPressure proposes, the LLM picks. 184 lines.

```
 1  """LLM-guided MaxPressure (`llm_guided_mp`): MaxPressure proposes, the LLM picks.
 2
 3  At each decision (same grid and timing as LLMController: llm.decision_interval,
      llm.min_green)
 4  MaxPressure ranks the green phases by pressure (control/maxpressure.py; ties as its
 5  `tie_break`, with the current phase left out when keeping it would reach max green). The LLM
 6  is offered MaxPressure's top `options` phases (2) and keeping the current phase, which is
 7  always allowed here because an intersection is only asked once min green is met, unless max
 8  green is near. It sees those options with pressure and queue plus, in cooperative mode, the
 9  neighbours (prompt version "guided"), and answers with one of them and a reason.
10
11  Emergency scenarios (env.emergency_type set): ambulances detected on the intersection's
      incoming
12  lanes within emergency.detection_range_m (sim/emergency.py, the same detection as the
13  preemption baseline) are listed in the prompt (approach, movement, distance, the phases that
14  serve it), the system prompt adds that listed ambulances must get through quickly, and the
15  best-ranked phase serving each such ambulance is added to the options if it is not there yet
16  (otherwise the LLM could not act on what it is told).
17
18  On a failed call (timeout, server error, invalid answer) MaxPressure's top choice is applied
```

```
and the failure is counted by type; LLMUnavailable and `max_consecutive_failures` failures in
a row stop the run, as for the other LLM controllers.

Each decision in decisions.jsonl records MaxPressure's ranking and top choice, and whether
 the
LLM disagreed with it; summary.json gets the disagreement counts.
"""
from __future__ import annotations

from typing import Any

from trafficsense.control.llm import LLMControllerBase
from trafficsense.control.maxpressure import TIE_BREAKS, phase_pressures
from trafficsense.llm import prompt
from trafficsense.sim.emergency import EmergencyVehicles
from trafficsense.sim.env import TrafficEnv


def mp_ranking(pressures: dict[int, float], current: int, exclude_current: bool,
               tie_break: str = "keep_current") -> list[int]:
    """Phases best first, as MaxPressure would choose: highest pressure; among equals the
    current phase first (keep_current), then the lowest index. ranking[0] ==
 choose_phase()."""
    cand = [p for p in pressures if not (exclude_current and p == current)]
    return sorted(cand, key=lambda p: (-pressures[p], not (tie_break == "keep_current" and p
 == current), p))


class LLMGuidedMPController(LLMControllerBase):
    PROMPTS = ("guided", "guided_v3")

    def __init__(self, cfg: dict[str, Any], name: str = "llm_guided_mp", client: Any = None):
        super().__init__(cfg, name, client)
        c = cfg["controllers"][name]
        self.n_options = int(c.get("options", 0))
        if self.n_options < 2:
            raise ValueError(f"controllers.{name}.options must be >= 2, got
 {c.get('options')!r}")
        self.tie_break = c.get("tie_break")
        if self.tie_break not in TIE_BREAKS:
            raise ValueError(f"controllers.{name}.tie_break must be one of {TIE_BREAKS}, got
 {self.tie_break!r}")
        if cfg.get("emergency", {}).get("detection_range_m") is None:
            raise KeyError("Config is missing emergency.detection_range_m")
        self.range_m = float(cfg["emergency"]["detection_range_m"])
        self._ev: EmergencyVehicles | None = None

    def act(self, env: TrafficEnv) -> None:
        for iid in self._due(env):
            self._decide(env, iid)

    def _decide(self, env: TrafficEnv, iid: str) -> None:
        cur = env.signals[iid].green_phase
        near_max = env.time_in_phase(iid) + self.interval >= self.max_green
        pressures = phase_pressures(env.intersection(iid), env.get_state(iid))
        ranking = mp_ranking(pressures, cur, near_max, self.tie_break)
        top = ranking[0]
        options = ranking[:self.n_options]
        if not near_max and cur not in options:
            options.append(cur)
        ambulances = []
        if env.emergency_type is not None:
            if self._ev is None:
                self._ev = EmergencyVehicles(env)
            inter = env.intersection(iid)
            for sg in self._ev.detect(env, iid, self.range_m):
                serving = [p for p in ranking if p in sg.serving_phases]
                if serving and serving[0] not in options:
                    options.append(serving[0])
                ambulances.append({"vehicle": sg.vehicle, "approach": sg.approach,
 "distance_m": sg.distance_m,
```

```
                                    "movement":
    prompt.movement_label(inter.movements[sg.movement]),
                                    "phases": serving})
            obs = prompt.observe(env, iid, options, self.mode, self.min_green)
            obs["ranking"] = ranking
            if env.emergency_type is not None:
                obs["emergency_range_m"] = self.range_m
                obs["ambulances"] = ambulances
            rec: dict[str, Any] = {"step": int(env.time), "intersection": iid, "current_phase":
    cur,
                                   "time_in_phase": env.time_in_phase(iid), "allowed":
    sorted(options),
                                   "mp_ranking": ranking, "mp_pressures": {str(p): pressures[p]
    for p in ranking},
                                   "mp_choice": top}
            if ambulances:
                rec["ambulances"] = ambulances
            out = self._ask(obs, options, rec)
            if out is None:
                phase = top
            else:
                rec["llm_phase"] = out[0]
                phase = self._accept(out[0], top, obs, ambulances, rec)
            switched = env.request_phase(iid, phase)
            if ambulances:
                rec["serves_nearest_ambulance"] = phase in ambulances[0]["phases"]
            rec.update(phase=phase, disagrees_with_mp=None if out is None else out[0] != top,
                       action=("MaxPressure fallback (failure), " if out is None else "")
                       + ("switch" if switched else "keep"))
            self.decisions.append(rec)

        def _accept(self, llm_phase: int, top: int, obs: dict[str, Any], ambulances:
    list[dict[str, Any]],
                    rec: dict[str, Any]) -> int:
            """The phase applied for a valid LLM answer (the guarded subclass may override
    it)."""
            return llm_phase

        def stats(self) -> dict[str, Any]:
            st = super().stats()
            ok = [d for d in self.decisions if d["ok"]]
            dis = [d for d in ok if d["disagrees_with_mp"]]
            st.update(
                mp_disagreements=len(dis),
                mp_disagreement_rate=len(dis) / len(ok) if ok else None,
                mp_disagreements_kept_current=sum(1 for d in dis if d["llm_phase"] ==
    d["current_phase"]),
                mp_disagreements_chose_lower_ranked=sum(1 for d in dis if d["llm_phase"] !=
    d["current_phase"]),
                mp_fallbacks=len(self.decisions) - len(ok),
                mp_disagreement_note="over successful calls: the LLM's answer != MaxPressure's
    top choice",
            )
            amb = [d for d in self.decisions if "ambulances" in d]
            if amb:
                st.update(ambulance_decisions=len(amb),
                          ambulance_decisions_serving_nearest=sum(1 for d in amb if
    d["serves_nearest_ambulance"]))
            # switches_requested in the base counts action == "switch" only; fallbacks also
    switch
            st["switches_requested"] = sum(1 for d in self.decisions if
    d["action"].endswith("switch"))
            return st

        def info(self) -> dict[str, Any]:
            return {**super().info(), "options": self.n_options, "tie_break": self.tie_break,
                    "detection_range_m": self.range_m,
                    "ranking": "maxpressure (phase_pressures over all green phases)"}


    class LLMGuardedMPController(LLMGuidedMPController):
```

```
143     """`llm_guarded_mp`: llm_guided_mp whose answer is applied only if its pressure is close
144     enough to the top phase's; otherwise MaxPressure's top choice is applied.
145
146     Guard: accept the LLM's phase if pressure(phase) >= top - (1 - guard_ratio) * |top|,
    where
147     top is the pressure of MaxPressure's top choice. For top >= 0 this is
148     pressure >= guard_ratio * top (0.8 x top for guard_ratio 0.8); the |top| form keeps the
149     top phase itself acceptable when every pressure is negative. Pressures are the ones
150     shown to the LLM (MaxPressure's without the free turns, which every phase serves).
151     A phase that serves a listed ambulance (emergency scenarios) is always accepted, so the
152     guard cannot block ambulance priority. Overrides are counted (guard_overrides) and each
153     decision records the LLM's answer (llm_phase), the threshold and whether the guard acted.
154     """
155
156     def __init__(self, cfg: dict[str, Any], name: str = "llm_guarded_mp", client: Any =
    None):
157         super().__init__(cfg, name, client)
158         r = cfg["controllers"][name].get("guard_ratio")
159         if r is None or not 0 < float(r) <= 1:
160             raise ValueError(f"controllers.{name}.guard_ratio must be in (0, 1], got {r!r}")
161         self.guard_ratio = float(r)
162
163     def _accept(self, llm_phase, top, obs, ambulances, rec):
164         shown = {p["id"]: p["pressure"] for p in obs["phases"]}
165         threshold = shown[top] - (1 - self.guard_ratio) * abs(shown[top])
166         for_ambulance = any(llm_phase in a["phases"] for a in ambulances)
167         override = shown[llm_phase] < threshold and not for_ambulance
168         rec.update(guard_threshold=threshold, guard_llm_pressure=shown[llm_phase],
    guard_override=override,
169                    guard_ambulance_exempt=for_ambulance)
170         return top if override else llm_phase
171
172     def stats(self) -> dict[str, Any]:
173         st = super().stats()
174         ok = [d for d in self.decisions if d["ok"]]
175         n = sum(1 for d in ok if d["guard_override"])
176         st.update(guard_overrides=n, guard_override_rate=n / len(ok) if ok else None,
177                   guard_ambulance_exemptions=sum(1 for d in ok if
    d["guard_ambulance_exempt"]),
178                   applied_differs_from_mp=sum(1 for d in self.decisions if d["phase"] !=
    d["mp_choice"]),
179                   guard_note="guard_override_rate over successful calls: the LLM's answer was
    replaced by "
180                              "MaxPressure's top choice")
181         return st
182
183     def info(self) -> dict[str, Any]:
184         return {**super().info(), "guard_ratio": self.guard_ratio}
```

6.6.5 `src/trafficsense/control/preemption.py`

MaxPressure with rule-based ambulance preemption (`preemption_mp`): the non-LLM baseline 70 lines.

```
 1  """MaxPressure with rule-based ambulance preemption (`preemption_mp`): the non-LLM baseline
 2  for emergency vehicles.
 3
 4  Same timing and pressures as MaxPressure (decision grid, own min green, current phase left
 5  out near max green). At a decision, if an ambulance is on an incoming lane within
 6  emergency.detection_range_m of the stop line (sim/emergency.py; not on a free turn), the
 7  nearest one gets green: among the green phases that serve its movement, the one with the
 8  highest pressure (ties: keep current, then lowest index). Keeping the current phase when it
 9  already serves the ambulance is part of that rule. Otherwise it is plain MaxPressure.
10
11  It sees exactly what llm_guided_mp is told about ambulances (same detection and range) and
12  acts on the same 5 s grid after the same min green, so the two differ only in the decision.
13  It does not cut a green short before min green, and does not hold green beyond max green.
```

```
Each preemption is logged to preemptions.jsonl; the count goes to summary.json
(provenance.controller_stats).
"""
from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from trafficsense.control.maxpressure import MaxPressureController, choose_phase
from trafficsense.sim.emergency import EmergencyVehicles
from trafficsense.sim.env import TrafficEnv


class PreemptionMPController(MaxPressureController):
    def __init__(self, cfg: dict[str, Any], name: str = "preemption_mp"):
        super().__init__(cfg, name)
        if cfg.get("emergency", {}).get("detection_range_m") is None:
            raise KeyError("Config is missing emergency.detection_range_m")
        self.range_m = float(cfg["emergency"]["detection_range_m"])
        self._ev: EmergencyVehicles | None = None
        self.log: list[dict[str, Any]] = []

    def _choose(self, env: TrafficEnv, iid: str) -> int:
        if env.emergency_type is None:
            return super()._choose(env, iid)
        if self._ev is None:
            self._ev = EmergencyVehicles(env)
        seen = self._ev.detect(env, iid, self.range_m)
        cur = env.signals[iid].green_phase
        near_max = env.time_in_phase(iid) + self.interval >= self.max_green
        pressures = self._pressures(env, iid)
        mp = choose_phase(pressures, cur, near_max, self.tie_break)
        if not seen:
            return mp
        s = seen[0]
        cand = {p: pressures[p] for p in s.serving_phases if p in pressures and not (near_max
    and p == cur)}
        if not cand:
            return mp
        phase = choose_phase(cand, cur, False, self.tie_break)
        self.log.append({"step": int(env.time), "intersection": iid, "vehicle": s.vehicle,
    "distance_m": s.distance_m,
                         "approach": s.approach, "turn": s.turn, "current_phase": cur,
    "phase": phase,
                         "maxpressure_choice": mp, "changed_decision": phase != mp})
        return phase

    def info(self) -> dict[str, Any]:
        return {**super().info(), "detection_range_m": self.range_m}

    def provenance(self) -> dict[str, Any]:
        return {"controller_stats": {"preemption_decisions": len(self.log),
                                     "changed_maxpressure_decision": sum(1 for x in self.log
    if x["changed_decision"]),
                                     "log": "preemptions.jsonl"}}

    def write_artifacts(self, out_dir: Path) -> None:
        with (out_dir / "preemptions.jsonl").open("w", encoding="utf-8") as f:
            for rec in self.log:
                f.write(json.dumps(rec) + "\n")
```

6.6.6 `src/trafficsense/llm/client.py`

Ollama client: native /api/chat with a JSON-schema `format` (structured output). 165 lines.

```
"""Ollama client: native /api/chat with a JSON-schema `format` (structured output).

Every call uses temperature and seed from the config (0 and a fixed seed by default), a
timeout, and `think: false`. The result carries what the server reported: the model name,
token counts and durations, plus the wall-clock latency measured here.

Errors are raised, never hidden:
  * LLMUnavailable  the server cannot be reached, or the model is not installed. A run
                    cannot continue; callers must let this propagate.
  * LLMTimeout      no answer within timeout_s.
  * LLMServerError  the server answered with an HTTP error or an "error" field, or a body
                    that is not the expected chat response.
Each error has `failure_type`, which the controller counts. A prompt that may not fit the
context window (prompt tokens + num_predict > num_ctx) raises ValueError: Ollama would
silently truncate it, so this is a configuration error that stops the run.

The HTTP session is injectable (`session=`) so tests can use an in-process fake; nothing in
the package starts a fake server.
"""
from __future__ import annotations

import time
from dataclasses import dataclass
from typing import Any

import requests


class LLMError(Exception):
    failure_type = "llm_error"
    retries: tuple[str, ...] = ()   # hosted client: why each retry of this call happened (e.g. "429")


class LLMUnavailable(LLMError):
    failure_type = "unavailable"


class LLMTimeout(LLMError):
    failure_type = "timeout"


class LLMServerError(LLMError):
    failure_type = "server_error"


@dataclass(frozen=True)
class ChatResult:
    content: str
    model: str                  # as reported by the server in the response
    latency_s: float            # wall clock, request sent -> response parsed
    prompt_tokens: int | None   # prompt_eval_count (None if the server did not report it)
    response_tokens: int | None  # eval_count
    total_duration_s: float | None
    load_duration_s: float | None
    done_reason: str | None
    reasoning_chars: int = 0      # thinking/reasoning text the server returned (0 when it is off)
    retries: tuple[str, ...] = ()  # hosted client: one entry per retried attempt (HTTP status or error)
    wait_s: float = 0.0            # hosted client: rate-limiter and backoff waits before the answered attempt


REQUIRED = ("host", "endpoint", "model", "timeout_s", "temperature", "seed", "num_ctx", "num_predict",
            "keep_alive")


class OllamaClient:
    provider = "ollama"
```

```
67
68      def __init__(self, host: str, model: str, endpoint: str = "/api/chat", timeout_s: float =
    60.0,
69                   temperature: float = 0.0, seed: int = 0, num_ctx: int = 4096, num_predict:
    int = 128,
70                   keep_alive: str = "30m", session: Any = None):
71          if not model:
72              raise ValueError("No LLM model given (llm.model in the config is null)")
73          if timeout_s <= 0:
74              raise ValueError(f"timeout_s must be positive, got {timeout_s}")
75          self.host = host.rstrip("/")
76          self.endpoint = endpoint
77          self.model = model
78          self.timeout_s = float(timeout_s)
79          self.options = {"temperature": float(temperature), "seed": int(seed), "num_ctx":
    int(num_ctx),
80                          "num_predict": int(num_predict)}
81          self.keep_alive = keep_alive
82          self.session = session if session is not None else requests.Session()
83
84      @classmethod
85      def from_config(cls, llm_cfg: dict[str, Any], session: Any = None, model: str | None =
    None) -> "OllamaClient":
86          if llm_cfg.get("provider") != "ollama":
87              raise ValueError(f"llm.provider must be 'ollama', got {llm_cfg.get('provider')!
    r}")
88          missing = [k for k in REQUIRED if k != "model" and llm_cfg.get(k) is None]   # model:
    checked in __init__
89          if missing:
90              raise KeyError(f"Config is missing llm.{missing}")
91          return cls(host=llm_cfg["host"], model=model or llm_cfg["model"],
    endpoint=llm_cfg["endpoint"],
92                     timeout_s=llm_cfg["timeout_s"], temperature=llm_cfg["temperature"],
    seed=llm_cfg["seed"],
93                     num_ctx=llm_cfg["num_ctx"], num_predict=llm_cfg["num_predict"],
94                     keep_alive=llm_cfg["keep_alive"], session=session)
95
96      # ---- HTTP ----------------------------------------------------------------------
97      def _request(self, method: str, path: str, json_body: dict | None = None,
98                   timeout: float | None = None) -> dict[str, Any]:
99          url = self.host + path
100         try:
101             r = self.session.request(method, url, json=json_body, timeout=timeout or
    self.timeout_s)
102         except requests.exceptions.Timeout as e:
103             raise LLMTimeout(f"{method} {url}: no answer within {timeout or
    self.timeout_s:.0f} s") from e
104         except requests.exceptions.ConnectionError as e:
105             raise LLMUnavailable(f"Cannot reach the Ollama server at {self.host}: {e}") from
    e
106         try:
107             body = r.json()
108         except ValueError:
109             body = None
110         if r.status_code != 200:
111             msg = body.get("error") if isinstance(body, dict) else r.text[:200]
112             if r.status_code == 404 and path != "/api/tags":
113                 raise LLMUnavailable(f"{method} {url}: HTTP 404 ({msg}); is model
    {self.model!r} pulled?")
114             raise LLMServerError(f"{method} {url}: HTTP {r.status_code}: {msg}")
115         if not isinstance(body, dict):
116             raise LLMServerError(f"{method} {url}: response is not a JSON object:
    {r.text[:200]!r}")
117         if "error" in body:
118             raise LLMServerError(f"{method} {url}: {body['error']}")
119         return body
120
121     # ---- model info ----------------------------------------------------------------
122     def server_version(self) -> str:
123         return str(self._request("GET", "/api/version", timeout=10)["version"])
124
```

```
125     def model_info(self) -> dict[str, Any]:
126         """Name, digest and details of the configured model, as listed by the server."""
127         tags = self._request("GET", "/api/tags", timeout=10).get("models", [])
128         for m in tags:
129             if self.model in (m.get("name"), m.get("model")):
130                 return {"name": m.get("name"), "digest": m.get("digest"), "size_bytes":
    m.get("size"),
131                         "details": m.get("details", {})}
132         raise LLMUnavailable(f"Model {self.model!r} is not installed on {self.host} "
133                              f"(installed: {[m.get('name') for m in tags]}); run `ollama pull
    {self.model}`")
134
135     def loaded_models(self) -> list[dict[str, Any]]:
136         """/api/ps: models in memory, with `size` and `size_vram` (bytes) as the server
    reports them."""
137         return self._request("GET", "/api/ps", timeout=10).get("models", [])
138
139     def unload(self, model: str | None = None) -> None:
140         """Ask the server to drop a model from memory now (keep_alive 0)."""
141         self._request("POST", "/api/generate", {"model": model or self.model, "keep_alive":
    0}, timeout=60)
142
143     # ---- chat ------------------------------------------------------------------------
144     def chat(self, messages: list[dict[str, str]], schema: dict[str, Any]) -> ChatResult:
145         body = {"model": self.model, "messages": messages, "format": schema, "stream": False,
    "think": False,
146                 "options": self.options, "keep_alive": self.keep_alive}
147         t0 = time.perf_counter()
148         resp = self._request("POST", self.endpoint, body)
149         latency = time.perf_counter() - t0
150         msg = resp.get("message")
151         if not isinstance(msg, dict) or not isinstance(msg.get("content"), str):
152             raise LLMServerError(f"Chat response has no message content: {str(resp)[:200]}")
153
154         def secs(k: str) -> float | None:
155             return resp[k] / 1e9 if isinstance(resp.get(k), (int, float)) else None
156
157         n_prompt = resp.get("prompt_eval_count")
158         if isinstance(n_prompt, int) and n_prompt + self.options["num_predict"] >
    self.options["num_ctx"]:
159             # Ollama silently drops the start of a prompt that does not fit; refuse instead.
160             raise ValueError(f"Prompt of {n_prompt} tokens + num_predict
    {self.options['num_predict']} exceeds "
161                              f"num_ctx {self.options['num_ctx']}; raise llm.num_ctx")
162         return ChatResult(content=msg["content"], model=str(resp.get("model", "")),
    latency_s=latency,
163                           prompt_tokens=resp.get("prompt_eval_count"),
    response_tokens=resp.get("eval_count"),
164                           total_duration_s=secs("total_duration"),
    load_duration_s=secs("load_duration"),
165                           done_reason=resp.get("done_reason"),
    reasoning_chars=len(msg.get("thinking") or ""))
```

6.6.7 `src/trafficsense/llm/prompt.py`

Prompt and response schema for LLM signal control (one call per intersection decision). 320 lines.

```
1  """Prompt and response schema for LLM signal control (one call per intersection decision).
2
3  observe() collects what the LLM is told, all measured from the simulator at decision time
     (lane
4  counts through env.lane_view(): exact, or with camera errors in the camera-error experiment):
5    * current green phase and seconds shown, min/max green, transition length;
6    * for each candidate green phase, its movements, each with
7        queue        stopped vehicles (speed < 0.1 m/s) on the movement's incoming lanes,
8        moving       vehicles on those lanes that are not stopped (approaching the stop line),
```

```
      downstream   stopped vehicles on the movement's exit lanes;
  * vehicle-type counts on each approach (all vehicles on its incoming lanes);
  * cooperative mode only, per neighbouring intersection: its current phase, the stopped
    vehicles there whose movement leads onto the road into us (and how many of them have
    green now), and the stopped vehicles on the road from us into it.
Free turns (CityFlow `turn_right`) run in every phase, including the transition, so the LLM
cannot affect them and they are left out. Movement names are neutral about the side of
the road: "straight", "cross-turn" (CityFlow turn_left, crossing the opposing flow).

Two prompt versions render the same observation:
  * v1 (Phase 3a, kept for the record): every movement's queue/moving/downstream, vehicle
    types per approach, neighbours; the rules say switching "wastes green time"; reason <= 25
    words. About 650 tokens.
  * v2: one line per phase with a precomputed pressure and queue, so the model does no
    arithmetic; neighbours as in v1; no vehicle types (to stay under ~300 tokens; pressure
    counts vehicles regardless of type, as MaxPressure does); the transition is stated as a
    fact without advice; reason <= 15 words.
  * guided (llm_guided_mp): MaxPressure ranks the phases; only the offered options (its top
    phases, and keeping the current one when allowed) are listed, in rank order, with their
    pressure and queue; neighbours as in v2; reason <= 15 words. In emergency scenarios the
    ambulances within detection range are listed, and the system prompt says so.
  * v3 and guided_v3 (Phase 3b, final round): v2 and guided with exactly two changes, a
    post-hoc fix of a defect found in the v2/hybrid analysis (docs/llm_benchmark.md): the
    prompts defined pressure but never said which way is better, and the models preferred
    low pressure. v3 (a) says "HIGHER pressure = the phase needs green MORE" and (b) moves
    the word limit out of the JSON placeholder, which qwen2.5:3b copied into its reasons
    ("At most 15 vehicles ..."). Built by text substitution on v2/guided, so nothing else
    differs (tested).
Phase pressure here is MaxPressure's (queue minus downstream queue, summed over the phase's
movements) without the free turns. Every green phase serves all free turns, so leaving them
out lowers every phase by the same amount and keeps MaxPressure's ranking.

The answer must be JSON {"phase": <allowed id>, "reason": "<= N words"} (N per version).
parse_response() raises InvalidResponse with a failure type for anything else; the
controller counts it.
"""
from __future__ import annotations

import json
from typing import Any

from trafficsense.sim.env import TrafficEnv

VERSIONS = ("v1", "v2", "guided", "v3", "guided_v3")
MAX_REASON_WORDS = {"v1": 25, "v2": 15, "guided": 15, "v3": 15, "guided_v3": 15}
TURN_LABEL = {"go_straight": "straight", "turn_left": "cross-turn"}
FREE_TURN = "turn_right"
MODES = ("cooperative", "isolated")

# v1: unchanged since the Phase 3a benchmark (tests pin its hash).
SYSTEM_PROMPT = """You control the traffic signal of one intersection in a city road network
 with mixed \
Indian traffic (two-wheelers, auto-rickshaws, cars, buses, trucks).
At each decision you choose which green phase to show next. Goal: keep vehicles moving and
 queues short \
across the whole network, so total travel time is low.
Rules of the signal:
- Choosing the current phase keeps it green. Choosing another phase first shows a
 {transition} s \
transition in which only free turns move, so switching too often wastes green time.
- A green phase cannot be ended before {min_green} s and is ended automatically at
 {max_green} s.
Data you get for each movement: queue = stopped vehicles waiting at the stop line; moving =
 vehicles \
driving towards the stop line; downstream = stopped vehicles on the exit road (a full exit
 road blocks \
the movement).
Answer with JSON only: {{"phase": <one of the allowed phase ids>, "reason": "<at most 25
 words>"}}."""

SYSTEM_PROMPT_V2 = """You pick the next green phase for one traffic signal in an Indian city.
```

```
     Goal: short \
 73  queues and low travel time network-wide.
 74  Changing phase starts a {transition} s transition in which only free turns move. A green
      lasts {min_green} to \
 75  {max_green} s.
 76  queue = stopped vehicles a phase serves; pressure = queue minus stopped vehicles on their
      exit roads.
 77  Reply JSON: {{"phase": <allowed id>, "reason": "<at most {words} words>"}}"""
 78
 79
 80  SYSTEM_PROMPT_GUIDED = """You help a MaxPressure controller run one traffic signal in an
      Indian city. Goal: \
 81  short queues and low travel time network-wide.
 82  MaxPressure ranks phases by pressure = stopped vehicles a phase serves minus stopped vehicles
      on their exit \
 83  roads; queue = stopped vehicles a phase serves. Choose one of the offered phases; neighbour
      data may help.
 84  Changing phase starts a {transition} s transition in which only free turns move. A green
      lasts {min_green} to \
 85  {max_green} s.
 86  {emergency}Reply JSON: {{"phase": <offered id>, "reason": "<at most {words} words>"}}"""
 87
 88  # v3: the only two changes to v2 / guided (see the module docstring).
 89  V3_EDITS = {
 90      "v2": [("pressure = queue minus stopped vehicles on their exit roads.",
 91              "pressure = vehicles waiting to go minus vehicles already stopped on their exit
      roads. "
 92              "HIGHER pressure = the phase needs green MORE."),
 93             ('"reason": "<at most {words} words>"}}',
 94              '"reason": "<short reason>"}}. The reason must be {words} words or fewer.')],
 95      "guided": [("pressure = stopped vehicles a phase serves minus stopped vehicles on their
      exit roads;",
 96                  "pressure = vehicles waiting to go minus vehicles already stopped on their
      exit roads; "
 97                  "HIGHER pressure = the phase needs green MORE;"),
 98                 ('"reason": "<at most {words} words>"}}',
 99                  '"reason": "<short reason>"}}. The reason must be {words} words or fewer.')],
100  }
101
102
103  def _v3(template: str, base: str) -> str:
104      for old, new in V3_EDITS[base]:
105          if template.count(old) != 1:
106              raise ValueError(f"v3 edit target not found exactly once in the {base} prompt:
      {old!r}")
107          template = template.replace(old, new)
108      return template
109
110
111  EMERGENCY_GUIDED = "Ambulances, if listed, must get through quickly: give green to a phase
      that serves them.\n"
112  SYSTEM_PROMPT_V3 = _v3(SYSTEM_PROMPT_V2, "v2")
113  SYSTEM_PROMPT_GUIDED_V3 = _v3(SYSTEM_PROMPT_GUIDED, "guided")
114
115
116  def movement_label(m) -> str:
117      return f"{m.approach} {TURN_LABEL[m.turn]}"
118
119
120  def phase_label(inter, phase) -> str:
121      """Short name of a green phase from its non-free movements, e.g. "W+E straight",
122      "W+E cross-turn", or "N only" for every movement of one approach."""
123      ms = [inter.movements[k] for k in phase.movements if inter.movements[k].turn !=
      FREE_TURN]
124      turns = [TURN_LABEL[t] for t in TURN_LABEL if any(m.turn == t for m in ms)]
125      sides = list(dict.fromkeys(m.approach for m in ms))
126      if len(turns) == 1:
127          return f"{'+'.join(sides)} {turns[0]}"
128      if len(sides) == 1:
129          return f"{sides[0]} only" if len(turns) == len(TURN_LABEL) else f"{sides[0]}
      {'+'.join(turns)}"
```

```
    return ", ".join(movement_label(m) for m in ms)


def response_schema(allowed: list[int], version: str) -> dict[str, Any]:
    """JSON schema passed as Ollama's `format`. maxLength bounds the generated text (latency);
    the word limit itself is checked in parse_response."""
    return {"type": "object",
            "properties": {"phase": {"type": "integer", "enum": sorted(allowed)},
                           "reason": {"type": "string", "maxLength": 8 * MAX_REASON_WORDS[version]}},
            "required": ["phase", "reason"]}


class InvalidResponse(ValueError):
    def __init__(self, failure_type: str, msg: str):
        super().__init__(msg)
        self.failure_type = failure_type


def parse_response(text: str, allowed: list[int], max_words: int) -> tuple[int, str]:
    try:
        obj = json.loads(text)
    except (json.JSONDecodeError, TypeError) as e:
        raise InvalidResponse("invalid_json", f"not JSON: {text[:200]!r}") from e
    if not isinstance(obj, dict) or set(obj) != {"phase", "reason"}:
        raise InvalidResponse("invalid_schema", f"expected exactly keys phase, reason: {text[:200]!r}")
    phase, reason = obj["phase"], obj["reason"]
    if isinstance(phase, bool) or not isinstance(phase, int):
        raise InvalidResponse("invalid_schema", f"phase is not an integer: {phase!r}")
    if not isinstance(reason, str) or not reason.strip():
        raise InvalidResponse("invalid_schema", f"reason is not a non-empty string: {reason!r}")
    if phase not in allowed:
        raise InvalidResponse("phase_not_allowed", f"phase {phase} not in allowed {sorted(allowed)}")
    if len(reason.split()) > max_words:
        raise InvalidResponse("reason_too_long", f"reason has {len(reason.split())} words (max {max_words})")
    return phase, reason.strip()


def _road_between(env: TrafficEnv, a: str, b: str) -> str | None:
    """The road from intersection a to intersection b, if any."""
    roads = set(env.intersection(a).outgoing_roads) & set(env.intersection(b).incoming_roads)
    if len(roads) > 1:
        raise ValueError(f"More than one road from {a} to {b}: {sorted(roads)}")
    return next(iter(roads), None)


def observe(env: TrafficEnv, iid: str, allowed: list[int], mode: str, min_green: int) -> dict[str, Any]:
    if mode not in MODES:
        raise ValueError(f"mode must be one of {MODES}, got {mode!r}")
    inter = env.intersection(iid)
    view = env.lane_view()   # exact, or with camera errors (sim/observation.py)
    waiting, count = view.waiting, view.count
    by_idx = {m.index: m for m in inter.movements}

    def mv(m) -> dict[str, Any]:
        q = sum(waiting[l] for l in m.in_lanes)
        return {"movement": movement_label(m), "queue": q, "moving": sum(count[l] for l in m.in_lanes) - q,
                "downstream": sum(waiting[l] for l in m.out_lanes)}

    phases = []
    for p in inter.green_phases:
        ms = [mv(by_idx[k]) for k in p.movements if by_idx[k].turn != FREE_TURN]
        phases.append({"id": p.index, "label": phase_label(inter, p), "movements": ms,
                       "queue": sum(m["queue"] for m in ms),
```

```
                        "pressure": sum(m["queue"] - m["downstream"] for m in ms)})
    on_approach: dict[str, list[str]] = {}
    for m in inter.movements:
        on_approach.setdefault(m.approach, []).extend(m.in_lanes)
    types = {a: view.type_counts(lanes, env.vehicle_types.names) for a, lanes in
sorted(on_approach.items())}

    obs: dict[str, Any] = {
        "intersection": iid, "time": int(env.time), "current_phase":
env.signals[iid].green_phase,
        "time_in_phase": env.time_in_phase(iid), "min_green": min_green, "max_green":
env.max_green,
        "transition_s": env.transition_time[iid], "phases": phases, "vehicle_types": types,
        "allowed": sorted(allowed),
    }
    if mode == "cooperative":
        obs["neighbours"] = [_neighbour(env, iid, nb, d, waiting) for nb, d in
sorted(inter.neighbours.items())]
    return obs


def _neighbour(env: TrafficEnv, iid: str, nb: str, direction: str, waiting: dict[str, int])
 -> dict[str, Any]:
    n = env.intersection(nb)
    sig = env.signals[nb]
    green = None if sig.in_transition else sig.green_phase
    served = set(n.phases[green].movements) if green is not None else set()
    to_us, from_us = _road_between(env, nb, iid), _road_between(env, iid, nb)
    toward = [m for m in n.movements if m.out_road == to_us]
    return {
        "id": nb, "direction": direction,
        "phase": green, "phase_movements": [movement_label(n.movements[k]) for k in
sorted(served)
                                            if n.movements[k].turn != FREE_TURN],
        "phase_label": None if green is None else phase_label(n, n.phases[green]),
        "queue_heading_to_us": sum(waiting[l] for m in toward for l in m.in_lanes),
        "of_which_green_now": sum(waiting[l] for m in toward if m.index in served or m.turn
== FREE_TURN
                                  for l in m.in_lanes),
        "queue_on_road_from_us": sum(waiting[l] for m in n.movements if m.in_road == from_us
for l in m.in_lanes),
    }


def render(obs: dict[str, Any]) -> str:
    """The user message: compact text, one line per phase/approach/neighbour."""
    lines = [f"Intersection {obs['intersection']}, time {obs['time']} s.",
             f"Current green: phase {obs['current_phase']}, shown for {obs['time_in_phase']}
s."]
    lines.append("Phases (movement: queue/moving/downstream):")
    for p in obs["phases"]:
        cur = " (current)" if p["id"] == obs["current_phase"] else ""
        ms = "; ".join(f"{m['movement']}: {m['queue']}/{m['moving']}/{m['downstream']}" for m
in p["movements"])
        lines.append(f"  phase {p['id']}{cur}: {ms}")
    lines.append("Vehicles on each approach:")
    for a, c in obs["vehicle_types"].items():
        lines.append(f"  {a}: " + (", ".join(f"{k} {v}" for k, v in c.items()) or "none"))
    if "neighbours" in obs:
        lines.append("Neighbouring intersections:")
        for nb in obs["neighbours"]:
            ph = "in transition" if nb["phase"] is None else \
                f"phase {nb['phase']} ({', '.join(nb['phase_movements'])})"
            lines.append(f"  {nb['direction']} {nb['id']}: {ph}; {nb['queue_heading_to_us']}
stopped there "
                         f"heading to us ({nb['of_which_green_now']} on green now); "
                         f"{nb['queue_on_road_from_us']} stopped on our road into it.")
    lines.append(f"Allowed phases: {obs['allowed']}. Reply with JSON only.")
    return "\n".join(lines)


```

```
253 def _neighbour_line(nb: dict[str, Any]) -> str:
254     ph = "in transition" if nb["phase"] is None else f"phase {nb['phase']}
    ({nb['phase_label']})"
255     return (f"{nb['direction']}: {ph}; {nb['queue_heading_to_us']} stopped toward us, "
256             f"{nb['of_which_green_now']} on green; {nb['queue_on_road_from_us']} stopped on
    our road to it")
257
258
259 def render_v2(obs: dict[str, Any]) -> str:
260     """v2 user message: one line per phase (pressure, queue), then neighbours."""
261     cur = next(p for p in obs["phases"] if p["id"] == obs["current_phase"])
262     lines = [f"Phase {cur['id']} ({cur['label']}) green for {obs['time_in_phase']} s.",
263              "phase: pressure, queue"]
264     for p in obs["phases"]:
265         lines.append(f"{p['id']} {p['label']}: {p['pressure']}, {p['queue']}")
266     if "neighbours" in obs:
267         lines.append("Neighbours:")
268         lines.extend(_neighbour_line(nb) for nb in obs["neighbours"])
269     every = [p["id"] for p in obs["phases"]]
270     left_out = [i for i in every if i not in obs["allowed"]]
271     lines.append("Allowed: all" if not left_out else
272                  f"Allowed: all except {', '.join(map(str, left_out))}" if len(left_out) <
    len(every) / 2 else
273                  f"Allowed: {', '.join(map(str, obs['allowed']))}")
274     return "\n".join(lines)
275
276
277 def render_guided(obs: dict[str, Any]) -> str:
278     """Guided user message: the offered options in MaxPressure's rank order
    (obs["ranking"])."""
279     cur = next(p for p in obs["phases"] if p["id"] == obs["current_phase"])
280     by_id = {p["id"]: p for p in obs["phases"]}
281     lines = [f"Phase {cur['id']} ({cur['label']}) green for {obs['time_in_phase']} s.",
282              "Options, best MaxPressure rank first (phase: pressure, queue):"]
283     for i in [r for r in obs["ranking"] if r in obs["allowed"]]:
284         p = by_id[i]
285         keep = " (keep current)" if i == obs["current_phase"] else ""
286         lines.append(f"{i} {p['label']}{keep}: {p['pressure']}, {p['queue']}")
287     if obs.get("ambulances"):
288         lines.append(f"Ambulances within {obs['emergency_range_m']:.0f} m:")
289         for a in obs["ambulances"]:
290             served = (f"served by phase {', '.join(map(str, a['phases']))}" if a["phases"]
    else
291                       "only the current phase serves it, which must end now (max green)")
292             lines.append(f"{a['movement']}, {a['distance_m']:.0f} m from the stop line;
    {served}")
293     if "neighbours" in obs:
294         lines.append("Neighbours:")
295         lines.extend(_neighbour_line(nb) for nb in obs["neighbours"])
296     return "\n".join(lines)
297
298
299 def messages(obs: dict[str, Any], version: str) -> list[dict[str, str]]:
300     if version == "v1":
301         sys = SYSTEM_PROMPT.format(transition=obs["transition_s"],
    min_green=obs["min_green"],
302                                    max_green=obs["max_green"])
303         user = render(obs)
304     elif version == "v2":
305         sys = SYSTEM_PROMPT_V2.format(transition=obs["transition_s"],
    min_green=obs["min_green"],
306                                       max_green=obs["max_green"],
    words=MAX_REASON_WORDS["v2"])
307         user = render_v2(obs)
308     elif version == "v3":
309         sys = SYSTEM_PROMPT_V3.format(transition=obs["transition_s"],
    min_green=obs["min_green"],
310                                       max_green=obs["max_green"],
    words=MAX_REASON_WORDS["v3"])
311         user = render_v2(obs)
312     elif version in ("guided", "guided_v3"):
```

```
        tpl = SYSTEM_PROMPT_GUIDED if version == "guided" else SYSTEM_PROMPT_GUIDED_V3
        sys = tpl.format(transition=obs["transition_s"], min_green=obs["min_green"],
                         max_green=obs["max_green"], words=MAX_REASON_WORDS[version],
                         emergency=EMERGENCY_GUIDED if "ambulances" in obs else "")
        user = render_guided(obs)
    else:
        raise ValueError(f"prompt version must be one of {VERSIONS}, got {version!r}")
    return [{"role": "system", "content": sys}, {"role": "user", "content": user}]
```

6.6.8 `src/trafficsense/sim/env.py`

CityFlow engine wrapper: per-intersection state and signal timing. 259 lines.

```
"""CityFlow engine wrapper: per-intersection state and signal timing.

CityFlow's set_tl_phase switches instantly, so this wrapper owns the signal logic:
  * a switch to a different green phase first shows the intersection's transition
    phase (read from the roadnet) for its full duration;
  * a green phase cannot be left before min_green;
  * a green phase that reaches max_green is ended by the wrapper, which switches to the
    next green phase in index order. Such forced switches are counted and reported.
"""
from __future__ import annotations

import json
import tempfile
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import cityflow
import yaml

from trafficsense.config import repo_root, resolve_path
from trafficsense.sim.observation import make_observer
from trafficsense.sim.roadnet import Intersection, Roadnet, load_roadnet
from trafficsense.sim.vehicles import VehicleTypes


@dataclass
class SignalState:
    green_phase: int            # current green, or the green being switched to during a transition
    in_transition: bool = False
    transition_left: int = 0    # steps of transition still to show
    time_in_phase: int = 0      # seconds the current green has been shown
    forced_switches: int = 0
    switches: int = 0


def _require(section: dict[str, Any], key: str, where: str) -> Any:
    if key not in section or section[key] is None:
        raise KeyError(f"Config is missing {where}.{key}")
    return section[key]


class TrafficEnv:
    def __init__(self, cfg: dict[str, Any], scenario: str, seed: int | None = None,
                 replay_dir: Path | None = None):
        """`replay_dir`: if given, CityFlow writes its replay log (vehicle positions per step) and
        roadnet log there (used by sim/replay.py only; logging does not change the run)."""
        if scenario not in cfg.get("scenarios", {}):
            raise KeyError(f"Unknown scenario {scenario!r}; known: {list(cfg.get('scenarios', {}))}")
        sc = cfg["scenarios"][scenario]
        sim = cfg["sim"]
        sig = cfg["signal"]

```

```
54          self.scenario = scenario
55          self.interval = float(_require(sim, "interval", "sim"))
56          if self.interval != 1.0:
57              # Timing below counts steps as seconds.
58              raise ValueError(f"sim.interval must be 1.0, got {self.interval}")
59          self.seed = int(seed if seed is not None else _require(sim, "seed", "sim"))
60          self.min_green = int(_require(sig, "min_green", "signal"))
61          self.max_green = int(_require(sig, "max_green", "signal"))
62          if not 0 < self.min_green <= self.max_green:
63              raise ValueError(f"Need 0 < min_green <= max_green, got {self.min_green},
    {self.max_green}")
64
65          root = repo_root()
66          self.roadnet_path = (root / _require(sc, "roadnet", f"scenarios.
    {scenario}")).resolve()
67          # A flow path may contain "{seed}": generated scenarios have one flow file per seed.
68          flow = str(_require(sc, "flow", f"scenarios.{scenario}"))
69          self.flow_per_seed = "{seed}" in flow
70          self.flow_path = (root / flow.format(seed=self.seed)).resolve()
71          for p in (self.roadnet_path, self.flow_path):
72              if not p.is_file():
73                  hint = sc.get("generate_hint", "")
74                  raise FileNotFoundError(f"Scenario file not found: {p}" + (f" ({hint})" if
    hint else ""))
75
76          vt = sc.get("vehicle_types")
77          self.vehicle_types_path = (root / vt).resolve() if vt else None
78          if self.vehicle_types_path is not None and not self.vehicle_types_path.is_file():
79              raise FileNotFoundError(f"scenarios.{scenario}.vehicle_types not found:
    {self.vehicle_types_path}")
80          self._vehicle_types: VehicleTypes | None = None
81          # Vehicle type tracked as an emergency vehicle (summary.json `emergency`); needs
    named types.
82          self.emergency_type: str | None = sc.get("emergency_vehicle_type")
83          if self.emergency_type is not None and self.vehicle_types_path is None:
84              raise KeyError(f"scenarios.{scenario}: emergency_vehicle_type needs
    vehicle_types")
85
86          self.roadnet: Roadnet = load_roadnet(self.roadnet_path)
87          self.transition_time: dict[str, int] = {}
88          for iid, inter in self.roadnet.intersections.items():
89              t = inter.transition_phase.duration
90              if t != int(t) or t <= 0:
91                  raise ValueError(f"{iid}: transition duration {t} is not a positive whole
    number")
92              self.transition_time[iid] = int(t)
93
94          engine_cfg = {
95              "interval": self.interval,
96              "seed": self.seed,
97              "dir": "",
98              "roadnetFile": str(self.roadnet_path),
99              "flowFile": str(self.flow_path),
100             "rlTrafficLight": True,
101             "laneChange": bool(_require(sim, "lane_change", "sim")),
102             "saveReplay": replay_dir is not None,
103             "roadnetLogFile": str(Path(replay_dir).resolve() / "roadnet_log.json") if
    replay_dir else "",
104             "replayLogFile": str(Path(replay_dir).resolve() / "replay_log.txt") if replay_dir
    else "",
105         }
106         # CityFlow only reads its config at construction, so a temp file is enough.
107         with tempfile.TemporaryDirectory() as tmp:
108             cfg_file = Path(tmp) / "cityflow_config.json"
109             cfg_file.write_text(json.dumps(engine_cfg), encoding="utf-8")
110             self.eng = cityflow.Engine(str(cfg_file), thread_num=int(_require(sim, "threads",
    "sim")))
111         self.engine_config = engine_cfg
112
113         self.signals: dict[str, SignalState] = {}
114         for iid, inter in self.roadnet.intersections.items():
```

```
115             first = inter.green_phases[0].index
116             self.eng.set_tl_phase(iid, first)
117             self.signals[iid] = SignalState(green_phase=first)
118 
119         # What controllers observe (sim/observation.py): exact by default; with camera errors
120         # for the camera-error experiment. Metrics always read the engine directly.
121         ocfg = cfg.get("observation")
122         em = resolve_path(cfg, "camera_error_model") if "camera_error_model" in
    cfg.get("paths", {}) else None
123         self.observer = make_observer(self, ocfg, em, self.seed)
124 
125     # ---- topology ----------------------------------------------------------------
126     @property
127     def intersection_ids(self) -> list[str]:
128         return self.roadnet.ids
129 
130     def intersection(self, iid: str) -> Intersection:
131         return self.roadnet.intersections[iid]
132 
133     # ---- time --------------------------------------------------------------------
134     @property
135     def time(self) -> float:
136         return self.eng.get_current_time()
137 
138     def step(self) -> None:
139         # Phase in force while the simulator advances; current_phase() read after step()
140         # already describes the NEXT step (e.g. the green that follows a finished
    transition).
141         self._shown = {iid: self.current_phase(iid) for iid in self.signals}
142         self.eng.next_step()
143         for iid, s in self.signals.items():
144             if s.in_transition:
145                 s.transition_left -= 1
146                 if s.transition_left <= 0:
147                     s.in_transition = False
148                     s.time_in_phase = 0
149                     self.eng.set_tl_phase(iid, s.green_phase)
150             else:
151                 s.time_in_phase += 1
152                 if s.time_in_phase >= self.max_green:
153                     greens = [p.index for p in self.intersection(iid).green_phases]
154                     nxt = greens[(greens.index(s.green_phase) + 1) % len(greens)]
155                     self._start_transition(iid, nxt)
156                     s.forced_switches += 1
157 
158     # ---- signal control ----------------------------------------------------------
159     def current_phase(self, iid: str) -> int:
160         """Phase index currently shown by the simulator (transition phase during a
    switch)."""
161         s = self.signals[iid]
162         return self.intersection(iid).transition_phase.index if s.in_transition else
    s.green_phase
163 
164     def shown_phase(self, iid: str) -> int:
165         """Phase the simulator showed during the last step()."""
166         if not hasattr(self, "_shown"):
167             raise RuntimeError("shown_phase() called before the first step()")
168         return self._shown[iid]
169 
170     def time_in_phase(self, iid: str) -> int:
171         return self.signals[iid].time_in_phase
172 
173     def in_transition(self, iid: str) -> bool:
174         return self.signals[iid].in_transition
175 
176     def can_switch(self, iid: str) -> bool:
177         s = self.signals[iid]
178         return not s.in_transition and s.time_in_phase >= self.min_green
179 
180     def request_phase(self, iid: str, phase: int) -> bool:
181         """Ask for a green phase. Returns True if a switch was started, False if `phase` is
```

```
182         already showing. Raises if the phase is not a green phase or a switch is not allowed
183         yet (in transition, or min_green not reached)."""
184         greens = {p.index for p in self.intersection(iid).green_phases}
185         if phase not in greens:
186             raise ValueError(f"{iid}: phase {phase} is not a green phase (greens:
    {sorted(greens)})")
187         s = self.signals[iid]
188         if s.in_transition:
189             raise RuntimeError(f"{iid}: phase requested during a transition")
190         if phase == s.green_phase:
191             return False
192         if s.time_in_phase < self.min_green:
193             raise RuntimeError(
194                 f"{iid}: switch requested after {s.time_in_phase}s of green; min_green is
    {self.min_green}s"
195             )
196         self._start_transition(iid, phase)
197         return True
198 
199     def _start_transition(self, iid: str, target: int) -> None:
200         s = self.signals[iid]
201         s.in_transition = True
202         s.transition_left = self.transition_time[iid]
203         s.green_phase = target
204         s.switches += 1
205         self.eng.set_tl_phase(iid, self.intersection(iid).transition_phase.index)
206 
207     # ---- observation ------------------------------------------------------------
208     @property
209     def vehicle_types(self) -> VehicleTypes:
210         """Type of each vehicle, from the flow file (named by
    scenarios.<name>.vehicle_types)."""
211         if self._vehicle_types is None:
212             named = None
213             if self.vehicle_types_path is not None:
214                 with self.vehicle_types_path.open(encoding="utf-8") as f:
215                     sc = yaml.safe_load(f)
216                 named = {**sc["vehicle_types"], **sc.get("emergency_vehicle_types", {})}
217             self._vehicle_types = VehicleTypes(self.flow_path, named)
218             if self.emergency_type is not None and self.emergency_type not in
    self._vehicle_types.names:
219                 raise KeyError(f"emergency_vehicle_type {self.emergency_type!r} is not a
    named vehicle type")
220         return self._vehicle_types
221 
222     @property
223     def emergency_types(self) -> list[str]:
224         """Names of the scenario's emergency vehicle types (never noised by the observer)."""
225         if self.vehicle_types_path is None:
226             return []
227         with self.vehicle_types_path.open(encoding="utf-8") as f:
228             return list(yaml.safe_load(f).get("emergency_vehicle_types", {}))
229 
230     def lane_view(self):
231         """Lane counts as the controllers see them now (exact, or with camera errors)."""
232         return self.observer.view()
233 
234     def get_state(self, iid: str) -> dict[str, Any]:
235         """Per-movement counts for one intersection, computed only from its own lanes, as
236         observed (lane_view)."""
237         inter = self.intersection(iid)
238         view = self.lane_view()
239         waiting, count = view.waiting, view.count
240         movements = []
241         for m in inter.movements:
242             movements.append({
243                 "index": m.index,
244                 "name": m.name,
245                 "approach": m.approach,
246                 "turn": m.turn,
247                 "queue": sum(waiting[l] for l in m.in_lanes),
```

```
                "vehicles": sum(count[l] for l in m.in_lanes),
                "out_queue": sum(waiting[l] for l in m.out_lanes),
            })
        return {
            "intersection": iid,
            "time": self.time,
            "phase": self.current_phase(iid),
            "green_phase": self.signals[iid].green_phase,
            "in_transition": self.signals[iid].in_transition,
            "time_in_phase": self.signals[iid].time_in_phase,
            "movements": movements,
        }
```

6.6.9 `src/trafficsense/sim/observation.py`

What the controllers observe: exact simulator counts, or counts with camera errors. 293 lines.

```
"""What the controllers observe: exact simulator counts, or counts with camera errors.

The simulator itself always stays exact (metrics read the engine directly). Controllers read
lane counts only through TrafficEnv.lane_view(), which returns a LaneView:

  * ExactObserver: the engine's own lane counts (identical to reading the engine directly).
  * NoisyObserver (camera-error experiment, docs/camera_error.md): one camera per road. At
    each simulator step at which some controller observes, a fresh snapshot is drawn (cached
    for that step, so every intersection sees the same snapshot; the next observation draws
    again, so no vehicle stays "blind"):
      - each vehicle on a lane is detected with p_detect of its detector class;
      - a detected vehicle gets a label drawn from p_label[class]; it is counted as a vehicle
        only if the label is a vehicle class (COUNTED: a vehicle labelled `person` is lost);
      - false detections: per road and per counted class j, a Poisson(false_det[j]) number of
        phantom vehicles, each on a uniformly chosen lane of the road, counted as stopped
        (ASSUMPTION: false positives of a fixed camera are mostly static clutter, and a stopped
        phantom is the case that changes queue-based decisions most);
      - a detected vehicle is stopped if its speed < 0.1 m/s (CityFlow's waiting definition);
      - emergency vehicles (the scenario's emergency types, i.e. ambulances) are never noised:
        IDD has no ambulance class, and their positions are ASSUMED to come from a separate
        channel (GPS/V2X). They are counted exactly, and sim/emergency.py reads the engine.
    Draws use a numpy Generator seeded from (run seed, noise level) and are logged per snapshot.

Simulator vehicle types map to detector classes through SIM_TO_DETECTOR.
"""
from __future__ import annotations

import collections
import hashlib
import json
import zlib
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any

import numpy as np

from trafficsense.config import repo_root

if TYPE_CHECKING:
    from trafficsense.sim.env import TrafficEnv

DETECTOR_CLASSES = ("car", "motorcycle", "autorickshaw", "bus", "truck", "bicycle", "person")
# Indian 2x2 vehicle types -> detector classes (docs/camera_error.md). Every non-emergency
# simulator type must be listed; an unknown type raises.
SIM_TO_DETECTOR = {"car": "car", "motorcycle": "motorcycle", "auto_rickshaw": "autorickshaw",
```

```
                    "bus": "bus", "truck": "truck"}
# Detector labels counted as vehicles (person is not a vehicle; bicycle is).
COUNTED = ("car", "motorcycle", "autorickshaw", "bus", "truck", "bicycle")
# Names used in vehicle-type counts: simulator names where one exists.
DETECTOR_TO_NAME = {**{d: s for s, d in SIM_TO_DETECTOR.items()}, "bicycle": "bicycle"}
STOP_SPEED = 0.1
LEVELS = ("perfect", "junction", "crowded", "uniform")
# Post-hoc decomposition (docs/camera_error.md 7.7, added after the results): a camera level
# with only one error component switched on, the others exact.
#   misses: measured p_detect, exact labels, no false detections
#   false: every vehicle detected, exact labels, measured false detections
#   confusion: every vehicle detected, measured labels (incl. loss to `person`), no false
 detections
COMPONENTS = ("misses", "false", "confusion")


@dataclass
class NoiseParams:
    key: str                                    # e.g. "junction", "uniform_p0.80"
    p_detect: dict[str, float]
    p_label: dict[str, dict[str, float]]
    false_det: dict[str, float]
    source: dict[str, Any] = field(default_factory=dict)

    def as_dict(self) -> dict[str, Any]:
        return {"key": self.key, "p_detect": self.p_detect, "p_label": self.p_label,
                "false_det_per_road_snapshot": self.false_det, "source": self.source}


def observation_key(ocfg: dict[str, Any] | None) -> str:
    ocfg = ocfg or {"level": "perfect"}
    level = ocfg.get("level")
    if level not in LEVELS:
        raise ValueError(f"observation.level must be one of {LEVELS}, got {level!r}")
    only = ocfg.get("only")
    if only is not None and (level not in ("junction", "crowded") or only not in COMPONENTS):
        raise ValueError(f"observation.only must be one of {COMPONENTS} on a camera level,
 got {only!r} on {level!r}")
    if level == "uniform":
        p = float(ocfg["p_detect"])
        if not 0 < p <= 1:
            raise ValueError(f"observation.p_detect must be in (0, 1], got {p}")
        return f"uniform_p{p:.2f}"
    return f"{level}_only_{only}" if only else level


def repo_relative(path: Path) -> str:
    """Path as recorded in result files: relative to the repo root when it lies inside it."""
    try:
        return str(path.resolve().relative_to(repo_root()))
    except ValueError:
        return str(path)


def noise_params(ocfg: dict[str, Any], error_model_path: Path) -> NoiseParams:
    """Parameters of a noisy level. junction: detection rates of medium + large objects
    (pooled over their ground truth); crowded: overall rates; both with the measured label
    probabilities and false detections. uniform: p_detect for every class, exact labels, no
    false detections."""
    key = observation_key(ocfg)
    if ocfg["level"] == "uniform":
        p = float(ocfg["p_detect"])
        return NoiseParams(key, {c: p for c in DETECTOR_CLASSES}, {c: {c: 1.0} for c in
 DETECTOR_CLASSES},
                           {c: 0.0 for c in COUNTED}, {"kind": "uniform", "p_detect": p})
    em = json.loads(Path(error_model_path).read_text())
    if list(em["classes"]) != list(DETECTOR_CLASSES):
        raise ValueError(f"{error_model_path}: classes {em['classes']} !=
 {DETECTOR_CLASSES}")
    if ocfg["level"] == "crowded":
        p_detect = {c: float(em["p_detect"][c]) for c in DETECTOR_CLASSES}
```

```
    else:
        p_detect = {}
        for c in DETECTOR_CLASSES:
            s = em["miss_rate_by_size"][c]
            gt = s["medium"]["gt"] + s["large"]["gt"]
            miss = s["medium"]["miss_rate"] * s["medium"]["gt"] + s["large"]["miss_rate"] *
s["large"]["gt"]
            p_detect[c] = round(1 - miss / gt, 4)
    p_label = {c: {j: float(v) for j, v in em["p_label_given_detected"][c].items() if v > 0}
for c in DETECTOR_CLASSES}
    for c, row in p_label.items():
        if abs(sum(row.values()) - 1) > 0.01:
            raise ValueError(f"{error_model_path}: label probabilities of {c} sum to
{sum(row.values())}")
    false_det = {c: float(em["false_det_per_image"][c]["value"]) for c in COUNTED}
    only = ocfg.get("only")
    if only in ("false", "confusion"):
        p_detect = {c: 1.0 for c in DETECTOR_CLASSES}
    if only in ("misses", "false"):
        p_label = {c: {c: 1.0} for c in DETECTOR_CLASSES}
    if only in ("misses", "confusion"):
        false_det = {c: 0.0 for c in COUNTED}
    sha = hashlib.sha256(Path(error_model_path).read_bytes()).hexdigest()
    source = {"kind": "error_model", "file": repo_relative(Path(error_model_path)), "sha256":
sha,
              "conf_threshold": em["conf_threshold"],
              "p_detect_from": "medium+large objects" if ocfg["level"] == "junction" else
"all objects"}
    if only:
        source["only"] = only
    return NoiseParams(key, p_detect, p_label, false_det, source)


class LaneView:
    """Lane counts as a controller sees them at one step."""

    def __init__(self, waiting: dict[str, int], count: dict[str, int], types: dict[str,
collections.Counter]):
        self.waiting, self.count, self._types = waiting, count, types

    def type_counts(self, lanes: list[str], order: list[str]) -> dict[str, int]:
        """Vehicle-type counts over `lanes`, in `order` (unknown names after it), zeros left
out."""
        c: collections.Counter = collections.Counter()
        for l in lanes:
            c.update(self._types.get(l, {}))
        names = list(order) + sorted(n for n in c if n not in order)
        return {n: c[n] for n in names if c[n]}


class ExactLaneView(LaneView):
    """The engine's own counts; type counts straight from the simulator's vehicle types."""

    def __init__(self, env: "TrafficEnv"):
        eng = env.eng
        super().__init__(eng.get_lane_waiting_vehicle_count(), eng.get_lane_vehicle_count(),
{})
        self._lane_veh, self._env = eng.get_lane_vehicles(), env

    def type_counts(self, lanes: list[str], order: list[str]) -> dict[str, int]:
        return self._env.vehicle_types.count([v for l in lanes for v in self._lane_veh[l]])


class ExactObserver:
    key = "perfect"

    def __init__(self, env: "TrafficEnv"):
        self.env = env

    def view(self) -> LaneView:
        return ExactLaneView(self.env)
```

```
177
178     def summary(self) -> dict[str, Any]:
179         return {"key": self.key, "noised": False}
180
181     def write_artifacts(self, out_dir: Path) -> None:
182         pass
183
184
185 class NoisyObserver:
186     def __init__(self, env: "TrafficEnv", params: NoiseParams, run_seed: int):
187         self.env, self.params = env, params
188         self.key = params.key
189         self.seed_entropy = [int(run_seed), zlib.crc32(params.key.encode())]
190         self.rng = np.random.default_rng(np.random.SeedSequence(self.seed_entropy))
191         self.exempt = set(env.emergency_types)
192         lanes = sorted(env.eng.get_lane_vehicle_count())
193         self.roads: dict[str, list[str]] = {}
194         for l in lanes:
195             self.roads.setdefault(l.rsplit("_", 1)[0], []).append(l)
196         self._cache_t: float | None = None
197         self._cache: LaneView | None = None
198         self.log: list[dict[str, Any]] = []
199         self._labels = {c: (list(r), np.array(list(r.values())) / sum(r.values())) for c, r
    in params.p_label.items()}
200
201     def view(self) -> LaneView:
202         t = self.env.time
203         if self._cache_t != t:
204             self._cache, self._cache_t = self._draw(), t
205         return self._cache
206
207     def _draw(self) -> LaneView:
208         eng, vt, p = self.env.eng, self.env.vehicle_types, self.params
209         lane_veh = eng.get_lane_vehicles()
210         speed = eng.get_vehicle_speed()
211         waiting: dict[str, int] = {}
212         count: dict[str, int] = {}
213         types: dict[str, collections.Counter] = {}
214         true_n: collections.Counter = collections.Counter()
215         detected: collections.Counter = collections.Counter()
216         uncounted_label: collections.Counter = collections.Counter()
217         phantoms: collections.Counter = collections.Counter()
218         for road, lanes in self.roads.items():
219             for l in lanes:
220                 w = n = 0
221                 tc: collections.Counter = collections.Counter()
222                 for v in lane_veh[l]:
223                     if v.endswith("_shadow"):
224                         continue
225                     t = vt.of(v)
226                     if t in self.exempt:
227                         label = t
228                     else:
229                         if t not in SIM_TO_DETECTOR:
230                             raise KeyError(f"Vehicle type {t!r} has no detector class
    (SIM_TO_DETECTOR)")
231                         c = SIM_TO_DETECTOR[t]
232                         true_n[c] += 1
233                         if self.rng.random() >= p.p_detect[c]:
234                             continue
235                         detected[c] += 1
236                         names, probs = self._labels[c]
237                         j = names[int(self.rng.choice(len(names), p=probs))] if len(names) >
    1 else names[0]
238                         if j not in COUNTED:
239                             uncounted_label[c] += 1
240                             continue
241                         label = DETECTOR_TO_NAME[j]
242                     n += 1
243                     w += speed[v] < STOP_SPEED
244                     tc[label] += 1
```

```
                waiting[l], count[l], types[l] = w, n, tc
            for j in COUNTED:
                lam = p.false_det.get(j, 0.0)
                if lam <= 0:
                    continue
                for _ in range(int(self.rng.poisson(lam))):
                    l = lanes[int(self.rng.integers(len(lanes)))]
                    waiting[l] += 1
                    count[l] += 1
                    types[l][DETECTOR_TO_NAME[j]] += 1
                    phantoms[j] += 1
        self.log.append({"t": int(self.env.time), "true": dict(true_n), "detected": dict(detected),
                         "detected_uncounted_label": dict(uncounted_label), "false_added": dict(phantoms)})
        return LaneView(waiting, count, types)

    def summary(self) -> dict[str, Any]:
        tot = {k: collections.Counter() for k in ("true", "detected", "detected_uncounted_label", "false_added")}
        for rec in self.log:
            for k in tot:
                tot[k].update(rec[k])
        snaps = len(self.log)
        return {
            "key": self.key, "noised": True, "params": self.params.as_dict(),
            "seed_entropy": self.seed_entropy, "rng": "numpy default_rng(SeedSequence(seed_entropy))",
            "emergency_types_not_noised": sorted(self.exempt),
            "snapshots": snaps, "roads": len(self.roads),
            "realized": {
                "true_vehicle_observations": dict(tot["true"]),
                "detection_rate": {c: round(tot["detected"][c] / n, 4) for c, n in tot["true"].items() if n},
                "lost_to_non_vehicle_label": dict(tot["detected_uncounted_label"]),
                "false_per_road_snapshot": {j: round(tot["false_added"][j] / (snaps * len(self.roads)), 4)
                                            for j in COUNTED} if snaps else {},
            },
            "log": "noise.jsonl",
        }

    def write_artifacts(self, out_dir: Path) -> None:
        with (out_dir / "noise.jsonl").open("w", encoding="utf-8") as f:
            for rec in self.log:
                f.write(json.dumps(rec) + "\n")


def make_observer(env: "TrafficEnv", ocfg: dict[str, Any] | None, error_model_path: Path | None, run_seed: int):
    key = observation_key(ocfg)
    if key == "perfect":
        return ExactObserver(env)
    if ocfg["level"] != "uniform" and (error_model_path is None or not Path(error_model_path).is_file()):
        raise FileNotFoundError(f"Camera error model not found: {error_model_path} (scripts/eval_detector.py)")
    return NoisyObserver(env, noise_params(ocfg, error_model_path), run_seed)
```

6.6.10 `src/trafficsense/perception/errors.py`

Detector error statistics for the camera-error model (docs/perception.md section 7). 191 lines.

```
"""Detector error statistics for the camera-error model (docs/perception.md section 7).

Input: per image, the detections (xyxy, conf, class) at a low confidence threshold and the
```

```
ground-truth boxes, all in the pixels of the image the detector saw (our 640 copies).

Matching, once per image and independent of the operating threshold: detections in order of
falling confidence, each matched to the unmatched ground-truth box with the highest IoU,
if that IoU is >= 0.5, ignoring class. Because matching is greedy in confidence order,
raising the threshold only drops the tail: a ground-truth box is detected at threshold t
exactly when its matched detection has conf >= t. So matching is done once and every
threshold is evaluated from the stored matches.

At a threshold, per ground-truth box: matched with the same class = true positive; matched
with another class = confusion; unmatched = miss. Per detection: unmatched = false
detection. miss rate = misses / GT, so 1 - recall = miss rate + confusion rate.
"""
from __future__ import annotations

import warnings
from dataclasses import dataclass, field

import numpy as np

SIZE_BUCKETS = ("small", "medium", "large")   # COCO: area < 32^2, < 96^2, >= 96^2 pixels


def box_iou(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """IoU matrix between xyxy boxes a (N, 4) and b (M, 4)."""
    tl = np.maximum(a[:, None, :2], b[None, :, :2])
    br = np.minimum(a[:, None, 2:], b[None, :, 2:])
    inter = np.prod(np.clip(br - tl, 0, None), axis=2)
    area_a = np.prod(a[:, 2:] - a[:, :2], axis=1)
    area_b = np.prod(b[:, 2:] - b[:, :2], axis=1)
    return inter / np.maximum(area_a[:, None] + area_b[None, :] - inter, 1e-9)


def size_bucket(xyxy: np.ndarray) -> np.ndarray:
    """0 small, 1 medium, 2 large, from box area in pixels."""
    area = np.prod(xyxy[:, 2:] - xyxy[:, :2], axis=1)
    return np.digitize(area, [32 ** 2, 96 ** 2])


def greedy_match(det_xyxy: np.ndarray, det_conf: np.ndarray, gt_xyxy: np.ndarray,
                 iou_thr: float = 0.5) -> tuple[np.ndarray, np.ndarray]:
    """Class-agnostic greedy matching. Returns (gt index per detection or -1, det index per
     GT or -1)."""
    det_gt = np.full(len(det_xyxy), -1, dtype=np.int64)
    gt_det = np.full(len(gt_xyxy), -1, dtype=np.int64)
    if len(det_xyxy) == 0 or len(gt_xyxy) == 0:
        return det_gt, gt_det
    iou = box_iou(det_xyxy, gt_xyxy)
    for d in np.argsort(-det_conf, kind="stable"):
        cand = np.where(gt_det < 0, iou[d], -1.0)
        g = int(np.argmax(cand))
        if cand[g] >= iou_thr:
            det_gt[d], gt_det[g] = g, d
    return det_gt, gt_det


@dataclass
class Matched:
    """All images of a split, flattened, with their matches (built by `match_split`)."""
    n_classes: int
    image_group: np.ndarray          # per image: index into `groups` (sequence, for the
     bootstrap)
    groups: list[str]
    image_camera: np.ndarray         # per image: index into `cameras`
    cameras: list[str]
    det_image: np.ndarray = field(repr=False)
    det_cls: np.ndarray = field(repr=False)
    det_conf: np.ndarray = field(repr=False)
    det_matched: np.ndarray = field(repr=False)   # bool: matched to some GT (at the matching
     threshold)
    det_size: np.ndarray = field(repr=False)
```

```
    gt_image: np.ndarray = field(repr=False)
    gt_cls: np.ndarray = field(repr=False)
    gt_size: np.ndarray = field(repr=False)
    gt_area: np.ndarray = field(repr=False)        # box area in pixels
    gt_match_conf: np.ndarray = field(repr=False)  # conf of the matched detection, -1 if none
    gt_match_cls: np.ndarray = field(repr=False)   # class of the matched detection, -1 if none

    @property
    def n_images(self) -> int:
        return len(self.image_group)


def match_split(images: list[dict], n_classes: int, iou_thr: float = 0.5) -> Matched:
    """images: dicts with group, camera, det_xyxy, det_conf, det_cls, gt_xyxy, gt_cls (numpy arrays)."""
    groups = sorted({im["group"] for im in images})
    cameras = sorted({im["camera"] for im in images})
    gi, ci = {g: i for i, g in enumerate(groups)}, {c: i for i, c in enumerate(cameras)}
    cols: dict[str, list] = {k: [] for k in ("det_image", "det_cls", "det_conf", "det_matched", "det_size",
                                             "gt_image", "gt_cls", "gt_size", "gt_area", "gt_match_conf", "gt_match_cls")}
    for i, im in enumerate(images):
        det_gt, gt_det = greedy_match(im["det_xyxy"], im["det_conf"], im["gt_xyxy"], iou_thr)
        nd, ng = len(im["det_conf"]), len(im["gt_cls"])
        cols["det_image"].append(np.full(nd, i))
        cols["det_cls"].append(im["det_cls"].astype(np.int64))
        cols["det_conf"].append(im["det_conf"].astype(np.float64))
        cols["det_matched"].append(det_gt >= 0)
        cols["det_size"].append(size_bucket(im["det_xyxy"]) if nd else np.zeros(0, np.int64))
        cols["gt_image"].append(np.full(ng, i))
        cols["gt_cls"].append(im["gt_cls"].astype(np.int64))
        cols["gt_size"].append(size_bucket(im["gt_xyxy"]) if ng else np.zeros(0, np.int64))
        cols["gt_area"].append(np.prod(im["gt_xyxy"][:, 2:] - im["gt_xyxy"][:, :2], axis=1) if ng else np.zeros(0))
        hit = gt_det >= 0
        cols["gt_match_conf"].append(np.where(hit, im["det_conf"][np.maximum(gt_det, 0)] if nd else -1.0, -1.0))
        cols["gt_match_cls"].append(np.where(hit, im["det_cls"][np.maximum(gt_det, 0)] if nd else -1, -1)
                                    .astype(np.int64))
    arr = {k: np.concatenate(v) if v else np.zeros(0) for k, v in cols.items()}
    return Matched(n_classes, np.array([gi[im["group"]] for im in images]), groups,
                   np.array([ci[im["camera"]] for im in images]), cameras, **arr)


def counts(m: Matched, thr: float, image_mask: np.ndarray | None = None) -> dict[str, np.ndarray]:
    """Raw counts at threshold `thr`, optionally restricted to a subset of images."""
    k = m.n_classes
    gsel = np.ones(len(m.gt_cls), bool) if image_mask is None else image_mask[m.gt_image]
    dsel = m.det_conf >= thr
    if image_mask is not None:
        dsel &= image_mask[m.det_image]
    detected = gsel & (m.gt_match_conf >= thr)
    conf_mat = np.zeros((k, k), np.int64)
    np.add.at(conf_mat, (m.gt_cls[detected], m.gt_match_cls[detected]), 1)
    fp = dsel & ~m.det_matched
    gt_size = np.zeros((k, 3), np.int64)
    miss_size = np.zeros((k, 3), np.int64)
    np.add.at(gt_size, (m.gt_cls[gsel], m.gt_size[gsel]), 1)
    missed = gsel & ~detected
    np.add.at(miss_size, (m.gt_cls[missed], m.gt_size[missed]), 1)
    return {
        "images": np.int64(m.n_images if image_mask is None else image_mask.sum()),
        "gt": np.bincount(m.gt_cls[gsel], minlength=k),
        "miss": np.bincount(m.gt_cls[missed], minlength=k),
        "confusion": conf_mat,                                  # rows true class, cols predicted
```

```
        "det": np.bincount(m.det_cls[dsel], minlength=k),
        "false_det": np.bincount(m.det_cls[fp], minlength=k),
        "gt_by_size": gt_size, "miss_by_size": miss_size,
    }


def rates(c: dict[str, np.ndarray]) -> dict[str, np.ndarray]:
    """Per-class rates from `counts` (NaN where the denominator is 0)."""
    with np.errstate(divide="ignore", invalid="ignore"):
        tp = np.diag(c["confusion"])
        return {
            "miss_rate": c["miss"] / c["gt"],
            "recall": tp / c["gt"],
            "precision": tp / c["det"],
            "false_det_per_image": c["false_det"] / c["images"],
            "miss_rate_by_size": c["miss_by_size"] / c["gt_by_size"],
        }


def f1_per_class(m: Matched, thr: float) -> np.ndarray:
    c = counts(m, thr)
    tp = np.diag(c["confusion"])
    fp, fn = c["det"] - tp, c["gt"] - tp
    with np.errstate(divide="ignore", invalid="ignore"):
        return np.where(2 * tp + fp + fn > 0, 2 * tp / (2 * tp + fp + fn), 0.0)


def choose_threshold(m: Matched, grid: np.ndarray) -> tuple[float, np.ndarray]:
    """Threshold on `grid` that maximises the mean F1 over classes (first one on ties); and the curve."""
    curve = np.array([f1_per_class(m, float(t)).mean() for t in grid])
    return float(grid[int(np.argmax(curve))]), curve


def count_error(m: Matched, thr: float) -> dict[str, np.ndarray]:
    """Per class, over images: mean |detected count - true count| and mean signed error."""
    k, n = m.n_classes, m.n_images
    pred = np.zeros((n, k), np.int64)
    true = np.zeros((n, k), np.int64)
    d = m.det_conf >= thr
    np.add.at(pred, (m.det_image[d], m.det_cls[d]), 1)
    np.add.at(true, (m.gt_image, m.gt_cls), 1)
    return {"mean_abs": np.abs(pred - true).mean(axis=0), "mean_signed": (pred - true).mean(axis=0)}


def bootstrap(m: Matched, thr: float, reps: int = 1000, seed: int = 0) -> dict[str, np.ndarray]:
    """95% percentile intervals of the per-class rates, resampling whole groups (sequences)."""
    per_group = [counts(m, thr, m.image_group == g) for g in range(len(m.groups))]
    keys = ("images", "gt", "miss", "confusion", "det", "false_det", "gt_by_size", "miss_by_size")
    stacked = {key: np.stack([pg[key] for pg in per_group]) for key in keys}
    rng = np.random.default_rng(seed)
    draws: dict[str, list] = {}
    for _ in range(reps):
        pick = rng.integers(0, len(per_group), len(per_group))
        r = rates({key: stacked[key][pick].sum(axis=0) for key in keys})
        for key, v in r.items():
            draws.setdefault(key, []).append(v)
    with warnings.catch_warnings():   # classes or size buckets absent from a split stay NaN
        warnings.simplefilter("ignore", RuntimeWarning)
        return {key: np.nanpercentile(np.stack(v), [2.5, 97.5], axis=0) for key, v in draws.items()}
```

6.6.11 Other source files

The remaining source files, by path, with their length and the first line of their own description (module docstring or header comment). Tests are in `tests/`, configuration in `configs/`.

Table 6.7 Source files not listed in full, in commit `1d0cfca4a5`

File	Lines	Description
scripts/analyze_llm_decisions.py	138	What the LLM controllers chose: replay each benchmark run from its decisions.jsonl and score
scripts/bench_image_io.py	136	Measure how fast training images can be read from their storage (CPU only, GPU hidden)
scripts/benchmark_llm.py	424	Benchmark local LLMs as the signal controller, then pick a model by a fixed rule
scripts/build_replays.py	43	Regenerate the Network Replay page's data from the committed runs, in one command
scripts/build_report.py	183	Build the project report, in the college capstone thesis format, into docs/report/report.docx
scripts/calibrate_indian_demand.py	259	Calibrate the Indian 2x2 demand levels against measured capacity, under MaxPressure
scripts/check_report_format.py	403	Check the built report (docs/report/report.docx and its PDF) against the college format in
scripts/eval_detector.py	280	Evaluate the trained detector and build the camera error model (docs/perception.md section 7)
scripts/fetch_collmlight_data.sh	37	Fetch CoLLMLight roadnets/flows (Synthetic, plus Jinan and Hangzhou if present)
scripts/generate_indian_flow.py	58	Generate Indian 2x2 flow files: one per demand level and seed
scripts/generate_indian_roadnet.py	57	Generate the Indian 2x2 roadnet from configs/scenarios/indian_2x2.yaml
scripts/make_baseline_tables.py	238	Generate the tables in docs/baselines.md from the baseline result files
scripts/make_camera_error.py	544	Generate the camera-error results (docs/camera_error.md tables + chart) from the runs
scripts/make_indian_tables.py	360	Aggregate the Indian 2x2 runs and generate the tables in docs/indian_scenario.md
scripts/make_model_size.py	617	Tables and chart of docs/model_size.md, from the run folders
scripts/make_paper.py	130	Keep every number in the IEEE paper (docs/paper/paper.tex) equal to the data, and build it
scripts/make_perception_results.py	128	Generate the detector result tables of docs/perception.md from the evaluation files
scripts/make_replays.py	111	Make the replays of the dashboard's Network Replay page (pre-registered in docs/dashboard.md)
scripts/make_report.py	92	Keep every number in README.md and the report (docs/report/*.md) equal to the data
scripts/make_reproducibility.py	211	Generate the tables in docs/reproducibility.md from the fresh-clone check's record
scripts/make_results.py	262	Generate the tables of docs/results.md (and the full-run tables of docs/emergency.md) from the
scripts/make_sample_clip.py	106	Make the Live Detection page's sample clip from IDD frames of OUR TEST split (official IDD val),
scripts/prepare_idd.py	77	Convert IDD Detection to YOLO labels (docs/perception.md, sections 2-4 and 5)
scripts/probe_hosted.py	154	Probe the hosted model of the model-size study (NVIDIA NIM) before any run

File	Lines	Description
scripts/render_idd_samples.py	84	Draw converted YOLO boxes on random IDD images: proof the labels are real and line up
scripts/resize_idd.py	99	Make a resized copy of the IDD images used by our splits (docs/perception.md section 6)
scripts/run_baselines.py	40	Run the baseline controllers on the CoLLMLight real-world datasets
scripts/run_camera_decomposition.py	86	Camera-error experiment, POST-HOC decomposition (added after the results; not pre-registered)
scripts/run_camera_error.py	175	Camera-error experiment (Phase 4c): every run of the pre-registered plan, resumable
scripts/run_emergency_experiments.py	204	Emergency-vehicle full experiments: design and report
scripts/run_emergency_smoke.py	205	Emergency-vehicle smoke tests (Phase 3b): short runs that check the ambulance extension
scripts/run_experiment.py	27	Run one controller on one scenario. Writes outputs/runs/_s/
scripts/run_full_experiments.py	187	Full experiments (Phase 3c): every run of the pre-registered plan, resumable
scripts/run_indian_experiments.py	41	Run the Indian 2x2 baseline experiments: controllers x demand levels x seeds
scripts/run_model_size.py	273	Model size vs control quality: an EXPLORATORY study, added after the main results
scripts/screenshot_dashboard.py	206	Launch the dashboard and screenshot every page and state with Playwright (Chromium)
scripts/start_llm_server.sh	22	Start Ollama server for CoLLMLight integration
scripts/train_detector.py	231	Train the YOLOv8 detector on our IDD splits (docs/perception.md section 6)
scripts/tune_maxpressure.py	132	Tune MaxPressure's decision interval: sweep it, then pick one setting by a fixed rule
scripts/tune_maxpressure_min_green.py	139	Tune MaxPressure's minimum green with the decision grid fixed at 5 s; pick one by a fixed rule
scripts/validation/collmlight_metrics.py	99	CoLLMLight's metric definitions, reimplemented on top of our env for the cross-check
scripts/validation/ diagnose_indian_breakdown.py	109	Where does the Indian 2x2 network break down under MaxPressure? Diagnostic for the doc
scripts/validation/flow_demand.py	105	Demand analysis of a CityFlow flow file, and the capacity of the busiest entry lane
scripts/validation/make_tables.py	212	Generate the tables in docs/validation.md from outputs/validation/*.json
scripts/validation/run_collmlight_baseline.py	190	Run CoLLMLight's own FixedTime / MaxPressure code, unmodified, and record its results
scripts/validation/run_matched.py	102	Run our env + controllers with CoLLMLight-matched (or default) settings and score the run
src/trafficsense/config.py	41	Config loading. All paths come from YAML and resolve relative to the repo root
src/trafficsense/control/fixedtime.py	64	Fixed-time control: cycle through a fixed list of green phases in index order
src/trafficsense/control/webster.py	188	Webster-tuned fixed-time control (Webster, 1958, Road Research Technical Paper 39)
src/trafficsense/dashboard/app.py	44	TrafficSense dashboard. Run from the repo root:
src/trafficsense/dashboard/data.py	165	Every file the dashboard shows comes through here
src/trafficsense/dashboard/screens/ live_detection.py	3	(no description)
src/trafficsense/dashboard/screens/ network_replay.py	3	(no description)
src/trafficsense/dashboard/screens/ overview.py	3	(no description)
src/trafficsense/dashboard/screens/results.py	3	(no description)

File	Lines	Description
src/trafficsense/dashboard/theme.py	175	Design tokens of the dashboard: colours, type, the shared Plotly template and the global CSS
src/trafficsense/dashboard/ui.py	203	UI building blocks shared by the pages: header, badges, source chips, chart cards, banners, tables
src/trafficsense/dashboard/views/detection.py	269	Live Detection: run the IDD-trained YOLOv8n on a video and show what it sees
src/trafficsense/dashboard/views/overview.py	275	Overview: the headline result, three stat tiles, the story in three steps and the architecture
src/trafficsense/dashboard/views/replay.py	291	Network Replay: two saved runs of the 2x2 grid side by side, drawn on a canvas
src/trafficsense/dashboard/views/results.py	612	Results: one tab per study, each with a takeaway line computed from the data, the charts and
src/trafficsense/experiment.py	161	Run one controller on one scenario and write summary.json + metrics.csv with provenance
src/trafficsense/llm/gpu.py	60	GPU checks for LLM runs (nvidia-smi): the LLM must have the GPU to itself (CLAUDE.md:
src/trafficsense/llm/hosted.py	260	Hosted chat client: an OpenAI-compatible /chat/completions API (NVIDIA NIM)
src/trafficsense/perception/detector.py	205	YOLO detector training helpers (docs/perception.md section 6)
src/trafficsense/perception/idd.py	210	IDD Detection (Pascal VOC) -> YOLO labels, following docs/perception.md sections 2-4
src/trafficsense/report/facts.py	436	Named numbers for the README and the report, each read from a file in outputs/ (or a config)
src/trafficsense/report/figures.py	565	Figures of the report, drawn from outputs/ into docs/report/figures/
src/trafficsense/report/numbers.py	249	Check (and refresh) every number in a Markdown document against the data
src/trafficsense/report/paper.py	398	Check (and refresh) every number in the IEEE paper (docs/paper/paper.tex) against the data
src/trafficsense/report/paper_figures.py	230	Figures of the IEEE paper, drawn from outputs/ into docs/paper/figures/ as vector PDFs
src/trafficsense/report/prompts.py	81	The prompts shown in the report's appendix, rendered by the controllers' own code
src/trafficsense/report/tables.py	564	Generated tables of README.md and the report. Each function returns Markdown for one
src/trafficsense/report/thesis.py	603	Build the project report as a Word file in the college capstone thesis format, and its PDF
src/trafficsense/selection.py	39	The pre-registered rule used to pick one baseline setting from a sweep
src/trafficsense/sim/capacity.py	278	Lane capacity and lane demand, measured on the simulator
src/trafficsense/sim/demand.py	188	Seeded traffic demand for generated scenarios (Indian 2x2)
src/trafficsense/sim/emergency.py	164	Emergency vehicles (ambulances): where they are, and what happens to them
src/trafficsense/sim/metrics.py	135	Metrics measured from the simulator, as defined in CLAUDE.md
src/trafficsense/sim/replay.py	268	Replay a saved run with extra logging, for the dashboard's Network Replay page
src/trafficsense/sim/roadnet.py	202	Parse a CityFlow roadnet into per-intersection lanes, movements, phases and neighbours
src/trafficsense/sim/vehicles.py	60	Vehicle type of each simulated vehicle, from the flow file

REFERENCES

[1] F. V. Webster, ‘Traffic signal settings’, Road Research Laboratory, Her Majesty’s Stationery Office, London, Road Research Technical Paper 39, 1958.

[2] A. G. Sims and K. W. Dobinson, ‘The Sydney coordinated adaptive traffic (SCAT) system philosophy and benefits’, *IEEE Transactions on Vehicular Technology*, vol. 29, no. 2, pp. 130–137, 1980, doi: 10.1109/T-VT.1980.23833.

[3] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton, ‘SCOOT – a traffic responsive method of coordinating signals’, Transport; Road Research Laboratory, Crowthorne, TRRL Laboratory Report 1014, 1981.

[4] P. Varaiya, ‘Max pressure control of a network of signalized intersections’, *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 177–195, 2013, doi: 10.1016/j.trc.2013.08.014.

[5] H. Wei, G. Zheng, V. Gayah, and Z. Li, ‘Recent advances in reinforcement learning for traffic signal control: A survey of models and evaluation’, *ACM SIGKDD Explorations Newsletter*, vol. 22, no. 2, pp. 12–18, 2021, doi: 10.1145/3447556.3447565.

[6] S. Lai, Z. Xu, W. Zhang, H. Liu, and H. Xiong, ‘LLMLight: Large language models as traffic signal control agents’. arXiv preprint arXiv:2312.16044, 2023. Available: https://arxiv.org/abs/2312.16044

[7] M. Wang, A. Pang, Y. Kan, M.-O. Pun, C. S. Chen, and B. Huang, ‘LLM-assisted light: Leveraging large language model capabilities for human-mimetic traffic signal control in complex urban environments’. arXiv preprint arXiv:2403.08337, 2024. Available: https://arxiv.org/abs/2403.08337

[8] Z. Yuan, S. Lai, and H. Liu, ‘CoLLMLight: Cooperative large language model agents for network-wide traffic signal control’, in *Proceedings of the international conference on learning representations (ICLR)*, 2026. Available: https://openreview.net/forum?id=KeJqoEVOeY

[9] Innovative Transport Solutions (iTrans) Pvt. Ltd., ‘Two- and three-wheelers in India’, Indian Institute of Technology Delhi, for the International Council on Clean Transportation (ICCT); the Institute for Transportation; Development Policy (ITDP), Final report, 2009.

[10] S. Chandra and U. Kumar, ‘Effect of lane width on capacity under mixed traffic conditions in India’, *Journal of Transportation Engineering*, vol. 129, no. 2, pp. 155–160, 2003, doi: 10.1061/(ASCE)0733-947X(2003)129:2(155).

[11] CSIR-Central Road Research Institute, *Indian highway capacity manual (indo-HCM)*. New Delhi: CSIR-Central Road Research Institute, 2017.

[12] T. V. Mathew and P. Radhakrishnan, ‘Calibration of microsimulation models for nonlane-based heterogeneous traffic at signalized intersections’, *Journal of Urban Planning and Development*, vol. 136, no. 1, pp. 59–66, 2010, doi: 10.1061/(ASCE)0733-9488(2010)136:1(59).

[13] H. Zhang *et al.*, 'CityFlow: A multi-agent reinforcement learning environment for large scale city traffic scenario', in *The world wide web conference (WWW '19)*, 2019, pp. 3620–3624. doi: 10.1145/3308558.3314139.

[14] P. Koonce *et al.*, 'Traffic signal timing manual', Federal Highway Administration, U.S. Department of Transportation, FHWA-HOP-08-024, 2008.

[15] L. Tassiulas and A. Ephremides, 'Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks', *IEEE Transactions on Automatic Control*, vol. 37, no. 12, pp. 1936–1948, 1992, doi: 10.1109/9.182479.

[16] B. Abdulhai, R. Pringle, and G. J. Karakoulas, 'Reinforcement learning for true adaptive traffic signal control', *Journal of Transportation Engineering*, vol. 129, no. 3, pp. 278–285, 2003, doi: 10.1061/(ASCE)0733-947X(2003)129:3(278).

[17] W. Genders and S. Razavi, 'Using a deep reinforcement learning agent for traffic signal control'. arXiv preprint arXiv:1611.01142, 2016. Available: https://arxiv.org/abs/1611.01142

[18] H. Wei, G. Zheng, H. Yao, and Z. Li, 'IntelliLight: A reinforcement learning approach for intelligent traffic light control', in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 2496–2505. doi: 10.1145/3219819.3220096.

[19] H. Wei *et al.*, 'PressLight: Learning max pressure control to coordinate traffic signals in arterial network', in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 1290–1298. doi: 10.1145/3292500.3330949.

[20] H. Wei *et al.*, 'CoLight: Learning network-level cooperation for traffic signal control', in *Proceedings of the 28th ACM international conference on information and knowledge management (CIKM)*, 2019, pp. 1913–1922. doi: 10.1145/3357384.3357902.

[21] C. Chen *et al.*, 'Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control', in *Proceedings of the AAAI conference on artificial intelligence*, 2020, pp. 3414–3421. doi: 10.1609/aaai.v34i04.5744.

[22] Ollama, *Ollama: Run large language models locally*. (2026). Available: https://github.com/ollama/ollama

[23] N. Buch, S. A. Velastin, and J. Orwell, 'A review of computer vision techniques for the analysis of urban traffic', *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 3, pp. 920–939, 2011, doi: 10.1109/TITS.2011.2119372.

[24] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, 'You only look once: Unified, real-time object detection', in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.

[25] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*. (2023). Available: https://github.com/ultralytics/ultralytics

[26] T.-Y. Lin *et al.*, ‘Microsoft COCO: Common objects in context’, in *Computer vision – ECCV 2014*, in Lecture notes in computer science, vol. 8693. Springer, 2014, pp. 740–755. doi: 10.1007/978-3-319-10602-1_48.

[27] K. Nellore and G. P. Hancke, ‘Traffic management for emergency vehicle priority based on visual sensing’, *Sensors*, vol. 16, no. 11, p. 1892, 2016, doi: 10.3390/s16111892.

[28] G. Varma, A. Subramanian, A. Namboodiri, M. Chandraker, and C. V. Jawahar, ‘IDD: A dataset for exploring problems of autonomous navigation in unconstrained environments’, in *IEEE winter conference on applications of computer vision (WACV)*, 2019, pp. 1743–1751. doi: 10.1109/WACV.2019.00190.

[29] Umapathi and R. Khatawkar, ‘Effect of carriageway width on PCU and capacity of two lane roads: A case study of Shivamogga district’, *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 6, no. 6, pp. 89–93, 2019.

[30] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, ‘The preregistration revolution’, *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018, doi: 10.1073/pnas.1708274114.

[31] A. Grattafiori *et al.*, ‘The Llama 3 herd of models’. arXiv preprint arXiv:2407.21783, 2024. Available: https://arxiv.org/abs/2407.21783

[32] Qwen Team, ‘Qwen2.5 technical report’. arXiv preprint arXiv:2412.15115, 2024. Available: https://arxiv.org/abs/2412.15115

[33] NVIDIA, ‘Nemotron 3 Ultra (nvidia/nemotron-3-ultra-550b-a55b) model card’. NVIDIA API catalog, build.nvidia.com, 2026.

BIODATA

Photo

Name : C Bhargav Kumar

Register No. : 23BCE8195

Mobile Number : _____

E-mail : bhargav.23bce8195@vitapstudent.ac.in

Permanent Address : _____

_____

_____

BIODATA

Photo

Name : Achalla Akul Lakhan

Register No. : 23BCE8015

Mobile Number : _____

E-mail : akul.23bce8015@vitapstudent.ac.in

Permanent Address : _____

_____

_____

BIODATA

Photo

Name : Darsi Amit

Register No. : 23BCE8288

Mobile Number : _____

E-mail : amit.23bce8288@vitapstudent.ac.in

Permanent Address : _____

_____

_____

BIODATA

Photo

Name : Mylapalli Abhishek Raj

Register No. : 23BCE9470

Mobile Number : _____

E-mail : abhishek.23bce9470@vitapstudent.ac.in

Permanent Address : _____

_____

_____

NOTE: It is MANDATORY for a student to attach all the PPT's, Sample Materials, Specification Sheets, Programming Codes and a 5-10 minutes demo Video of the Project Digitally In CD. Stick the Compact Disk (CD) in the final page of the Thesis after binding it.